Mục lục

Mục lục 1
LỜI MỞ ĐẦU 4
Chương 1: PHẦN MỀM VÀ KỸ NGHỆ PHÂN MỀM 1
1.1. Khái niệm về phần mềm 1
1.2. Sự phát triển của lĩnh vực kỹ nghệ phần mềm 2
1.3. Phân loại phần mềm 2
1.4. Kỹ nghệ phần mềm 3
1.5. Công nghệ phần mềm và web 7
1.6. Đạo đức ngành công nghệ phần mềm 8
1.7. Các quy trình công nghệ phần mềm 9
1.7.1. Mô hình thác nước 11
1.7.2. Mô hình tăng trưởng 13
1.7.3. Mô hình bản mẫu (prototyping) 15
1.7.4. Mô hình tái sử dụng 16
1.7.5. Quy trình RUP (Rational Unified Process) 17
1.7.6. Quản lý dự án phần mềm 22
TỔNG KẾT CUỐI CHƯƠNG 26
CÂU HỎI ÔN TẬP CHƯƠNG 1 28
Chương 2: KHẢO SÁT-PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU 29
2.1. Vai trò của phân tích và đặc tả yêu cầu 29
2.2. Phân loại yêu cầu 29
2.2.1. Yêu cầu chức năng 32
2.2.2. Yêu cầu phi chức năng 33
2.3. Tài liệu phân tích yêu cầu 36
2.4. Các hoạt động phân tích và đặc tả yêu cầu 37
2.4.1. Xác định tính khả thi 38

2.4.2. Thu thập và phân tích hệ thống 39
2.4.3. Chuyển đổi yêu cầu sang dạng chuẩn 42
2.4.4. Kiểm tra các yêu cầu 50
TỔNG KẾT CUỐI CHƯƠNG 51
CÂU HỎI ôn tập CHƯƠNG 2 52
Chương 3: THIẾT KẾ PHẦN MỀM 54
3.1. Đặc điểm của quá trình thiết kế phần mềm 54
3.2. Thiết kế kiến trúc hệ thống 54
3.3. Thiết kế các mô hình cấu trúc 59
3.3.1. Xác định các đối tượng 59
3.3.2. Xây dựng thẻ CRC và sơ đồ lớp 60
3.4. Sử dụng các sơ đồ thiết kế 63
3.5. Thiết kế dữ liệu 67
3.6. Thiết kế giao diện 71
3.6.1. Thiết kế giao diện đầu và thực đơn 74
3.6.2. Giao diện nhập liệu 76
3.6.3. Giao diện báo cáo 78
3.7. Thiết kế xử lý mô hình 3 lớp 79
TỔNG KẾT CUỐI CHƯƠNG 80
CÂU HỎI ÔN TẬP CHƯƠNG III 81
Chương 4: KIỂM TRA CHẤT LƯỢNG PHẦN MỀM 82
4.1. Độ tin cậy của phần mềm 82
4.2. Khái niệm kiểm tra và chiến lược kiểm tra phần mềm 84
4.2.1. Đặc điểm kiểm tra phần mềm 84
4.2.2. Chiến lược kiểm tra phần mềm 86
4.3. Kiểm thử và các công cụ trợ giúp kiểm thử 89
4.3.1. Khái niệm kiểm thử 89
4.3.2. Phân loại một số công cụ kiểm thử tự động 91

TỔNG KẾT CUỐI CHƯƠNG 93
CÂU HỎI ÔN TẬP CHƯƠNG IV 94
Chương 5: BẢO TRÌ PHẦN MỀM VÀ QUẢN LÝ THAY ĐỔI PHẦN MỀM 95
5.1. Hoạt động bảo trì phần mềm và phân loại 95
5.2. Đặc điểm của bảo trì phần mềm 96
5.2.1. Bảo trì có cấu trúc đối với bảo trì không cấu trúc 96
5.2.2. Chi phí bảo trì 97
5.2.3. Một số vấn đề khác 98
5.3. Công việc bảo trì phần mềm và một số hiệu ứng lề 99
5.3.1. Khả năng bảo trì 99
5.3.2. Các công việc bảo trì 100
5.3.3. Một số hiệu ứng lề của công việc bảo trì 102
5.4. Một số hình thức bảo trì phần mềm 104
5.4.1. Bảo trì mã chương trình xa lạ 104
5.4.2. Công nghệ phản hồi và công nghệ tái sử dụng 104
5.4.3. Bảo trì phòng ngừa 105
5.4.4. Chiến lược phần mềm thành phần 105
5.5. Quản lý thay đổi phần mềm 106
5.5.1. Các thủ tục quản lý thay đổi 106
5.5.2. Ghi quyết định theo thời gian 109
5.5.3. Quản lý thay đổi tài liệu 109
TỔNG KẾT CUỐI CHƯƠNG 109
CÂU HỏI ÔN TẬP CHƯƠNG V 110
TÀI LIỆU THAM KHẢO 111

LỜI MỞ ĐẦU

ĐỐI TƯỢNG SỬ DỤNG

Bài giảng này được thiết kế dành cho các sinh viên khi bắt đầu vào giai đoạn chuyên ngành sau khi sinh viên đã có kiến thức cơ bản về ngôn ngữ lập trình và cơ sở dữ liệu, đủ để có thể xây dựng một phần mềm vừa và nhỏ. Ngoài ra, các đối tượng người đọc khác muốn tìm hiểu sơ lược về quy trình phát triển một phần mềm từ điểm xuất phát ban đầu đến chuyển giao phần mềm cũng có thể tham khảo tập bài giảng này.

YÊU CẦU

Khi sử dụng bài giảng này, người học cần có các kiến thức cơ bản về lập trình, lập trình hướng đối tượng, cơ sở dữ liệu. Để thực hiện các yêu cầu của bài giảng, người học cần có chương trình hỗ trợ thiết kế UML như Microsoft Visio.

Thêm vào đó, yêu cầu của bài giảng khi kết thúc học phần mềm phải thiết kế và xây dựng được phần mềm nên người học tự lựa chọn và tự trang bị cho mình kiến thức về ngôn ngữ lập trình và công cụ lập trình như C#, Java, PHP, studio.net để tạo các ứng dụng.

CẤU TRÚC BÀI GIẢNG

Bài giảng gồm 5 chương:

chương 1: phần mềm và kỹ nghệ phần mềm

chương 2: khảo sát-phân tích và đặc tả yêu cầu

chương 3: thiết kế phần mềm

chương 4: kiểm tra chất lượng phần mềm

chương 5: bảo trì phần mềm và quản lý thay đổi phần mềm

tài liệu tham khảo

NỘI DUNG BÀI GIẢNG

Nội dung bài giảng đề cập đến khái niệm về kỹ nghệ phần mềm, quy trình phát triển phần mềm từ giai đoạn đầu đến giai đoạn chuyển giao phần mềm cho khách hàng.

Đầu tiên người học được trang bị kiến thức thế nào kỹ nghệ phần mềm, sự khác biệt giữa kỹ nghệ phần mềm và phát triển một phần mềm cá nhân, và các quy trình phát triển phần mềm.

Sau đó người học được cung cấp kiến thức chi tiết các giai đoạn cơ bản trong các quy trình, áp dụng phương pháp thiết kế hướng đối tượng mới nhất để cụ thể hóa các hoạt động. Các giai đoạn bao gồm: đặc tả, phân tích yêu cầu; thiết kế phần mềm, kiểm tra chất lượng và bảo trì phần mềm. Trong các giai đoạn đó, bài giảng chú trọng nhiều đến giai đoạn phân tích và thiết kế. Học viên được cung cấp kiến thức về lý thuyết phân tích thiết kế, rèn luyện kỹ năng sử dụng uml trong việc xây dựng các sơ đồ. Nội dung mang tính chất lý thuyết nhiều, tuy nhiên cũng xen nhiều ví dụ cụ thể để người học dễ hiểu hơn, cụ thể hơn.

ĐIỂM MỚI CỦA BÀI GIẢNG

Các bài giảng trước đây áp dụng phương pháp phân tích thiết kế hướng cấu trúc để minh họa các thao tác trong quy trình phát triển phần mềm. Đây là phương pháp cổ điển, đã và đang có đóng góp lớn vào ngành công nghệ phần mềm. Tuy nhiên, phương pháp phân tích hướng đối tượng là xu hướng mới và phù hợp với các hệ thống mới hiện nay trong khi phương pháp trước đây không còn đáp ứng được. Việc tiếp cận đến với phương pháp này là rất cần thiết để bắt kịp nhịp phát triển của ngành khoa học công nghệ phần mềm. Trong bài giảng này, tất cả các ví dụ minh họa, phương pháp phân tích, thiết kế đều sử dụng phương pháp này.

HƯỚNG DẪN SỬ DỤNG

Bài giảng được bố cục theo chương, đề mục, nội dung. Trong mỗi mục hoặc mục con của nội dung: kiến thức về nội dung và các yêu cầu sẽ được trình bày, tiếp theo là các ví dụ minh họa. Đầu mỗi chương có nêu phần kiến thức sẽ đề cập trong chương. Các mục được đánh số thứ tự dựa trên chương theo trình tự: thứ tự chương, thứ tự mục, thứ tự mục con. Hình ảnh được đánh số theo chương theo trình tự: thứ tự chương, thứ tự. Cuối mỗi chương sẽ là phần tổng kết để nhắc lại các kiến thức chính yếu đã trình bày trong chương và câu hỏi cũng cố.

Chương 1: PHẦN MỀM VÀ KỸ NGHỆ PHÂN MỀM

Rất nhiều người viết chương trình phục vụ nhu cầu cá nhân, nhưng đó không thuộc về ngành công nghệ phần mềm. Trong chương này, các khái niệm về kỹ nghệ phần mềm, phân loại phần mềm, các kiến thức liên quan đến quá trình phát triển phần mềm được đề cập một cách tổng quát nhằm mang lại một cái nhìn tổng quan về công nghệ phần mềm.

1.1. KHÁI NIỆM VỀ PHẦN MỀM

Nhiều người nghĩ rằng phần mềm chỉ đơn giản là một từ chỉ đến chương trình, có thể là chương trình tự viết hay chương trình được mua về sử dụng. Thực chất trong công nghệ phần mềm, phần mềm không chỉ là đơn thuần là chương trình, mà nó bao gồm tất cả các tài liệu, cở sở dữ liệu cần phải có để chương trình hoạt động chính xác.

Một hệ thống phần mềm chuyên nghiệp thường được phát triển bởi nhiều người. Hệ thống bao gồm một số các chương trình riêng biệt, các tập tin được sử dụng để thiết lập các chương trình này. Nó có thể bao gồm các tài liệu mô tả cấu trúc chương trình, và website cho người sử dụng download các thông tin sản phẩm phần mềm.

Định nghĩa phần mềm: Phần mềm là chương trình máy tính và các tài liệu liên quan. Các sản phẩm phần mềm có thể được thiết kế cho một khách hàng cụ thể nào đó hoặc cho thị trường nói chung.

Trong công nghệ phần mềm, phần mềm được chia làm 2 loại:

- **Phần mềm chung**: Được phát triển bởi một tổ chức độc lập và được bán trên thị trường với khách hàng có nhu cầu.
- **Phần mềm tùy biến**: Là phần mềm được phát triển cho một khách hàng cụ thể như các phần mềm cho một quy trình nghiệp vụ cụ thể (phần mềm quản lý doanh nghiệp) hoặc cho một thiết bị điện tử nào đó.

Một phần mềm tốt là phần mềm cung cấp các chức năng, hiệu năng yêu cầu cho người sử dụng. Nó có thể sử dụng được, đáng tin cậy và có thể bảo trì.

Các thuộc tính phần mềm:

- **Khả năng duy trì**: Phần mềm có thể đáp ứng được yêu cầu khi nhu cầu của khách hàng thay đổi.
- **Bảo mật và tin cậy**: Phần mềm nên bao gồm hệ thống đặc tính như tin cậy, đúng đắn, an toàn và bảo mật. Những phần mềm này phải không những không gây thiệt hại về mặt vật lý cũng như kinh tế khi hệ thống gặp lỗi mà người dùng "xấu" cũng khó có thể truy cập và gây hại cho hệ thống.
- **Hiệu quả**: Phần mềm không lãng phí tài nguyên như là vi xử lý, bộ nhớ. Do đó tính chất này thể hiện qua thời gian xử lý, lượng tiêu thụ bộ nhớ, khả năng

phản hồi.

- **Tương thích**: Phần mềm phải được chấp nhận bởi đúng người dùng mà phần mềm đã thiết kế cho, có thể hiểu và sử dụng được bởi đúng đối tượng và tương thích với các phần mềm mà họ đã dùng.

1.2. SỰ PHÁT TRIỂN CỦA LĨNH VỰC KỸ NGHỆ PHẦN MỀM

Công nghệ phần mềm có một lịch sử phát triển lâu dài. Từ năm 1940, các phần mềm đầu tiên được viết bằng tay. Đến những năm 1950 ra đời phần mềm biên dịch Macro Assembler và phần mềm thông dịch đã được tạo ra nhằm tối ưu hóa chương trình. Khái niệm kỹ nghệ phần mềm đã được bàn tới từ những năm 1960, nhiều công cụ ra đời nhằm nâng cao chất lượng phần mềm. Năm 1990, phương pháp lập trình hướng đối tượng ra đời, dẫn đến sự thay đổi lớn trong quá trình thiết kế và xây dựng phần mềm. Từ đó cho đến nay, các nhà khoa học đã đầu tư nghiên cứu trên nhiều lĩnh vực, khía cạnh nhằm đẩy mạnh phát triển kỹ nghệ phần mềm.

1.3. PHÂN LOẠI PHẦN MỀM

Dựa vào phạm vi và chức năng, phần mềm được phân ra thành các loại sau:

Ứng dụng đơn (Stand-alone applications) là những ứng dụng chạy trên máy cục bộ, như là máy tính cá nhân (PC). Nó bao gồm tất cả các chức năng cần thiết và không cần kết nối mạng. Ví dụ như những ứng dụng văn phòng trên máy PC, chương trình CAD, photoshop.

Ứng dụng tương tác dựa trên giao dịch (Interactive transaction-based applications) là những ứng dụng thực thi trên máy tính từ xa hoặc truy cập bởi người dùng từ máy tính của họ hoặc từ các thiết bị đầu cuối. Rõ ràng nó có bao gồm các ứng dụng web như thương mại điện tử nơi mà khách hàng tương tác với hệ thống mua bán sản phẩm và dịch vụ từ xa. Lớp ứng dụng này còn bao gồm hệ thống doanh nghiệp, nơi mà người dùng có quyền truy xuất tới hệ thống của họ qua trình duyệt web hoặc một chương trình khách đặc biệt hoặc dịch vụ đám mây như mail, chia sẻ hình ảnh. Ứng dụng tương tác thường kết hợp với dữ liệu lưu trữ lớn, dữ liệu này được truy cập, cập nhật bởi các ứng dụng.

Hệ thống điều khiển nhúng (Embedded control systems) là phần mềm hệ thống điều khiển các thiết bị phần cứng. Có rất nhiều phần mềm nhúng hơn so với các loại phần mềm khác. Ví dụ như phần mềm điều khiển điện thoại di động, phần mềm chống bó cứng phanh ô tô, phần mềm lò vi sóng điều khiển tiến trình nấu.

Hệ thống xử lý hàng loạt(Batch processing systems) là các hệ thống nghiệp vụ

được thiết kế để xử lý dữ liệu có số lượng lớn và rất lớn. Chúng xử lý số lượng lớn các giá trị đầu vào đơn lẻ tạo ra các giá trị đầu ra tương ứng. Ví dụ hệ thống thanh toán lương, thanh toán điện thoại, tiền điện …

Hệ thống giải trí (Entertainment systems) là hệ thống dành cho cá nhân và dùng để giải trí cho người dùng. Hầu hết các hệ thống này là trò chơi. Chất lượng tương tác với người dùng là đặc tính quan trọng.

Hệ thống mô phỏng và giả lập (Systems for modeling and simulation) là hệ thống được phát triển bởi các nhà khoa học và kỹ sư để mô hình hóa các tiến trình hoặc tình huống vật lý mà bao gồm rất nhiều tương tác riêng biệt. Đây là những tương tác tính toán cường độ lớn đòi hỏi những hệ thống song song để thực hiện.

Hệ thống thu thập dữ liệu (Data collection systems) là hệ thống thu thập dữ liệu từ môi trường, chúng thường sử dụng một loạt các cảm biến, gửi dữ liệu về hệ thống xử lý. Phần mềm cần phải tương tác với các cảm biến và được cài đặt trong môi trường không thân thiện như trong động cơ hoặc tại một địa điểm xa.

Hệ thống của các hệ thống (Systems of systems) là hệ thống được kết hợp với số lượng lớn các phần mềm khác. Một vài hệ thống trong số đó có thể là sản phẩm phần mềm thông thường như những chương trình bảng tính. Một vài hệ thống khác viết bằng hợp ngữ.

Tuy nhiên phân cách giữa các loại hệ thống này mờ. Giả sử phát triển một phần mềm trò chơi cho điện thoại di động, bạn cũng phải tính đến các ràng buộc khác như điện năng, tương tác phần cứng như người phát triển phần mềm điện thoại như vậy có sự mập mờ giữa phần mềm giải trí và phần mềm nhúng. Những phần mềm xử lý lớn thường dùng chung với các hệ thống nền web. Ví dụ trong một công ty, chi phí vận chuyển có thể được nhập thông qua ứng dụng web, nhưng ứng dụng xử lý số liệu này theo hàng tháng.

1.4. KỸ NGHỆ PHẦN MỀM

Phần mềm là chương trình máy tính và các tài liệu liên quan. Các sản phẩm phần mềm có thể được thiết kế cho một khách hàng cụ thể nào đó hoặc cho thị trường nói chung.

Kỹ nghệ phần mềm hay công nghệ phần mềm là ngành kỹ thuật liên quan đến tất cả các khía cạnh sản xuất phần mềm.

Các hoạt động nền tảng của kỹ nghệ phần mềm là đặc tả phần mềm (Software specification), phát triển phần mềm (software development), kiểm nghiệm phần mềm

(software validation), và tiến hóa phần mềm (software evolution).

Kỹ nghệ phần mềm là ngành công nghiệp liên quan đến mọi khía cạnh của việc phát triển phần mềm từ giai đoạn đầu đặc tả phần mềm cho đến giai đoạn cuối bảo trì phần mềm sau khi chuyển giao cho người dùng. Trong cụm từ kỹ nghệ phần mềm, kỹ nghệ ám chỉ các phần mềm được xây dựng bởi việc áp dụng lý thuyết, phương pháp, công cụ một cách chọn lọc và cố gắng tạo ra các giải pháp giải quyết vấn đề ngay cả khi không có lý thuyết và công cụ để áp dụng. Ngoài ra các giải pháp còn phải bó buộc trong phạm vi công nghệ, thời gian và kinh phí của tổ chức. Các khía cạnh công nghệ phần mềm không chỉ liên quan đến các tiến trình kỹ thuật phát triển phần mềm. Nó còn bao gồm các hoạt động như quản lý dự án phần mềm và sự phát triển các công cụ, phương thức và lý thuyết hỗ trợ xây dựng phần mềm.

Nhìn chung, các kỹ sư phần mềm luôn làm việc thông qua cách tiếp cận có hệ thống và có tổ chức, và thường sử dụng cách thức hiệu quả nhất để sản xuất phần mềm chất lượng cao. Tuy nhiên, trong một số trường hợp cụ thể, cách thức "không chính thống" lại mang lại hiểu quả hơn, và thường là cho các trường hợp phát triển hệ thống nền web, một hệ thống đòi hỏi sự pha trộn giữa kỹ thuật phần mềm và kỹ thuật đồ họa.

Công nghệ phần mềm liên quan đến hai lĩnh vực khoa học máy tính và hệ thống thông tin. Những kiến thức về khoa học máy tính rất cần thiết cho công nghệ phần mềm giống như các kiến thức vật lý cần thiết cho các kỹ sư điện tử. Tuy nhiên, lý thuyết về khoa học máy tính thường hay áp dụng cho các chương trình tương đối nhỏ, khó áp dụng cho các hệ thống lớn, những vấn đề phức tạp mà đòi hỏi cần có giải pháp phần mềm. Bên cạnh đó hệ thống thông tin liên quan đến tất cả các khía cạnh của việc phát triển, tiến hóa một hệ thống phức tạp trong đó phần mềm đóng vai trò chính. Vì vậy hệ thống thông tin có liên quan đến phát triển phần cứng, thiết kế các tiến trình, các quy định và phát triển hệ thống cũng như công nghệ phần mềm. Các chuyên gia hệ thống có liên quan đến việc đặc tả hệ thống, định nghĩa kiến trúc tổng thể và tích hợp các thành phần khác nhau tạo nên hệ thống hoàn chỉnh. Họ ít liên quan đến công nghệ của các thành phần (phần cứng, phần mềm,…).

Các thách thức của ngành công nghệ phần mềm là đối mặt với sự đa dạng ngày càng tăng, thời gian phát triển phần mềm ngày càng rút ngắn. Chi phí phần mềm chủ yếu dành 60% cho phát triển phần mềm và 40 % cho việc kiểm thử phần mềm. Đối với các phần mềm viết cho khách hàng, chi phí tùy chỉnh thường vượt quá chi phí phát triển ban đầu. Do đó công nghệ phần mềm quan trọng với 2 lý do sau:

- Ngày càng nhiều cá nhân và tập thể dựa vào các hệ thống phần mềm lớn.

Chúng ta cần phải có khả năng sản xuất các phần mềm có giá trị và có độ tin cậy cao một cách kinh tế và nhanh chóng.

- Trong thời gian dài, chi phí phát triển phần mềm theo phương pháp và kỷ thuật công nghệ phần mềm rẻ hơn so với việc phát triển phần mềm theo kiểu dự án cá nhân. Hầu hết các hệ thống, phần chi phí lớn nhất là dành cho việc sửa đổi chương trình sau khi đã đem sử dụng.

Cách tiếp cận có hệ thống được sử dụng trong công nghệ phần mềm được gọi là quy trình phần mềm. Một quy trình phần mềm là một chuỗi các hoạt động dẫn đến việc sản xuất ra phần mềm. Có bốn hoạt động cơ bản được phổ biến trong tất các loại quy trình phần mềm.

- **Đặc tả phần mềm (Software specification)** là giai đoạn mà khách hàng và kỹ sư cùng định nghĩa và đưa ra các ràng buộc trong cách thức hoạt động của phần mềm.
- **Phát triển phần mềm (Software development)** là giai đoạn phần mềm được thiết kế và lập trình.
- **Kiểm thử phần mềm (Software validation)** là giai đoạn kiểm tra để chắc chắn phần mềm đầy đủ các yêu cầu.
- **Tiến hóa phần mềm (Software evolution)** là giai đoạn phần mềm được chỉnh sửa đáp ứng các yêu cầu thay đổi của khách hàng cũng như thị trường.

Các loại hệ thống khác nhau cần các quy trình phát triển khác nhau. Ví dụ, một phần mềm thời gian thực cần phải được đặc tả hoàn thiện trước khi phát triển. Một hệ thống thương mại điện tử thì vừa đặc tả vừa phát triển. Vì vậy có những hoạt động có thể được tổ chức một cách khác nhau, mô tả ở mức độ khác nhau tùy thuộc vào loại phần mềm.

Các công nghệ phần mềm khác nhau được sử dụng khác nhau cho mỗi loại hệ thống bởi vì các phần mềm này có đặc tính khác biệt nhau. Ví dụ một phần mềm nhúng trong ô tô sẽ được viết vào ROM khi cài đặt xe. Vì vậy sẽ rất đắt khi thay đổi. Những hệ thống này cần sự kiểm định rất cẩn thận và chi phí rất cao để giảm nguy cơ thu hồi lại xe sau khi bán để sửa xe. Rất ít tương tác với người dùng do đó không cần dùng quy trình phát triển mẫu thử giao diện.

Một số yếu tố nền tảng của công nghệ phần mềm có thể áp dụng cho tất cả loại phần mềm:

- Các phần mềm nên được phát triển bởi các quy trình phát triển mà tổ chức lập trình am hiểu. Ngoài ra, tổ chức phát triển phần mềm cũng nên lập kế hoạch quy trình phát triển, phải hiểu rõ sẽ sản xuất cái gì và khi nào hoàn thiện. Dĩ nhiên, tổ chức nên lựa chọn các quy trình khác nhau cho các loại phần mềm

khác nhau.

- Độ tin cậy và hiệu xuất rất quan trọng cho mọi loại hệ thống. Phần mềm phải được như mong đợi và không gặp lỗi trông quá trình hoạt động, tránh khỏi được các tấn công từ bên ngoài, hoạt động hiệu quả và không lãng phí tài nguyên.
- Cần hiểu biết và quản lý được đặc tả và yêu cầu của phần mềm. Chúng ta cần phải biết sự khác biệt giữa khách hàng và người dùng hệ thống mong đợi từ phần mềm, và bạn phải xử lý điều đó để có một hệ thống hữu ích trong thời gian và ngân sách nhất định.
- Phải sử dụng hiệu quả tất cả những tài nguyên sẵn có. Điều này có nghĩa luôn ưu tiên sử dụng những phần mềm đã có hơn là viết một phần mềm mới nếu có thể.

Không có một phương thức công nghệ phần mềm tổng thể hoặc kỷ thuật có thể áp dụng cho tất cả loại phần mềm. Tuy nhiên cũng có 3 vấn đề chung ảnh hưởng tất các loại phần mềm:

- **Sự không đồng nhất tăng dần**: Hệ thống đòi hỏi được vận hành và phân phối qua mạng bao gồm nhiều loại máy tính và thiết bị di động. Cũng vì như vậy mà phần mềm cần phải thực thi được trên điện thoại di động. Phần mềm còn phải tích hợp được với các phần mềm cũ được viết bởi nhiều ngôn ngữ khác. Thách thức ở đây là phát triển kỹ thuật để xây dựng phần mềm đáng tin cậy, linh hoạt đủ để đương đầu với sự không đồng nhất về nền tảng.
- **Kinh tế xã hội thay đổi**: Đối với các nền kinh tế mới cộng với các công nghệ mới sẵn có, kinh tế xã hội đang thay đổi một cách nhanh chóng. Họ cần thay đổi các thành phần phần cũ của họ, nhanh chóng phát triển các phần mềm mới. Nhiều kỹ thuật công nghệ phần mềm truyền thống tốn nhiều thời gian và chuyển giao hệ thống mới lâu hơn dự kiến, điều đó giảm giá trị của phần mềm đối với khách hàng.
- **An toàn và đáng tin cậy**: Một phần mềm có liên quan đến tất cả các khía cạnh trong cuộc sống nên phần mềm phải an toàn và đáng tin cậy. Đặc biệt là đối với các phần mềm truy cập từ xa thông qua giao diện web hoặc dịch vụ web. Chúng ta cần đảm bảo các người dùng độc hại không thể tấn công phần mềm và an toàn thông tin luôn được duy trì.

Cần phải chú ý rằng những vấn đề nền tảng trên đây không bao gồm ngôn ngữ lập trình và quá trình cài đặt. Ví dụ một ngôn ngữ dùng để lập trình web thì không phù hợp với phát triển phần mềm nhúng.

1.5. CÔNG NGHỆ PHẦN MỀM VÀ WEB

Sự phát triển của World Wide Web đã có ảnh hưởng sâu rộng đến người sử dụng. Nguyên thủy, Web chỉ là nơi lưu trữ thông tin truy xuất rộng và nó rất ít ảnh hưởng đế hệ thống phần mềm. Những hệ thống phần mềm lúc này chạy trên máy cá nhân và chỉ được truy xuất từ bên trong tổ chức. Khoảng năm 2000, web bắt đầu tiến hóa, ngày càng nhiều chức năng được đưa thêm vào trình duyệt web. Từ đó nhiều hệ thống dựa trên nền web được xây dựng. Thay vì chỉ thông qua giao diện đặc thù như truyền thống, người dùng có thể truy xuất bằng cách sử dụng trình duyệt web. Từ đó một phạm vi rộng các hệ thống được phát triển cung cấp các dịch vụ sáng tạo thông qua web. Những dịch vụ này thường được tài trợ bởi các quảng cáo và người dùng không phải trả phí.

Cũng như các sản phẩm khác, sự phát triển của trình duyệt web cho phép thực thi một số chương trình nhỏ và tiền xử lý dữ liệu tại máy trạm (máy địa phương hay máy khách) dẫn đến sự tiến hóa trong quy trình nghiệp vụ và tổ chức phần mềm. Thay vì viết một phần mềm và xuất bản trên máy tính cá nhân (Personal Computer – PC), phần mềm nay được xuất bản trên máy chủ web. Điều này sẽ giảm chi phí khá nhiều khi thay đổi, cập nhật phần mềm vì không cần cài đặt phần mềm trên mỗi máy tính cá nhân. Nó cũng giảm chi phí phát triển giao diện người dùng. Hệ quả là nhiều doanh nghiệp đã chuyển sang sử dụng giao diện nền web để tương tác với hệ thống phần mềm của công ty bất cứ ở đâu có thể.

Giai đoạn phát triển tiếp theo của các hệ thống nền tảng web là dịch vụ web. Dịch vụ web là những thành phần phần mềm được sinh ra với chức năng cụ thể, hữu ích và được truy cập thông qua web. Ứng dụng được xây dựng dựa trên sự tích hợp các dịch vụ web, mà chúng có thể được cung cấp từ nhiều công ty khác nhau. Sự liên kết này là liên kết động nên ứng dụng có thể sử dụng các dịch vụ khác nhau trong mỗi lần thực thi.

Trong những năm gần đây, khái niệm phần mềm là một dịch vụ đã được xây dựng. Với khái niệm này, phần mềm sẽ không chạy trên máy tính cá nhân mà chạy trên điện toán đám mây và được truy xuất qua internet. Các dịch vụ như web mail cũng là hệ thống mây điện tử. Một hệ thống đám mây điện tử là một số lượng lớn hệ thống máy tính kết nối với nhau, và được sử dụng bởi nhiều người dùng. Người dùng không mua phần mềm mà chỉ chi trả theo cách sử dụng phần mềm đến đâu, hoặc có thể miễn phí nếu có quảng cáo hiển thị.

Sự ra đời của web dẫn đến sự thay đổi lớn trong cách thức quy trinh nghiệp vụ được tổ chức. Trước đó, các ứng dụng doanh nghiệp chủ yếu là một khối, là một chương trình đơn lẻ, chạy trên máy đơn hoặc nhóm máy. Trao đổi thông tin chỉ trong nội bộ trong tổ chức. Bây giờ, phần mềm được phân phối rộng, có lúc trên toàn thế giới. Ứng dụng doanh nghiệp không chỉ được lập trình từ đầu mà được tiến hóa mở rộng, tái sử dụng các thành phần và chương trình đã có.

Sự thay đổi cách thức phần mềm được tổ chức, dẫn tới sự thay đổi về cách thức thiết kế hệ thống trên nền web, ví dụ:

- Tái sử dụng phần mềm đã trở thành cách tiếp cận chính cho việc xây dựng hệ thống nền web. Khi xây dựng các hệ thống này, chúng ta nghĩ đến làm thế nào kết hợp chúng từ các thành phần và các hệ thống có sẵn.
- Không thể xác định tất cả các yêu cầu ngay lúc đầu. Hệ thống nền web nên được phát triển và chuyển giao từng bước.
- Giao diện người dùng bị bó hẹp bởi trình duyệt web. Mặc dù kỹ thuật Ajax (Holdener, 2008) mạng lại nhiều tương tác với giao diện nhưng vẫn rất khó sử dụng. Chủ yếu sử dụng web form với các mã cục bộ. Giao diện ứng dụng trên hệ nền web thường nghèo nàn hơn trên giao diện hệ trên máy PC.

1.6. ĐẠO ĐỨC NGÀNH CÔNG NGHỆ PHẦN MỀM

Như các lĩnh vực kỹ thuật khác, công nghệ phần mềm cũng được thực hiện trong khuôn khổ đạo đức, và pháp luật do xã hội quy định. Là một kỹ sư công nghệ phần mềm, chúng ta phải có trách nhiệm rộng lớn hơn đơn thuần chỉ là áp dụng các công nghệ kỹ thuật. Chúng ta không được sử dụng khả năng và kỹ thuật của mình để cư xử không trung thực, mang tiếng xấu đến ngành công nghệ phần mềm. Ngoài các ràng buộc của pháp luật, cũng có một số chuẩn mực do ngành đặt ra như sau:

- **Sự cẩn mật**: Chúng ta cần tôn trọng tính bảo mật của đơn vị chủ quản hoặc khách hàng không phân biệt có thoã thuận ký kết giữ bí mật hay không.
- **Quyền hạn:** Chúng ta không được nhầm lẫn quyền hạn của mình, không được cố ý làm việc ngoài quyền hạn của bản thân.
- **Quyền sở hữu trí tuệ:** Nên ý thức về luật bản quyền tại nơi mình làm việc để bảo đảm quyền sở hữu trí tuệ của người lao động và khách hàng.
- **Sử dụng máy tính sai lệch**: Chúng ta không nên sử dụng các kỹ năng kỹ thuật để sử dụng sai lệch mục đích máy tính của người khác. Sự sai lệch này có thể thay đổi từ nhỏ như chơi game trên máy làm việc đến rất lớn như cài đặt virus hoặc các phần mềm xấu.

Ngoài ra còn có 8 chuẩn mực mà một kỹ sư công nghệ phần mềm cần tuân theo:

- **Cộng đồng:** Kỹ sư công nghệ phần mềm phải hành động phù hợp với lợi ích cộng đồng
- **Khách hàng và người chủ quản**: Kỹ sư công nghệ phần phải làm việc theo cách mà mang lại lợi ích cho khách hàng và người tuyển dụng, phù hợp với lợi

ích cộng đồng.

- **Sản phẩm**: Kỹ sư công nghệ phần mềm bảo đảm sản phẩm và các sửa đổi có liên quan của họ phải đạt các tiêu chuẩn chuyên nghiệp cao nhất có thể.
- **Sự suy đoán**: Kỹ sư công nghệ phần mềm cần duy trì tính toàn vẹn trong các phán xét chuyên môn.
- **Quản lý:** Các nhà quản lý hoặc lãnh đạo công nghệ phần mềm cần đưa ra cách tiếp cận quản lý xây dựng vào bảo trì phần mềm có đạo đức.
- **Chuyên nghiệp**: kỹ sư công nghệ phần mềm nâng cao tính toàn vẹn, uy tín nghề nghiệp của mình phù hợp với lợi ích cộng đồng.
- **Đồng nghiệp**: Kỹ sư công nghệ phần mềm cần phải công bằng, và ủng hộ đồng nghiệp của mình.
- **Tự giác:** Kỹ sư công nghệ phần mềm luôn thực hành, học tập, và thúc đẩy cách thức làm việc có đạo đức, chuyên nghiệp suốt thời gian làm việc của mình.

1.7. CÁC QUY TRÌNH CÔNG NGHỆ PHẦN MỀM

Một quy trình phần mềm là tập hợp các hoạt động dẫn đến sản xuất một sản phẩm phần mềm. Các hoạt động này có thể liên quan đến việc phát triển phần mềm từ đầu bằng ngôn ngữ lập trình cơ bản như C hoặc Java. Tuy nhiên, đối với ứng dụng doanh nghiệp thì không cần thiết phát triển theo cách này. Các phần mềm này thường được phát triển bằng cách mở rộng, thay đổi hệ thống đã có hoặc cấu hình, tich hợp với các thành phần hệ thống, các phần mềm sẵn có khác.

Có rất nhiều quy trình phần mềm khác nhau nhưng tất cả đều bao gồm 4 hoạt động cơ bản:

- **Đặc tả phần mềm**: Định nghĩa các chức năng phần mềm, các ràng buộc trên các hoạt động của phần mềm.
- **Thiết kế và cài đặt phần mềm**: Phần mềm cần phải đúng với các đặc tả.
- **Kiểm định phần mềm**: Phần mềm cần được kiểm định để chắc chắn đúng với khách hàng muốn.
- **Tiến hóa phần mềm**: Phần mềm cần phải tiến hóa được đáp ứng nhu cầu thay đổi của khách hàng.

Trong một số hình thức, các hoạt động này là một phần của quy trình phần mềm. Trong thực tế, chúng là những hoạt động phức tạp, bao gồm nhiều các hoạt động thành

phần như việc kiểm định lại yêu cầu, thiết kế kiến trúc, kiểm tra các thành phần … Có những hoạt động hỗ trợ như viết tài liệu và quản lý cấu hình phần mềm.

Khi chúng ta mô tả và thảo luận về các tiến trình, chúng ta thường nói về các hoạt động của những tiến trình này như là mô hình đặc tả, thiết kế giao diện, thứ tự của những hoạt động. Việc mô tả các tiến trình này bao gồm:

- **Sản phẩm**: Đó là đầu ra của các hoạt động trong quy trình. Ví dụ, nếu đầu ra của hoạt động thiết kế kiến trúc có thể là một bản mô hình hóa kiến trúc phần mềm.
- **Vai trò**: Các phản ánh trách nhiệm của con người liên quan đến quy trình. Ví dụ vai trò là người quản lý dự án, người phân công công việc, người lập trình …
- **Điều kiện trước và sau**: Đó là những điều đứng trước hoặc sau hoạt động của quy trình được kích hoạt hoặc một sản phẩm được làm ra. Ví dụ trước khi thiết kế kiến trúc, điều kiện trước là tất cả các yêu cầu đã được khách hàng duyệt qua; sau khi hoạt động này hoàn thành, điều kiện sau có thể là mô hình UML mô tả kiến trúc đã được xem qua.

Quy trình phần mềm phức tạp và giống như tất các quy trình sáng tạo và đầy chất xám, dựa trên sự phán đoán và quyết định của con người. Không có quy trình nào lý tưởng mà hầu hết các tổ chức đã sử dụng làm quy trình phát triển phần mềm riêng cho họ. Các quy trình đã được tiến hóa, tận dụng được các điểm mạnh nhân lực trong tổ chức và các đặc tính đặc trưng của hệ thống đang phát triển. Đối với một số hệ thống, như những hệ thống quan trọng, yêu cầu một quy trình phát triển rất có cấu trúc. Đối với hệ thống doanh nghiệp, với sự thay đổi yêu cầu nhanh chóng, một quy trình phát triển linh hoạt, ít chuẩn hóa thì có thể thích hợp hơn.

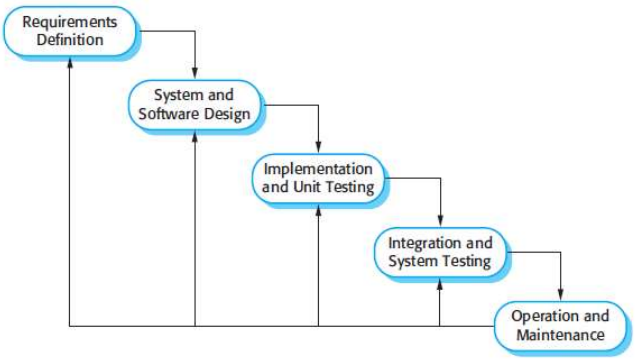
Thỉnh thoảng, quy trình phần mềm được phân loại hoặc là quy trình theo kế hoạch (either plan-driven processes) hoặc là quy trình nhanh (agile processes). Quy trình hướng kế hoạch là quy trình mà ở đó tất các hoạt động được lên kế hoạch trước và có tiến độ đo lường lại kế hoạch này, trong khi đó quy trình nhanh là kế hoạch được bổ sung dần, và dễ dàng thay đổi quy trình để phản ánh sự thay đổi yêu cầu của khách hàng.

Mỗi một quy trình thể hiện quy trình đó theo một quán điểm cụ thể, vì vậy chỉ cung cấp một phần thông tin của quy trình đó. Ví dụ quy trình thể hiện theo các hoạt động thì không thể hiện vai trò của con người trong đó. Các quy trình mà chúng ta xem xét ở đây đều nhìn dưới góc độ cấu trúc, thứ tự các hoạt động của quy trình.

Chúng là sự trừu tượng của quy trình mà có thể dùng để diễn tả phương thức tiếp cận phát triển phần mềm khác nhau. Chúng là những quy trình khung mà có thể mở rộng, thay đổi thích nghi để tạo những quy trình công nghệ phần mềm.

1.7.1. Mô hình thác nước

Mô hình này lấy các hoạt động nền là đặc tả, phát triển, kiểm thử và tiến hóa, thể hiện chúng là những pha của quy trình tách biệt như là đặc tả yêu cầu, thiết kế phần mềm, cài đặt, kiểm thử …



Hình 1-1: Mô hình thác nước

Đây là mô hình quy trình phát triển phần mềm xuất bản đầu tiên xuất phát từ các quy trình công nghệ trước đó. Bởi vì tính đi xuống từ pha này sang pha khác, nên mô hình này được gọi là mô hình thác nước hay vòng đời phần mềm. Mô hình thác nước là một ví dụ của quy trình hướng kế hoạch, chúng ta cần phải đặt kế hoạch và xếp lịch tất các hoạt động của quy trình trước khi bắt đầu làm việc trên đó.

Các giai đoạn của mô hình thác nước trực tiếp phản ánh các hoạt động phát triển nền tảng:

- **Phân tích và định nghĩa các yêu cầu**: Các dịch vụ, quy định, mục tiêu của hệ thống được thiết lập bởi việc tham khảo với người dùng hệ thống. Sau đó, chúng được định nghĩa chi tiết phục vụ cho việc đặc tả hệ thống.
- **Thiết kế phần mềm và hệ thống**: Quy trình thiết kế hệ thống chỉ ra các yêu cầu về phần cứng hoặc phần mềm bằng cách thiết kế kiến trúc hệ thống chung. Thiết kế phần mềm liên quan đến việc xác định, mô tả sự trừu tượng của hệ thống phần mềm cơ bản và mối quạn hệ của chúng.
- **Cài đặt và kiểm thử các thành phần**: Suốt giai đoạn này, thiết kế phần mềm được thực hiện như một tập hợp các chương trình hoặc các đơn vị chương trình. Việc kiểm thử liên quan đến từng đặc tả của chúng.

- **Tích hợp và kiểm thử hệ thống**: Từng đơn vị chương trình riêng biệt hoặc các chương trình riêng biệt được tích hợp lại với nhau và kiểm thử như là một hệ thống hoàn chỉnh để bảo đảm rằng yêu cầu phần mềm đã được thỏa mãn.
- **Vận hành và bảo trì**: Thông thường, đây là giai đoạn dài nhất của vòng đời phần mềm. Hệ thống được cài đặt và đưa vào sử dụng thực tế. Bảo trì liên quan đến việc sửa lỗi mà đã không khám phá ra trong giai đoạn trước, cải tiến việc thực hiện của các đơn vị thành phần hệ thống và tăng cường các dịch vụ của hệ thống khi có các yêu cầu mới.

Về nguyên tắc, kết quả của mỗi pha là các tài liệu mà đã được thông qua. Các pha sau chỉ được thực hiện khi pha trước kết thúc. Trong thực thế các giai đoạn này giao thoa lên nhau và cung thông tin cho nhau. Trong quá trình thiết kế, những vấn đề với yêu cầu được chỉ ra. Trong khi viết chương trình, những vấn đề thiết kế được tìm thấy …Quy trình phần mềm không chỉ đơn giản là mô hình tuyến tính đơn giản mà còn liên quan đến những phản hồi giữa các pha với nhau. Tài liệu được sinh ra ở mỗi pha có thể được thay đổi phản ánh các thay đổi này.

Bởi vì chi phí của việc làm ra và duyệt qua các tài liệu, việc lặp lại sẽ tốn kém và liên quan nhiều đến việc phải làm lại. Vì vậy, sau một số lần lặp nhỏ, thường một số phần bị đóng băng, ví dụ như phần đặc tả, và tiếp tục với các giai đoạn phát triển sau. Vấn đề sẽ được bỏ lại và chờ các giải pháp sau, hoặc bỏ qua hoặc lập trình để xử lý. Việc đống băng các yêu cầu có nghĩa hệ thống sẽ không thực hiện cái mà người dùng muốn. Nó có thể dẫn đến cấu trúc hệ thống kém khi các vấn đề thiết kế được thực hiện bằng các tiểu xảo lập trình.

Trong suốt pha cuối của vòng đời hệ thống, phần mềm được đưa vào sử dụng. Lỗi và các thiếu sót trong yêu cầu phần mềm lúc ban đầu bộc lộ. Lỗi về thiết kế và lập trình nổi lên và nhu cầu chức năng mới được xác định. Hệ thống phải bảo trì để mang lại lợi ích người dùng. Sửa các lỗi này có thể phải lập lại các giai đoạn trước của quy trình.

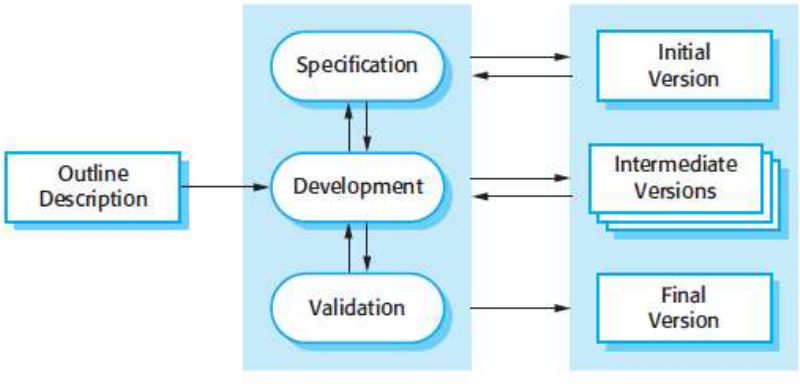
Quy trình thác nước nhất quán hơn các quy trình khác. Mỗi giai đoạn của mô hình quy trình thác nước đều có sản phẩm tài liệu. Điều này làm cho quy trình rõ ràng do đó các nhà quản lý có thể giám sát tiến độ so với kế hoạch phát triển. Vấn đề chính của nó là các phần phân chia không linh hoạt của dự án trong mỗi giai đoạn tách biệt. Sự thỏa thuận (về yêu cầu hệ thống) được thực hiện từ pha đầu của quy trình nên dẫn đến có khó trong việc đối phó với những thay đổi yêu cầu của khách hàng.

Về nguyên tắc, mô hình thác nước nên được dùng khi mà các yêu cầu được hiểu rõ,

gần như không thay đổi trong quá trình phát triển. Nó là một mô hình phổ biến, dễ quản lý cho cả dự án nên các quy trình phần mềm dựa trên mô hình thác nước vẫn được dùng rộng rãi.

1.7.2. Mô hình tăng trưởng

Cách tiếp cận này gần giống với mô hình thác nước, xen kẻ các hoạt động đặc tả, phát triển và kiểm thử. Hệ thống được phát triển như những dãy các phiên bản với mỗi phiên bản thêm các hoạt động từ phiên bản trước.



Hình 1-2: Mô hình tăng trưởng

Quy trình phát triển tăng trưởng dựa vào ý tưởng phát triển bản gốc (quy trình thác nước), thêm hoạt động chuyển giao, lấy phản hồi từ khách hàng và tiến hóa nó qua các phiên bản cho đến khi hoàn thành một hệ thống đầy đủ. Các hoạt động đặc tả, phát triển và kiểm định được xen kẽ chứ không tách biệt như quy trình gốc, cùng với các phản hồi nhanh chóng thu được qua các hoạt động này.

Phát triển phần mềm tăng trưởng là cách tiếp cận nhanh, và tốt hơn cách tiếp cận thác nước cho hầu hết các hệ thống doanh nghiệp, thương mại điện tử và cá nhân. Phát triển tăng trưởng phản ánh cách thức mà chúng ta giải quyết vấn đề. Chúng ta hiếm khi đề ra một phương pháp giải quyết vấn đề trọn vẹn từ sớm nhưng về sau, từng bước một, chúng ta nhận ra chúng ta gặp lỗi. Bằng cách phát triển phần mềm theo cách tăng trưởng, sẽ tốn ít chi phí hơn, dễ dàng hơn để thay đổi trong phần mềm khi nó đang trong quá trình đang phát triển.

Mỗi lần thêm, hoặc mỗi một phiên bản, hệ thống kết hợp với chức năng mà người dùng cần. Thông thường, trong những lần tăng thêm ban đầu, hệ thống bao gồm các chức năng quan trọng, cần thiết để hoạt động. Có nghĩa rằng khách hàng có thể cải tiến hệ thống trong giai đoạn đầu phát triển để thấy rằng phần mềm có cung cấp những điều được yêu cầu. Nếu không đúng, thì tại lần tăng thêm này cần thay đổi và có thể

thêm chức năng mới cho lần thêm sau.

So với mô hình thác nước, mô hình tăng trưởng có ba cái lợi quan trọng:

- **Chi phí thay đổi yêu cầu của khách hàng giảm**. Số lượng tài liệu phải làm lại cũng ít hơn nhiều so với mô hình thác nước.
- **Dễ dàng để nhận phản hồi từ khách hàng**. Trên những sản phẩm trong quá trình phát triển đã làm. Khách hàng có thể cho ý kiến phần mềm và thấy được phần mềm đã được thực hiện đến đâu. Khách hàng khó mà đánh giá tiến độ thực hiện từ bản thiết kế.
- **Có thể chuyển giao và phát triển phần mềm nhanh hơn**, ngay cả khi tất cả các chức năng chưa được cài đặt hết. Khách hàng có thể sử dụng và thu nhận giá trị từ phần mềm sớm hơn so với mô hình thác nước hay nói cách khác chó thể chuyển giao từng phần cho khách hàng trong quá trình phát triển phần mềm.

Các hình thức của mô hình tăng trưởng hiện nay được sử dụng phổ biến. Phương pháp tiếp cận này có thể là cách tiếp cận hướng kế hoạch, cũng có thể là tiếp cận nhanh hoặc trộn cả hai. Trong cách hướng kế hoạch, số lần tăng thêm được xác định trước, nếu áp dụng cách tiếp cận nhanh, thì các bước tăng trưởng ban đầu được xác định, nhưng các bước tăng vào giai đoạn phát triển thì phụ thuộc tiến độ và ưu tiên của khách hàng.

Mặc dù mô hình phát triển tăng trưởng có nhiều điểm mạnh, nó cũng không phải là không có vấn đề. Từ góc độ quản lý, quy trình tăng trưởng có hai vấn đề:

- **Quy trình không rõ ràng**. Người quản lý cần những lần chuyển giao bình thường để đo lường tiến độ. Nếu hệ thống được phát triển nhanh chóng, sẽ không ảnh hưởng đến chi phí để sản xuất tài liệu phản ánh từng phiên bản của hệ thống.
- **Cấu trúc hệ thống có xu hướng suy giảm khi các yêu cầu mới thêm vào**. Trừ khi dành thêm thời gian và tiền bạc cho việc phân tích lại để cải thiện phần mềm, những thay đổi đó sẽ làm hỏng cấu trúc của hệ thống. Những hợp nhất thay đổi của phần mềm trong tương lai trở nên càng khó khăn và tốn chi phí hơn.

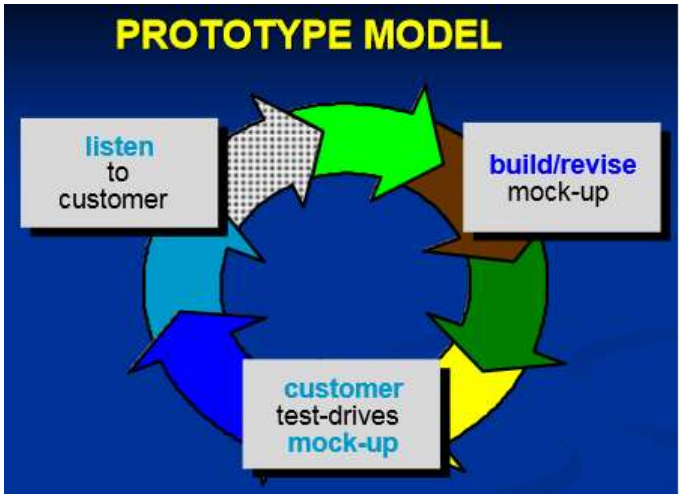
Các vấn đề của việc phát triển tăng trưởng trở nên gay gắt hơn đối với các hệ thống phức tạp và có đời sống dài, khi mà có nhiều đội khác nhau phát triển các phần khác nhau của hệ thống. Hệ thống lớn cần một hệ khung và kiến trúc ổn định và trách nhiệm

của các đội khác nhau khi làm một phần của hệ thống cần phải tôn trọng kiến trúc đó. Điều này phải được đã được dự kiến từ trước hơn là được phát triển tăng dần.

Chúng ta có thể phát triển hệ thống tăng dần và đưa nó cho người dùng để lấy ý kiến, không thực sự xuất bản nó và cài đặt trong môi trường của khách hàng. Sự chuyển giao và phát triển tăng dần có nghĩa rằng phần mềm được thực sự sử dụng trong thực tế. Điều này không phải lúc nào cũng thực hiện được vì khi thử nghiệm phần mềm mới có thể làm hỏng quy trình nghiệp vụ.

1.7.3. Mô hình bản mẫu (prototyping)

Mô hình bản mẫu dựa trên ý tưởng xây dựng một bản mẫu ban đầu, và đưa cho người dùng xem xét, sau đó tinh chỉnh mẫu thử qua nhiều phiên bản cho đến khi thỏa mãn yêu cầu của người sử dụng thì dừng lại. Mẫu thử ban đầu như là cơ chế để xác định, nhận diện chính xác yêu cầu của khách hàng. Các mẫu thử có thể bị loại bỏ hoặc phát triển thành sản phẩm cuối cùng.



Hình 1-3: Mô hình bản mẫu

Trong mỗi lần xây dựng mẫu thử, đều thông qua các hoạt động thu thập yêu cầu, thiết kế, cài đặt và kiểm thử.

Với quy trình này, khách hàng có thể tương tác sớm với hệ thống, khách hàng và người lập trình làm việc với nhau, tìm ra các yêu cầu mới. Mô hình cho phép thiết kế và phát triển mềm dẽo qua nhiều vòng lặp. Những mẫu thử ban đầu chủ yếu chú trọng đến giao diện để mô tả và thu thập yêu cầu sát thực tế trong thời gian ngắn. Khách hàng dễ dàng nắm bắt phần mềm thông qua mẫu thử hơn là bản thiết kế.

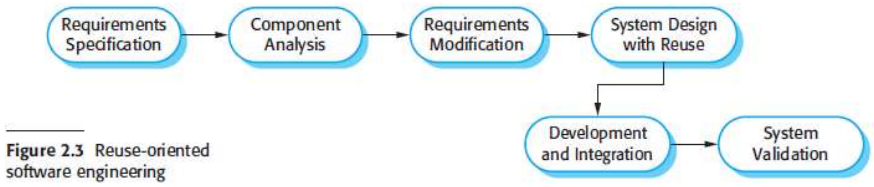
Tuy vậy, phần mềm phát triển theo quy trình này thường có cấu trúc nghèo nàn, với những lựa chọn không tốt từ phía khách hàng. Hệ thống có chất lượng không tốt,

khó bảo trì về sau. Khách hàng dễ có những nhầm lẫn giữa mẫu thử và phần mềm hoàn thiện, sẽ hối thúc tổ chức phát triển trong thời gian ngắn.

Mô hình này thường được sử dụng khi các yêu cầu không rõ ràng, không ổn định và không nhận được thông tin đầy đủ từ phía khách hàng. Trong trường hợp người phát triển chưa chắc chắn nên sử dụng giải thuật hay kiến trúc nào tối ưu cũng có thể sử dụng quy trình này.

1.7.4. Mô hình tái sử dụng

Cách tiếp cận này dựa trên số lượng lớn các thành phần phần mềm cần thiết. Quy trình phát triển phần mềm chú trọng vào việc tích hợp các thành phần này vào hệ thống hơn là phát triển từ đầu.



Hình 1-4: Mô hình tái sử dụng

Trong phần lớn các dự án phần mềm, có một số phần mềm được tái sử dụng. Điều này thường xảy ra khi người làm việc trong dự án biết bản thiết kế và mã lệnh mà chúng tương tự với yêu cầu. Người ta tìm, thay đổi chúng theo nhu cầu, kết hợp chúng lại tạo thành hệ thống của họ.

Việc tái sử dụng này diễn ra bất kỳ quy trình nào được sử dụng. Dù sao đi nữa, trong thế kỷ 21, quy trình phát triển phần mềm chú trọng vào việc tái sử dụng các phần mềm đã có sẵn ngày càng trở nên được sử dụng rộng rãi. Cách thức tái sử dụng dựa trên một số lượng lớn thành phần phần mềm có thể tái sử dụng và một phần mềm khung tích hợp cho việc kết hợp các thành phần đó. Thỉnh thoảng, những thành phần này tự nó là một hệ thống có thể cung cấp một chức năng nhất định nào đó ví dụ như xử lý văn bản hoặc bảng tính.

Mặc dù pha ban đầu đặc tả yêu cầu và kiểm thử thì giống với các quy trình phần mềm khác, các giai đoạn trung gian của quy trình hướng tái sử dụng thì khác. Các giai đoạn này là:

- **Phân tích thành phần**: Đưa ra các đặc tả phần mềm, thực hiện tìm kiếm các thành phần thực hiện các yêu cầu. Thường không có thành phần nào phù hợp chính xác và chỉ sử dụng được khi cung cấp thêm một số chức năng.

- **Chỉnh sửa yêu cầu**: Trong suốt giai đoạn này, yêu cầu được phân tích và sử dụng các thông tin về các thành phần đã khám phá ra. Chúng được thay đổi để phản ánh đúng các thành phần đã có. Những nơi không thể chỉnh sửa, pha phân tích có thể thực hiện lại tìm kiếm giải pháp thay thế.
- **Thiết kế hệ thống với tái sử dụng**: Trong suốt pha này, hệ thống khung được thiết kế hoặc một phần mềm khung được tái sử dụng. Người thiết kế tính đến các thành phần tái sử dụng và tổ chức phần mềm khung cho việc này. Một vài phần mềm mới có thể phải thiết kế nếu như các thành phần tái sử dụng không có sẵn.
- **Phát triển và tích hợp**: Phần mềm mà không thể tìm kiếm ở ngoài thì được phát triển. Những thành phần tự phát triển, các thành phần thu thập được và các thành phần thương mại được tích hợp để tao ra một hệ thống mới. Trong mô hình này, hệ thống tích hợp được coi là một phần của giai đoạn phát triển chứ không phải là một hoạt động tách biệt.

Có 3 loại thành phần phần mềm mà có thể dùng trong quy trình hướng tái sử dụng:

- **Dịch vụ web** được phát triển dựa trên các tiêu chuẩn dịch vụ và nó có sẵn từ xa.
- **Tập hợp các đối tượng** được phát triển như gói sản phẩm phần mềm được tích hợp với một thành phần khung như .NET hoặc J2EE.
- **Các hệ thống phần mềm riêng biệt** mà được cấu hình để sử dụng trong một môi trường nào đó.

Công nghệ tái sử dụng có những điểm mạnh rõ ràng, đó là giảm số lượng phần mềm phải phát triển, và có thể giảm chi phí và rủi ro. Nó thường dẫn tới việc chuyển giao phần mềm nhanh hơn. Tuy nhiên những thỏa thuận yêu cầu ban đầu là chắc chắn mà trong quá trình phát triển đã có thể bị thay đổi, việc này có thể dẫn tới hệ thống không đáp ứng nhu cầu thực sự của người dùng. Thêm vào đó, khi hệ thống tiến hóa trong tương lai có thể mất kiểm soát. Các phiên bản mới của thành phần tích hợp không nằm trong sự kiểm soát của tổ chức sử dụng nó.

1.7.5. Quy trình RUP (Rational Unified Process)

RUP là một quy trình vòng lặp phát triển phần mềm được tạo ra bởi công ty Rational Software, một bộ phận của IBM từ năm 2002 (IBM Rational). RUP không phải là một quy trình bó hẹp cụ thể đơn nhất mà là một nền tảng quy trình thích ứng với sự phát triển các tổ chức và các nhóm dự án phần mềm, tất cả sẽ chọn các yếu tố

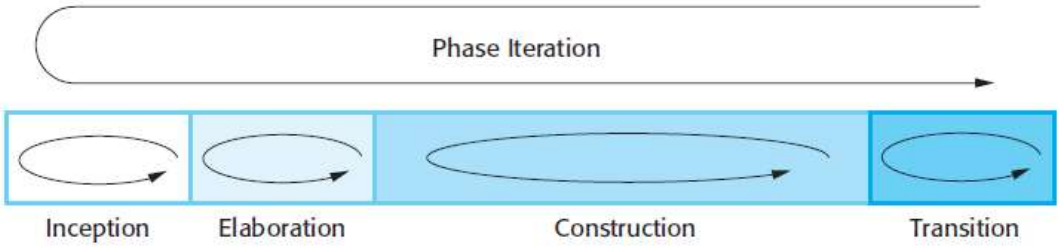
cần thiết của quy trình để phù hợp với nhu cầu, quy mô của công ty, dự án và sản phẩm.

RUP thường được mô tả từ 3 góc nhìn:

- Góc nhìn động là góc nhìn thể hiện các giai đoạn của quy trình theo thời gian.
- Góc nhìn tĩnh là góc nhìn mô tả các hoạt động của quy trình.
- Một góc nhìn thực tế đề nghị các thói quen tốt.

RUP chủ yếu mô tả kết hợp giữa góc nhìn động và góc nhìn tĩnh xen kẽ nhau trong một mô hình duy nhất nhưng điều này làm cho quy trình khó hiểu hơn với việc tách biệt các góc nhìn.

RUP là mô hình phân thành 4 giai đoạn riêng lẽ nhưng khác với 4 giai đoạn trong mô hình thác nước. Các giai đoạn này gần với nghiệp vụ hơn so với kỹ thuật như trong mô hình thác nước. Hình sau mô tả 4 giai đoạn của RUP:



Hình 1-5: Các giai đoạn của mô hình RUP

- **Khởi động (inception):** mục tiêu của hệ thống này là thành lập các trường hợp nghiệp vụ (business case) cho hệ thống. Trong giai đoạn này, cần phải chỉ ra tất cả các thực thể bên ngoài (con người, hệ thống) tương tác với hệ thống và định nghĩa những tương tác này. Các thông tin này được sử dụng để đánh giá khả năng đóng góp của hệ thống cho tổ chức. Nếu sự đóng góp này nhỏ, dự án phần mềm có thể bị hủy sau giai đoạn này.
- **Phác thảo (Elaboration)**: Mục tiêu của giai đoạn này là nghiên cứu lĩnh vực đang giải quyết, thiết lập kiến trúc hệ thống, phát triển kế hoạch dự án, chỉ ra những điểm rủi ro chính mà dự án có thể gặp phải. Khi hoàn thành dự án này, bản phân tích yêu cầu của hệ thống, có thể là tập hợp các ca sử dụng UML, bản mô tả kiến trúc hệ thống, và kế hoạch phát triển phần mềm.
- **Xây dựng (Construction):** Giai đoạn này liên quan đến thiết kế hệ thống, lập trình và kiểm thử. Những phần của hệ thống được phát triển song song và tích

hợp lại trong suốt pha này. Khi hoàn thành, sản phẩm phải có một hệ thống phần mềm hoạt động được và tài liệu đi kèm, tất cả sẵn sàng chuyển giao cho người dùng.

- **Chuyển giao (Transition)**: Là giai đoạn cuối tập trung vào việc chuyển giao hệ thống từ nhà phát triển đến cộng đồng người dùng và làm cho nó vận hành trong môi trường thực. Thỉnh thoảng điều này được bỏ qua trong nhiều quy trình phần mềm nhưng thực tế đây là hoạt động tốn chi phí và gặp nhiều vấn đề. Sau khi hoàn thành giai đoạn này, chúng ta có bản ghi chép về hệ thống hoạt động chính xác trong môi trường vận hành.

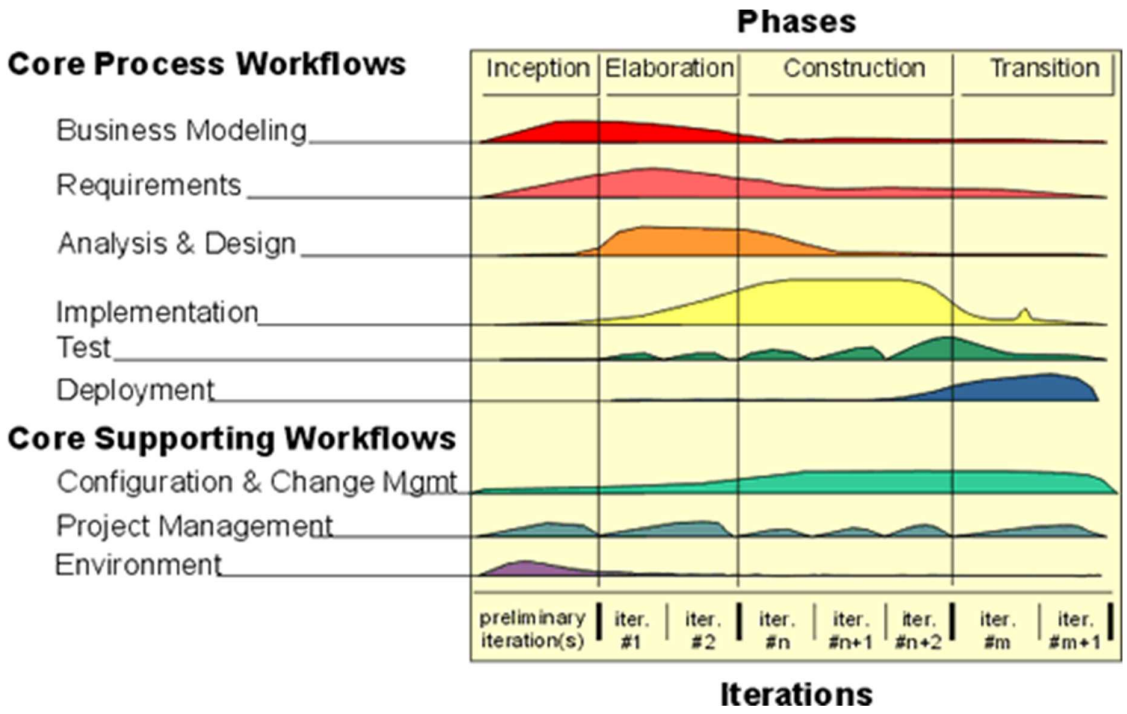
Các vòng lặp của RUP được hỗ trợ bằng 2 cách. Mỗi pha có thể được thực hiện lặp lại với kết quả tăng dần. Thêm vào đó, tất cả các pha cũng có thể lặp lại một cách tăng trưởng như trong hình 1-5. Có 6 tiến trình công việc lõi và 3 công việc phụ. RUP đã được thiết kế kết hợp chung với ngôn ngữ UML, cho nên việc mô tả các tiến trình công việc hương kết hợp với mô hình UML như biểu đồ tuần tự, biểu đồ đối tượng.

Các tiến trình công việc chính như sau:

- **Mô hình hóa nghiệp vụ**: mô tả cấu trúc và quy trình nghiệp vụ.
- **Xác định yêu cầu**: mô tả nghiệp vụ bằng phương pháp “use case”.
- **Phân tích và thiết kế**: mô tả kiến trúc hệ thống thông qua các sơ đồ phân tích thiết kế như sơ đồ kiến trúc, sơ đồ các thành phần, sơ đồ đối tượng và sơ đồ tuần tự.
- **Lập trình**: Thực hiện các công việc lập trình bằng ngôn ngữ lập trình. Hệ thống tự tạo mã thông qua bản thiết kế sẽ hỗ trợ rất nhiều trong giai đoạn này.
- **Thử nghiệm**: mô tả các tình huống và kịch bản thử nghiệm, tiến hành thử nghiệm hệ thống phần mềm.
- **Triển khai**: đưa hệ thống phần mềm vào sử dụng.

Các tiến trình công việc hỗ trợ:

- **Quản trị cấu hình và quản trị thay đổi**: kiểm soát các thay đổi và duy trì sự hợp nhất của các thành phần dự án
- **Quản trị dự án**: quản lý toàn bộ quá trình làm việc của dự án
- **Môi trường**: đảm bảo các hạ tầng cần thiết để có thể phát triển được hệ thống



Hình 1-6: Quy trình RUP

Lợi thế của góc nhìn động và góc nhìn tĩnh là các giai đoạn phát triển không kết hợp với một tiến trình công việc cụ thể. Về nguyên tắc ít nhất tất cả tiến trình công việc trong RUP có thể được hoạt động trong tất cả các pha của quy trình. Trong những pha sớm, công việc cần nhiều đầu tư nhất là mô hình hóa nghiệp vụ và mô hình hóa yêu cầu, sau đó, những pha sau là thử nghiệm và triển khai.

Góc nhìn thực tế của mô hình RUP diễn tả những thói quen tốt trong công nghệ phần mềm được đề nghị sử dụng trong phát triển hệ thống. Sáu thói quen nền tảng được đề nghị như sau:

- Xây dựng kế hoạch tuần hoàn trong đó mở rộng hệ thống dựa vào ưu tiên của khách hàng và phát triển những tính năng được ưu tiên hàng đầu trong giai đoạn đầu của quy trình phát triển.
- Quản lý tài liệu yêu cầu của khách hàng một cách rõ ràng và theo dõi những thay đổi trên các yêu cầu này. Phân tích các tác động trên sự thay đổi của hệ thống trước khi chấp nhận chúng.
- Sử dụng sơ đồ kiến trúc thành phần để cấu trúc hóa kiến trúc hệ thống thành các thành phần khác nhau.
- Các mô hình phần mềm trực quan sử dụng các mô hình UML để thể hiện góc nhìn tĩnh và góc nhìn động của phần mềm.

- Kiểm tra chất lượng phần mềm để bảo đảm rằng phần mềm đạt các chuẩn mực chất lượng đề ra.

Một quy trình mô tả ai (who) đang làm gì (what), làm như thế nào (how) và làm khi nào (when). RUP sử dụng 4 yếu tố chính trong mô hình.

- **Worker:** định nghĩa các hành vi, trách nhiệm của một cá nhân hoặc một nhóm làm việc với nhau (còn gọi là team).
- **Activity:** của một worker là một đơn vị công việc được giao cho một cá nhân thực hiện. Activity có mục đích rõ ràng, thường là yêu cầu tạo mới hoặc cập nhật một số artifacts như là model, class, hoặc plan. Mỗi một activity được phân công cho một workder rõ ràng.
- **Artifact:**Kết quả của quá trình phát triển các RUP được gọi là các Artifact, bao gồm các mô hình và các bộ tài liệu sau:
    - **Mô hình nghiệp vụ**: mô tả tổ chức nghiệp vụ của hệ thống cần xây dựng.
    - **Mô hình tình huống sử dụng**: xác định các chức năng của hệ thống phần mềm.
    - **Mô hình phân tích và thiết kế**: xác định các đối tượng của hệ thống phần mềm để giải quyết vấn đề của bài toán.
    - **Mô hình triển khai**: xác định kiến trúc phần cứng và phần mềm hệ thống cần thiết để triển khai.
    - **Mô hình thử nghiệm**: xác định các bước mà hệ thống sẽ được kiểm tra.
    - **Bộ tài liệu về xác định yêu cầu hệ thống**: mô tả những gì hệ thống cần làm.
    - **Bộ tài liệu thiết kế**: mô tả hệ thống sẽ được xây dựng như thế nào.
    - **Bộ tài liệu lập trình**: mô tả các thành phần ứng dụng được phát triển như thế nào.
    - **Bộ tài liệu triển khai**: mô tả cấu trúc triển khai hệ thống.
- **Workflow** là một chuỗi các hành động diễn ra liên tiếp nhằm tạo ra kết quả có giá trị rõ ràng. Trong UML, workflow có thể diễn đạt bằng sơ đồng tuần tự (sequence diagram), sơ đồ hợp tác (collaboration diagram) hoặc sơ đồ hoạt động (activity diagram).

RUP không phải là quy trình phù hợp cho phát triển tất cả các loại phần mềm ví dụ như phần mềm nhúng. Tuy nhiên, nó mang lại cách tiếp cận kết hợp các loại mô hình đã nêu. Vấn đề quan trọng nhất trong RUP là sự tách biệt các pha và tiến trình công việc và công đoạn triển khai phần mềm vào mội trường người dùng là một phần của quy trình. Các pha là động và được gắn mục tiêu riêng. Tiến trình công việc là tĩnh và là những hoạt động kỹ thuật không kết hợp với một pha nào đơn lẻ, nhưng mà có thể dùng trong suốt quy trình phát triển và đạt được mục tiêu mỗi pha.

1.7.6. Quản lý dự án phần mềm

Quản lý dự án phần mềm là một phần quan trọng của kỹ nghệ phần mềm. Nhu cầu dự án cần được quản lý bởi vì công nghệ phần mềm chuyên nghiệp luôn chú trọng đến các ràng buộc về kinh phí và thời gian của tổ chức. Công việc của người quản lý dự án là bảo đảm dự án phần mềm thỏa mãm và vượt trên các ràng buộc này và tạo ra phần mềm chất lượng cao. Việc quản lý dự án tốt không đảm bảo dự án sẽ thành công. Tuy nhiên, việc quản lý tồi thường là nguyên nhân của sự thất bại sau: phần mềm chuyển giao châm, chi phí cao hơn so với ước tính ban đầu hoặc không đạt sự mong đợi của khách hàng.

Những tiêu chí thành công cho quản lý dự án là:

- Chuyển giao phần mềm cho khách hàng đúng thời gian thỏa thuận.
- Duy trì chi phí dự án trong phạm vi kinh phí chung.
- Phần mềm thỏa mãn mong đợi của khách hàng
- Duy trì đội phát triển vui vẻ và làm việc tốt, cộng tác tốt.

Không thể viết một bản mô tả công việc của người quản lý dự án. Công việc có thể thay đổi hoàn toàn phụ thuộc vào tổ chức và phần mềm phát triển. Tuy nhiên hầu hết các nhà quản lý chịu trách nhiệm các hoạt động dưới đây:

- **Lập kế hoạch dự án**: Người quản lý dự án có trách nhiệm lập kế hoạch, ước lượng lịch phát triển dự án và phân công công việc cho mọi người. Họ kiểm tra công việc bảo đảm rằng nó đang được vận hành theo đúng yêu cầu, giám sát tiến trình, kiểm tra việc phát triển đúng hạn và trong phạm vi kinh phí.
- **Báo cáo**: Người quản lý dự án thường phải làm báo cáo về tiến độ của dự án với khách hàng và với quản lý của tổ chức phát triển phần mềm. Họ phải có khả năng truyền tải thông tin trong phạm vi rộng các cấp độ từ góc độ kỹ thuật chi tiết cho đến tóm tắt quản lý. Họ phải viết các văn bản ngắn gọn, súc tích từ trừu

tượng cho tới chi tiết của bản báo cáo. Họ còn phải trình bày các thông tin này trong suốt quá trình xem xét tiến độ dự án.

- **Quản lý rủi ro**: Người quản lý dự án phải định lượng được rủi ro mà có thể ảnh hưởng đến dự án, xem xét rủi ro, và hành động khi sự cố xảy ra.
- **Quản lý nhân sự**: Người quản lý có trách nhiệm quản lý cả đội. Họ phải chọn ra nhân sự cho đội của họ và thiết lập cách làm việc để mang lại hiệu năng cao cho cả đội.
- **Viết dự thảo (proposal):** Giai đoạn đầu tiên trong dự án phần mềm liên quan đến việc viết bản dự thảo kế hoạch để lấy được hợp đồng công việc. Bản dự thảo mô tả các đối tượng của dự án và làm thế nào để nó được tiến hành. Nó thường bao gồm chi phí, lịch trình dự tính và lý giải tại sao hợp đồng dự án nên trao cho đội của mình. Viết dự thảo là công việc tối quan trọng vì sự sống còn của nhiều công ty phần mềm dựa trên số lượng dự án và hợp đồng được chấp nhận. Do đó không có sự hướng dẫn nào cho công việc này, việc viết dự thảo là kỹ năng mà chúng ta có được thông qua thực tế và kinh nghiệm.

a. Quản lý rủi ro

Quản lý rủi ro là một trong những công việc quan trọng của quản lý dự án. Quản lý rủi ro liên quan đến việc dự đoán rủi ro mà có thể ảnh hưởng đến lịch trình dự án hoặc chất lượng của phần mềm đang phát triển, sau đó có thể có các biện pháp ngăn chặn những rủi ro này. Có thể chia rủi ro làm 3 loại:

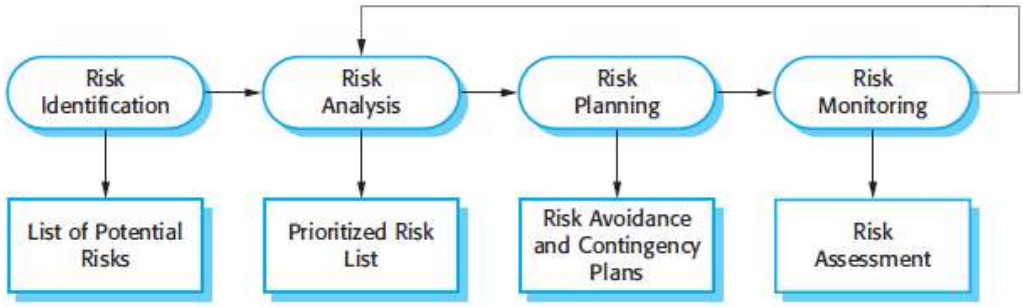
- Rủi ro dự án ảnh hưởng tới lịch trình và tài nguyên của dự án.
- Rủi ro sản phẩm: những rủi ro này ảnh hưởng đến chất lượng và hiệu suất của sản phẩm phát triển.
- Rủi ro kinh doanh: những rủi ro này ảnh hương đến tổ chức phát triển phần mềm

Một số rủi ro thường gặp khi phát triển mọi loại phần mềm:

Rủi ro	Đôi tượng chịu ảnh hưởng	Mô tả
Sự rút lui của đội ngũ nhân viên	Dự án	Đội ngũ kinh nghiệm rời dự án trước khi nó hoàn thành.

thay đổi về mặt quản lý	Dự án	Có sự thay đổi về mặt quản lý tổ chức với các độ ưu tiên khác nhau.
Phần cứng không sẵn sàng	Dự án	Phần cứng cần thiết cho dự án không được phân bổ theo lịch trình
Yêu cầu thay đổi	Dự án và sản phẩm	Số lượng lớn yêu cầu bị thay đổi
Độ trễ về kỹ thuật	Dự án và sản phẩm	Một số đặc tả giao diện không được đưa vào kế hoạch
Kích thước không dự đoán trước	Dự án và sản phẩm	Kích thước của dự án không dự đoán đúng
Công cụ hỗ trợ hoạt động hiệu suất thấp	Sản phẩm	Các công cụ hỗ trợ không hoạt động như dự tính
Kỹ thuật thay đổi	Kinh doanh	Nền tảng kỹ thuật mà phần mềm xây dựng đã bị vượt trội bởi công nghệ khác
Sản phẩm cạnh tranh	Kinh doanh	Sản phẩm cạnh tranh đã được xuất bản trên thị trường trước khi sản phẩm hoàn thiện.

Các bước quản lý rủi ro như sau:



Hình 1-7: Các bước quản lý rủi ro

- **Chỉ ra các rủi ro (Risk Identification)**: Chúng ta cần xác định những rủi ro có thể gặp phải về dự án, sản phẩm hay kinh doanh. Sản phẩm của giai đoạn này là danh sách rủi ro tiềm ẩn.

- **Phân tích rủi ro (Risk Analysis)**: Đánh giá khả năng và hậu quả những rủi ro này. Sản phẩm là danh sách đánh giá theo độ ưu tiên của nguy cơ rủi ro.
- **Kế hoạch rủi ro (Risk planning)**: Lên kế hoạch để giải quyết cá rủi ro bằng cách tránh né chúng hoặc giảm thiểu các ảnh hưởng của nó lên dự án. Sản phẩm của giai đoạn này là bản kế hoạch giảm thiểu rủi ro.
- **Giám sát rủi ro (Risk monitoring)**: Thường xuyên đánh giá rủi ro và kế hoạch giảm thiểu rủi ro, xem xét lại chúng khi chúng ta tìm hiểu thêm về các nguy cơ này. Sản phẩm của giai đoạn này là bản ước lượng rủi ro.

b. Quản lý con người

Nhân sự là tài sản lớn nhất của công ty phần mềm. Tốn rất nhiều chi phí để tuyển dụng và duy trì các nhân sự giỏi, và đặc biệt nó phụ thuộc vào người quản lý phần mềm có thể đảm bảo được lợi nhuận cho nhà đầu tư hay không. Đối với công ty thành công, nhân sự luôn tôn trọng tổ chức và được gắn liền với những công việc phản ánh đúng kỹ năng và kinh nghiệm của họ.

Thông thường, một nhà quản lý dự án phần mềm hiểu được các đặc điểm kỹ thuật ảnh hưởng đến công việc phát triển phần mềm. Tuy nhiên, ngược lại, một kỹ sư công nghệ phần mềm lại không cần thiết phải có khả năng quản lý nhân sự. Họ thường có kỹ năng tốt trong kỹ thuật nhưng lại thiếu kỹ năng mềm để thúc đẩy đội của làm việc tốt hơn. Một người quản lý dự án cần ý thức trước được các vấn đề tiềm ẩn này và cố gắng nâng cao khả năng quản lý con người.

Thông thường có 4 nhân tố tối quan trọng trong quản lý con người:

- **Thống nhất**: Mội người trong một đội cần nên có sự so sánh giữa các thành viên. Không ai mong đợi mọi người được phần thưởng như nhau cũng như sự cống hiến của họ không được đánh giá đúng.
- **Tôn trọng**: Mỗi người khác nhau sẽ có những kỹ năng khác nhau và người quản lý nên tôn trọng những điểm khác biệt này. Tất cả các thành viên của đội nên có cơ hội được cống hiến. Trong một vài trường hợp, chúng ta nhận thấy một vài nhân viên không phù hợp với đội, họ không thể tiếp tục làm việc chung, chúng ta cần nhanh chóng đưa ra quyết định sớm trong giai đoạn đầu của dự án.
- **Lắng nghe**: Mọi người sẽ làm việc hiệu quả hơn khi người ta cảm thấy được người khác lắng nghe, quan tâm, thực hiện dự định của họ. Một môi trường làm việc, nơi mà tất cả các quan điểm dù là của nhân viên mới được xem xét, là rất

quan trọng.

- **Trung thực**: Là một nhà quản lý, chúng ta nên luôn trung thực với những gì đang diễn ra tốt hay xấu trong dự án. Chúng ta cũng nên trung thực về các kiến thức kỹ thuật, vì nếu chúng ta che đậy, lúc mọi người phát hiện được chúng ta sẽ đánh mất sự tôn trọng của mọi người.

c. Làm việc nhóm

Tất cả các phần mềm chuyên nghiệp đều được phát triển bởi một nhóm khoảng từ 2 tới hàng trăm người. Một nhóm lớn sẽ chia thành nhiều nhóm nhỏ, mỗi nhóm chịu trách nhiệm phát triển một phần của dự án. Thông thường, một nhóm làm phần mềm không quá 10 người. Đối với nhóm nhỏ, các vấn đề về giao tiếp giảm xuống. Mọi người biết lẫn nhau, và có thể ngồi chung bàn để thảo luận các vấn đề về dự án trong khi phát triển.

Đặt mọi người vào một nhóm mà họ có kinh nghiệm, kỹ năng và tính cách cân bằng là nhiệm vụ tối quan trọng. Tuy nhiên thành công của nhóm không chỉ đơn giản là kết hợp các thành viên mà các thành viên này phải có sự kết dính và tinh thần đồng đội. Mọi người có liên quan đều được thúc đẩy bởi thành công của đội cũng như đạt được các mục tiêu cá nhân.

Trong một nhóm kết dính, các thành viên nghĩ về nhóm nhiều hơn là về cá nhân. Các thành viên luôn trung thành nhóm. Điều này làm cho nhóm luôn mạnh mẽ, có thể đương đầu với những vấn đề mà không lường trước được.

Có 3 nhân tố chung ảnh hưởng đến quá trình làm việc nhóm:

- **Nhân sự của nhóm**: Chúng ta cần trộn nhiều người trong nhóm phần mềm liên quan đến các hoạt động khác nhau như thảo luận với khách hàng, lập trình, kiểm thử và làm tài liệu.
- **Tổ chức nhóm**: Một nhóm nên được tổ chức sao cho các cá nhân thể hiện tốt nhất năng lực và nhiệm vụ của họ có thể hoàn thành như mong đợi.
- **Trao đổi về kỹ thuật và quản lý**: Sự trao đổi tốt giữa các thành viên và giữa các nhóm làm phần mềm với các bên liên quan là rất cần thiết.

TỔNG KẾT CUỐI CHƯƠNG

Ngành công nghiệp liên quan đến mọi khía cạnh sản xuất phần mềm gọi là ngành kỹ nghệ phần mềm. Các hoạt động chính trong phát triển phần mềm bao gồm đặc tả phần mềm, phát triển phần mềm, kiểm nghiệm phần mềm và tiến hóa phần mềm. Có

rất nhiều yếu tố ảnh hưởng đến phát triển phần mềm, công nghệ web là một trong những yêu tố khiến ngành công nghệ phần mềm có những bước chuyển nhảy vọt.

Tập hợp các hoạt động để sản xuất phần mềm gọi là chu trình sản xuất phần mềm. Có nhiều loại chu trình sản xuất phần mềm như chu trình thác nước, chu trình tái sử dụng tăng trưởng hay RUP. Việc lựa chọn quy trình nào phụ thuộc tổ chức sản xuất phần mềm, loại phần mềm và tổ chức mua phần mềm.

Một trong những yếu tố quyết định thành công của việc sản xuất phần mềm đó là quản lý dự án phần mềm. Công việc của người quản lý dự án phần mềm rất phức tạp, đòi hỏi người này phải có kiến thức rộng từ về mặt chi tiết kỹ thuật đến các mức trừu tượng. Các hoạt động chính của người quản lý dự án phần mềm là lập kế hoạch dự án, báo cáo, quản lý rủi ro, quản lý nhân sự và viết dự thảo.

CÂU HỎI ÔN TẬP CHƯƠNG 1

1.1. Định nghĩa phần mềm? Hãy nêu các loại phần mềm thường gặp?

1.2. Định nghĩa công nghệ phần mềm? Các yếu tô chung nền tảng trong công nghệ phần mềm? Các yếu tố ảnh hưởng đến phát triển phần mềm.

1.3. Hãy nêu các tiêu chí quan trọng đạo đức của người kỹ sư công nghệ phần mềm?

1.4. Nêu ý nghĩa và các trường hợp thường sử dụng các quy trình phần mềm trong chương 1? So sánh và đánh giá các quy trình này.

1.5. Hãy nên tầm quan trọng của người quản lý dự án phần mềm?

1.6. Giả sử xây dựng một phần mềm phòng chống khủng bố tiềm ẩn bằng cách theo giỏi và ghi nhận hoạt động của các nhóm người có nguy cơ cao. Việc xây dựng phần mềm này có vi phạm đạo đức nghề hay không? Tại sao?

Chương 2: KHẢO SÁT-PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU

Như đã giới thiệu trong chương 1, giai đoạn khảo sát và phân tích để xác định dự án phần mềm có tính khả thi trong phạm vi công nghệ và kinh phí cho phép. Nắm bắt các yêu cầu tổng quát cho đến chi tiết của phần mềm để làm tài liệu cho bộ phận thiết kế phần mềm. Trong chương này chúng ta sẽ tìm hiểu và vai trò của bước khỏa sát và phân tích và đặc tả yêu cầu, các kỹ thuật thu thập và mô hình hóa các yêu cầu này. Sau khi kết thúc chương này, người học được trang bị đầy đủ các kiến thức để phân tích yêu cầu của một phần mềm.

2.1. VAI TRÒ CỦA PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU

Phân tích và đặc tả là giai đoạn đầu trong quá trình sản xuất phần mềm. Nó có vai trò quan trọng, quyết định sự thành công của dự án phần mềm.

Vai trò của giai đoạn này:

- Xác định quy trình nghiệp vụ của hệ thống. Nhà phân tích gặp gỡ khách hàng, tiếp xúc với hệ thống thực tế, quan sát và nghiên cứu tài liệu để xác định tổng thể quy trình nghiệp vụ hệ thống.
- Xác định khả thi dự án phần mềm. Trước khi tiến hành các nghiên cứu các yêu cầu chi tiết của hệ thống, nhà phân tích ước lượng được phạm vi của hệ thống phần mềm, khả năng thực hiện phần mềm trong phạm vi công nghệ, thời gian, kinh phí, cơ sở hạ tầng mà đối tác có thể thực hiện được. Từ đó cho ra kết luận tính khả thi của dự án phần mềm.
- Xác định yêu cầu chức năng, phi chức năng của hệ thống. Nhà phân tích sử dụng các kỹ thuật như quan sát, phỏng vấn, nghiên cứu tài liệu… để xác định đầy đủ các yêu cầu của hệ thống.
- Chi tiết và chuẩn hóa các yêu cầu thông qua hoạt động mô hình hóa nghiệp vụ, từ xác định thêm các chức năng cần thiết đảm bảo hoạt động tốt cho hệ thống phần mềm. Trong hoạt động này, nhà phân tích thường sử dụng các sơ đồ như sơ đồ hoạt động, sơ đồ ca sử dụng để mô hình hóa.

2.2. PHÂN LOẠI YÊU CẦU

Trong quá trình thực hiện hợp đồng, người ta thường viết bản mô tả phần mềm ở dưới dạng trừu tượng hóa, đơn giản hóa để mô tả ngắn gọn yêu cầu để người dùng có thể nhanh chóng hiểu về phần mềm và chấp nhận hợp đồng. Sau khi hợp đồng đã được thỏa thuận, các yêu cầu về phần mềm sẽ được viết chi tiết hơn, để người dùng có thể

kiểm định lại chức năng mà phần mềm cung cấp có thỏa mãn nhu cầu người dùng chưa, và là tài liệu giúp người dùng xác định phần mềm sau khi người dùng cung cấp đúng theo hợp đồng hay chưa. Như vậy cách mô tả yêu cầu phần mềm chia làm 2 loại, yêu cầu ở mức trừu tượng gọi là yêu cầu người dùng, yêu cầu ở mức chi tiết gọi là yêu cầu hệ thống.

- Yêu cầu người dùng là những lời lẽ được viết bằng ngôn ngữ tự nhiên, cùng với các sơ đồ mô tả về những dịch vụ mà phần mềm cung cấp cho hệ thống người dùng và các ràng buộc về phần mềm trong lúc hoạt động.
- Yêu cầu hệ thống là các bản mô tả chi tiết hơn về chức năng, dịch vụ và các quy định của hệ thống. Bản yêu cầu hệ thống cần phải mô tả chính xác những gì được cài đặt trong hệ thống vì nó là một phần của hợp đồng phần mềm.

Đối tượng người đọc cũng khác nhau giữa hai cách thể hiện yêu cầu này:

Yêu cầu	Người đọc
Yêu cầu người dùng	Người quản lý khách hàng Người dùng cuối hệ thống Các khách hàng Người quản lý hợp đồng Kiến trúc sư hệ thống
Yêu cầu hệ thống	Người dùng hệ thống cuối Các khách hàng Kiến trúc sư hệ thống Người phát triển hệ thống

Ví dụ trong hệ thống quản lý bệnh án chúng ta có yêu cầu sau:

- Yêu cầu người dùng:

1. Hệ thống quản lý bệnh nhân sẽ xuất báo cáo quản lý hàng tháng về chi phí thuốc đã sử dụng của mỗi quầy trong tháng vừa qua.

- Đặc tả yêu cầu hệ thống:

1.1. Trong ngày làm việc cuối cùng của mỗi tháng, một bảng tóm tắt về thuốc đã kê đơn và chi phí của nó được tạo ra.

1.2. Hệ thống sẽ tự động tạo báo cáo để in sau 17h30 vào ngày cuối cùng mỗi tháng

1.3. Một báo cáo được tạo ra cho mỗi phòng khám và sẽ được liệt kê chi tiết tên thuốc, tổng số lượt kê đơn, tổng số liều lượng sử dụng và tổng giá trị thuốc.

1.4. Nếu thuốc có nhiều liều lượng sử dụng khác nhau thì các báo cáo khác nhau tạo ra cho các liều lượng này.

1.5. Quyền truy cập xem tất cả các báo cáo này sẽ được hạn chế theo quyền của người dùng đã đươc liệt kê trong danh sách quản lý truy cập người dùng.

Yêu cầu của phần mềm thường chia làm 2 loại đó là yêu cầu chức năng và yêu cầu phi chức năng.

- Yêu cầu chức năng là những mô tả về dịch vụ mà hệ thống phải cung cấp, hệ thống phải tương tác như thế nào với dữ liệu đầu vào hay phải ứng xử thế nào trong những tình huống nhất định và có thể liệt kê những gì không nên làm.
- Yêu cầu phi chức năng là những ràng buộc về dịch vụ hoặc các chức năng mà hệ thống cung cấp như ràng buộc về thời gian, về cách thức phát triển, và các tiêu chuẩn chất lượng. Những yêu cầu phi chức năng thường áp dụng cho toàn bộ hệ thống thay vì từng chức năng hay dịch vụ nhỏ.

Trong thực tế, 2 loại yêu cầu này không phân biệt một cách rạch ròi như định nghĩa ở trên. Ví dụ trong yêu cầu người dùng có yêu cầu liên quan đến bảo mật, hạn chế người dùng truy cập tới nội dung nào đó và nó là yêu cầu phi chức năng. Nhưng trong định nghĩa chi tiết hơn trong đặc tả yêu cầu hệ thống, cần phải cài đặt thêm chức năng kiểm tra người dùng. Điều này chứng tỏ các yêu cầu không độc lập lẫn nhau, một yêu cầu này thường liên quan đến yêu cầu khác. Vì vậy yêu cầu hệ thống không chỉ đưa ra các dịch vụ hoặc tính năng mà hệ thống đòi hỏi, nó còn chỉ ra những chức năng cần phải có để đảm bảo các dịch vụ và tính năng của hệ thống đã nêu trên hoạt động tốt.

2.2.1. Yêu cầu chức năng

Yêu cầu chức năng của một hệ thống mô tả các công việc mà hệ thống nên làm. Những yêu cầu này phụ thuộc vào loại phần mềm phát triển, người dùng của phần mềm và phương pháp viết đặc tả. Khi mô tả những yêu cầu này, chúng thường được miêu tả một cách trừu tượng để người dùng hệ thống có thể hiểu được. Khi viết chi tiết hơn về các chức năng, các yêu cầu này cần mô tả dữ liệu đầu vào, dữ liệu đầu ra, các ngoại lệ …

Yêu cầu chức năng hệ thống bao trùm từ những yêu cầu chung nhất của hệ thống cho đến những yêu cầu hệ thống cụ thể phải làm gì, phản ánh các hoạt động hàng ngày thông thường của hệ thống thực tế.

Ví dụ đây là các yêu cầu chức năng của hệ thống quản lý bệnh nhân dùng để duy trì thông tin bệnh nhân sử dụng dịch vụ điều trị tâm lý:

1. Người dùng có thể tìm kiếm danh sách hẹn cho mọi phòng khám
2. Mỗi ngày, tại mỗi phòng khám, hệ thống sẽ tạo ra danh sách các bệnh nhân có cuộc hẹn trong ngày

Các yêu cầu trên được trích ra từ bản phân tích yêu cầu người dùng, rõ ràng chúng cần được mô tả chi tiết hơn để có thể thống nhất giữa người dùng và người phát triển. Sự thiếu chính xác trong đặc tả yêu cầu có thể gây ra rất nhiều vấn đề trong quá trình phát triển về sau. Ví dụ trong yêu cầu 1 ở trên, người dùng tại sao lại có thể tìm kiếm danh sách hẹn trong mọi phòng khám? Đối với bệnh nhân có vấn đề về tâm thần, người ta có thể hẹn ở một phòng khám này, nhưng lại đến một phòng khám khác. Thông thường, người phân tích dễ nhầm lẫn việc tìm kiếm chỉ thực hiện trên phòng khám mà nhân viên đó làm việc. Những sự mơ hồ trong đặc tả yêu cầu có thể phát sinh những thay đổi sau khi chương trình được chuyển giao, điều này làm chậm quá trình hoàn thành phần mềm và tăng chi phí đáng kể.

Về nguyên tắc, đặc tả yêu cầu chức năng của hệ thống cần phải hoàn chỉnh và nhất quán. Hoàn chỉnh có nghĩa rằng tất các dịch vụ mà người dùng đòi hỏi phải được định nghĩa. Nhất quán có nghĩa rằng các yêu cầu đó thống nhất, không mâu thuẫn lẫn nhau. Trong thực tế, đối với một hệ thống lớn và phức tạp, khó có thể đạt được một bản phân tích hoàn chỉnh và nhất quán. Một nguyên nhân là rất dễ gây lỗi trong quá trình viết, soạn thảo. Ngoài ra có rất nhiều bên liên quan, mà chính nhu cầu của họ đôi khi lại mâu thuẫn lẫn nhau. Vấn đề có thể chỉ thấy được sau quá trình phân tích lâu dài hoặc sau khi chuyển giao cho khách hàng.

2.2.2. Yêu cầu phi chức năng

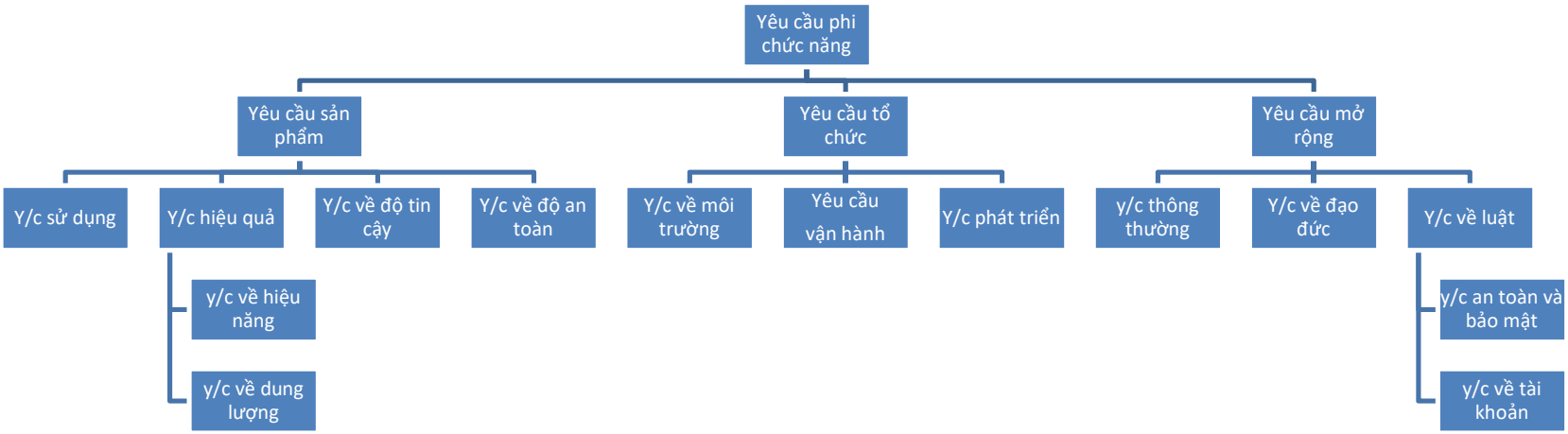
Yêu cầu phi chức năng là những yêu cầu không trực tiếp liên quan đến một dịch vụ nhất định nào đó mà hệ thống cung cấp cho người dùng. Chúng có thể liên quan đến những thuộc tính quan trọng như độ tin cậy, thời gian hồi đáp hay công xuất lưu trữ. Ngoài ra, người ta còn có thể định nghĩa những ràng buộc liên quan đến việc cài đặt như khả năng sử dụng các thiết bị nhập xuất, và giao tiếp dữ liệu với các hệ thống khác.

Những yêu cầu phi chức năng như hiệu năng, độ an toàn, khả năng hoạt động liên tục thường chỉ ra các thuộc tính của toàn bộ hệ thống. Chúng thường khắt khe hơn các yêu cầu chức năng. Người dùng hệ thống có thể xoay sở nếu một chức năng hệ thống nào đó không hoạt động đúng. Nhưng nếu một yêu cầu phi chức năng không đạt yêu cầu, ví dụ tính an toàn, hoặc hiệu năng của hệ thống không đạt yêu cầu, thì cả hệ thống không hoạt động được và trở nên vô dụng.

Việc xác định yêu cầu chức năng thông thường đơn giản hơn so với xác định các yêu cầu phi chức năng, và việc cài đặt cũng tương tự. Hai lý do chính là:

- Yêu cầu phi chức năng có thể ảnh hưởng đến toàn bộ kiến trúc của hệ thống chứ không chỉ một chức năng riêng lẻ. Ví dụ để đảm bảo hiệu năng chương trình như yêu cầu, chúng ta tổ chức làm sao cho giảm thiểu việc trao đổi thông tin giữa các thành phần phần mềm.
- Một yêu cầu phi chức năng có thể đòi hỏi phải cài đặt rất nhiều các chức năng để thực hiện. Ví dụ yêu cầu về độ an toàn có thể dẫn đến hàng loạt các yêu cầu chức năng phát sinh như hệ thống kiểm soát người dùng, hệ thống xác thực một cửa v.v.. Ngoài ra, có những yêu cầu này lại hạn chế các yêu khác. Ví dụ độ bảo mật cao có thể làm tăng thời gian thực hiện giao dịch, vô tình làm giảm tính hiệu năng của chương trình.

Các yêu cầu phi chức năng sinh ra phụ thuộc vào nhu cầu người dùng và chịu ảnh hưởng bởi kinh phí, các quy định của tổ chức, và sự tương tác với các hệ thống phần mềm hoặc các nhân tố bên ngoài. Sơ đồ phân loại yêu cầu phi chức năng:



Hình 2-1: Yêu cầu phi chức năng

Từ sơ đồ ta có thể thấy những yêu cầu phi chức năng bắt nguồn từ các tính chất của phần mềm hay yêu cầu phần mềm, của tổ chức phát triển phần mềm và từ các nguồn mở rộng:

- **Yêu cầu phần mềm**: Những yêu cầu này thường chỉ ra các ràng buộc cách thức hoạt động phần mềm. Ví dụ những yêu cầu về hiệu năng thể hiện trên khả năng thời gian hệ thống thực thi, cần bao nhiêu bộ nhớ. Yêu cầu về độ tin cậy chỉ ra tỉ lệ lỗi mà hệ thống gặp phải …
- **Yêu cầu tổ chức**: Những yêu cầu này mở rộng từ yêu cầu hệ thống đến các điều luật của tổ chức khách hàng hay tổ chức phát triển phần mềm. Ví dụ những yêu cầu này bao gồm các tiến trình hoạt động xác định hệ thống vận hành như thế nào, môi trường và tiêu chuẩn phát triển phần mềm, ngôn ngữ lập trình hay hệ điều hành vận hành hệ thống.
- **Yêu cầu mở rộng**: Các yêu cầu này bao trùm các nhân tố bên ngoài hệ thống. Nó bao gồm các yêu cầu thông thường như xử lý dữ liệu ngân hàng, các yêu cầu pháp luật đảm bảo hệ thống hoạt động hợp pháp, có đạo đức và đảm bảo hệ thống được chấp nhận bởi người dùng và cộng đồng nói chung.

- Yêu cầu phần mềm: Hệ thống phải hoạt động suốt thời gian làm việc từ 8h30 đến 5h30 mỗi ngày. Thời gian ngưng hoạt động trong giờ làm việc không quá 5 giây trong một ngày.
- Yêu cầu tổ chức: Người dùng của hệ thống cần phải được xác thực bởi thẻ y tế của họ.

Ví dụ các yêu cầu phi chức năng của hệ thống quản lý bệnh nhân:

Các yêu cầu phi chức năng thông thường được nêu ra rất chung chung. Ví dụ yêu cầu chương trình phải hoạt động nhanh, hiệu quả, người dùng dễ sử dụng. Điều này gây trở ngại nhiều cho người phát triển. Người phân tích nên cụ thể hóa các tiêu chí. Ví dụ thời gian thực hiện một tiến trình tối đa 5 giây. Nhân viên y tế có thể sử dụng hệ thống sau 5 tiếng tập huấn … Bảng dưới đây thể hiện một số cách thức cụ thể hóa các yêu cầu phi chức năng.

Tiêu chí	Thể hiện
Tốc độ	Thời gian xử lý giao dịch (bao nhiêu giây) Thời gian phản hồi người dùng Thời gian làm tươi màn hình
Dung lượng	Bao nhiêu MB

	Số lượng Rom chip
Độ tin cậy	Thời gian lỗi Khả năng không hoạt động Tỉ lệ xảy ra lỗi Thời gian thực hiện được
Độ mạnh mẽ	Thời gian khởi động lại sau khi lỗi Tỉ lệ sự kiện gây ra lỗi Khả năng hư dữ liệu khi bị lỗi
Khả năng di chuyển	Số lượng đối tượng của hệ thống Tỉ lệ đối tượng độc lập

2.3. TÀI LIỆU PHÂN TÍCH YÊU CẦU

Tài liệu phân tích yêu cầu hay còn gọi là tài liệu đặc tả là tài liệu chính thống mô tả những gì hệ thống sẽ được phát triển và cài đặt. Nó bao gồm yêu cầu người đối với hệ thống, và các đặc tả chi tiết của hệ thống. Thỉnh thoảng đặc tả người dùng và hệ đặc tả hệ thống được tính hợp thành một, hoặc bản đặc tả yêu cầu người dùng được đưa vào phần giới thiệu. Nếu có số lượng lớn yêu cầu thì phần đặc tả chi tiết được thể hiện ở phần riêng.

Bản phân tích yêu cầu cần có đầy đủ thông tin để trao đổi với khách hàng, đồng thời cũng đủ chi tiết chính xác để người phát triển có thể lập trình, kiểm lỗi và mở rộng về sau. Đặc biệt đối với phần mềm được phát triển bởi nhiều công ty khác nhau, thì bản phân tích yêu cầu này cần phải chính xác và chi tiết hơn so với một công ty đảm nhiệm toàn bộ công việc.

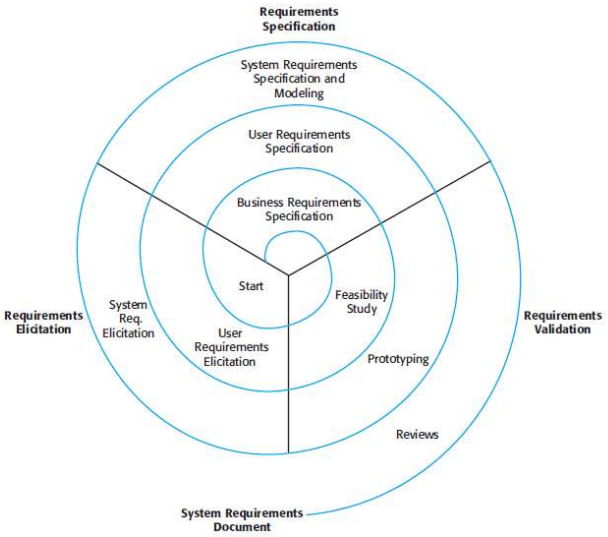
Mỗi tổ chức phần mềm có quy định về bản đặc tả, tuy nhiên thông thường nó bao gồm những phần sau:

Chương	Mô tả
Lời nói đầu	Xác định đối tượng người đọc, lịch sử các phiên bản và tóm tắt các thay đổi theo từng phiên bản
Giới thiệu	Diễn tả nhu cầu của hệ thống một cách ngắn gọn. Trong đó có giới thiệu hệ thống vận hành như thế nào, giao tiếp với môi trường, và mang lại lợi ích gì cho tổ chức sử dụng.
Cụm từ viết tắt	Giới thiệu các cụm từ viết tắt, các thuật ngữ
Yêu cầu người dùng	Nội dung bao gồm các yêu cầu của người dùng, bao gồm cả những yêu cầu chức năng và phi chức năng. Phần này được viết

	bằng ngôn ngữ tự nhiên hoặc các sơ đồ ký hiệu mà người dùng có thể hiểu được. Các tiêu chuẩn, tiêu chí sản phẩm cũng nên được liệt kê trong phần này.
Kiến trúc hệ thống	Phần này nên giới thiệu về tổng quát về kiến trúc hệ thống, sự phân phối chức năng qua các mô đun. Các thành phần nào tái sử dụng nên được đánh dấu.
Đặc tả hệ thống	Phân này mô tả chi tiết các yêu cầu chức năng và phi chức năng. Nếu có giao tiếp với hệ thống khác, trong phần này cũng bao gồm giao diện của hệ thống với hệ thống đó.
Mô hình hệ thống	Phần này bao gồm các mô hình mang tính đồ họa thể hiện quan hệ giữa các thành phần hệ thống, hệ thống và môi trường xung quanh. Có thể sử dụng như mô hình đối tướng, mô hình luồng dữ liệu…
Tiến hóa phần mềm	Mô tả những giả thuyết thay đổi hệ thống khi có sự thay đổi về phần cứng, nhu cầu người dùng …Điều này giúp tránh những thiết kế mà hạn chế khả năng thay đổi hệ thống trong tương lai.
Phụ lục	Phần này cung cấp các thông tin chi tiết liên quan đến thông tin liên quan các ứng dụng được phát triển ví dụ như phần cứng, mô tả cơ sở dữ liệu. Phần cứng tối thiểu hay tối ưu nhất cho hệ thống. Các định nghĩa logic dữ liệu được sử dụng bởi hệ thống và mối quan hệ giữa các dữ liệu.
Chỉ mục	Đây là tài liệu dài nên phần chỉ mục là cần thiết, các từ khóa đánh dấu theo thứ tự chữ cái.

2.4. CÁC HOẠT ĐỘNG PHÂN TÍCH VÀ ĐẶC TẢ YÊU CẦU

Quy trình này bao gồm 4 hoạt động chính: kiểm tra hệ thống có mang lại lợi ích cho doanh nghiệp và có tính khả thi, phát hiện yêu cầu (thu thập và phân tích), chuyển các yêu cầu sang dạng chuẩn (đặc tả), và kiểm tra những yêu cầu đó có thực sự là những gì khách hàng mong muốn (kiểm tra).



Hình 2-2: Các hoạt động đặc tả

Các hoạt động của quy trình này là các hoạt động lặp lại đan xen như hình xoắn ốc ở trên. Giai đoạn đầu của quy trình tập trung vào việc hiểu quy trình nghiệp vụ tổng quát của hệt thống, các yêu cầu phi chức năng, và yêu cầu người dùng nói chung. Sau đó chúng ta tập trung vào việc thu thập và hiểu các yêu cầu hệ thống chi tiết. Số lần lập lại phụ thuộc vào tổ chức, độ phức tạp yêu cầu tại tổ chức mà hệ thống được xây dựng.

Trong bản phân tích yêu cầu, yêu cầu người dùng thông thường được viết bằng ngôn ngữ tự nhiên. Ngoài ra, những phần còn lại có thể sử dụng ngôn ngữ có cấu trúc, biểu bảng, các mô hình hệ thống, ký hiệu hình ảnh hoặc công thức toán học …

2.4.1. Xác định tính khả thi

Trước khi thực hiện tìm hiểu yêu cầu chi tiết của hệ thống, cần phải ước tính sơ lược tính khả thi của dự án. Một số bước chuẩn bị như xác định về cơ sở hạ tầng, hệ thống hiện tại. Những nhược điểm mà hệ thống hiện tại đang gặp phải, quy mô và mong muốn hệ thống trong tương lại. Ước lượng lợi nhuận mà hệ thống mới mang lại bao gồm lợi ích về mặt thời gian (các quy trình mới tối ưu hơn), nhân sự (giảm bớt nhân sự do sự tin học hóa), văn phòng phẩm … và chi phí xây dựng và duy trì phần mềm. Các mong muốn về hệ thống mới có thể thực hiện được với cơ sở hạ tầng hiện tại hay không? Thời gian xây dựng hệ thống là bao nhiêu? Trong khoảng thời gian đó, yêu cầu đã có thể thay đổi nhiều không? Tất cả các kết quả trên chỉ mang tính tương đối, tuy nhiên cũng có thể ước lượng được tính khả thi của hệ thống trước khi đi vào tìm hiểu chi tiết yêu cầu.

2.4.2. Thu thập và phân tích hệ thống

Sau khi nghiên cứu về tính khả thi, bước tiếp theo là thu thập và phân tích yêu cầu. Trong hoạt động này, kỹ sư phần mềm làm việc cùng với khách hàng và người dùng cuối để tìm hiểu về ứng dụng và lĩnh vực liên quan, những dịch vụ hệ thống cung cấp, các yêu về hiệu năng hệ thống, và phần cứng …

Thứ tự thực hiện trong quá trình này là:

- **Phát hiện các yêu cầu**: trao đổi với các bên liên quan của hệ thống. Ở giai đoạn này chúng ta sử dụng một số kỹ thuật để thu thập yêu cầu như phỏng vấn, nghiên cứu tài liệu. Chúng ta có thể sử dụng các mẫu thử, các giả thiết để khách hàng có thể hiểu hệ thống sẽ như thế nào.
- **Phân loại yêu cầu**: xác định yêu cầu nào là yêu cầu chức năng, phi chức năng, nhóm các chức năng gần lại.
- **Thảo luận và đánh giá**: Đánh giá độ ưu tiên của các yêu cầu, sắp xếp và thảo luận lại các yêu cầu chưa rõ.
- **Mô tả chi tiết**: Đặc tả từng yêu cầu, yêu cầu người dùng, yêu cầu hệ thống, yêu cầu phi chức năng và yêu cầu chức năng.

Trong quá trình xác định yêu cầu, phỏng vấn là kỹ thuật không thể bỏ qua. Phỏng vấn được chia làm 2 loại:

- **phỏng vấn đóng**: là kiểu phỏng vấn mà khách hàng trả lời một số câu hỏi đã có sẵn
- **phỏng vấn mở**: không có câu hỏi trước, đội thu thập yêu cầu sẽ cùng với các bên liên quan cùng thảo luận một loạt vấn đề để hai bên cùng hiểu rõ nhu cầu về phần mềm.

Trong thực tế các cuộc phỏng vấn thường kết hợp hai loại trên. Sau khi trả lời các câu hỏi có sẵn, thông thường nảy sinh một số vấn đề cần thảo luận.

Phỏng vấn là phương pháp dễ nhất để có cái nhìn tổng quát về nghiệp vụ của tổ chức và các bên liên quan, họ tương tác với hệ thống như thế nào, những khó khăn mà họ gặp phải với hệ thống hiện tại. Thông thường người được phỏng vấn khá hào hứng khi họ nói về công việc của họ. Tuy nhiên phương pháp phỏng vấn không giúp tìm hiểu được yêu cầu về ứng dụng phần mềm. Đối với họ, công việc hàng ngày trở nên quá quen thuộc, họ dễ dàng bỏ qua những bước quá cơ bản, và sử dụng nhiều thuật ngữ làm kỹ sư công nghệ phần mềm hiểu lầm.

Ngoài hình thức phỏng vấn, người ta còn sử dụng phương pháp hoạt cảnh (Scenarios). Phương pháp này mô tả lại một tình huống trong thực tế, các tương tác của người dùng với hệ thống có thể thể hiện chi tiết, từ đó kỹ sư công nghệ phần mềm có thể thu thập được đầy đủ các yêu cầu thực tế và cũng hiểu rõ về nghiệp vụ hệ thống trong thực tế hơn. Một hoạt cảnh bao gồm:

- Mô tả về hệ thống và những người dùng nhắm đến của hệ thống bắt đầu.
- Một phần mô tả chuỗi sự kiện thông thường của hệ thống.
- Một phần mô tả những sự kiện sai lệch và cách thức xử lý.
- Thông tin các hoạt động khác có thể xảy ra đồng thời.
- Phần mô tả hệ thống khi hoạt cảnh kết thúc.

Ví dụ hoạt cảnh thu thập lịch sử trong hệ thống chữa bệnh tâm thần:

Ca sử dụng (use cases) là phương pháp phát hiện yêu cầu được giới thiệu đầu tiên năm 1993, và bây giờ là công cụ quan trọng trong ngôn ngữ UML. Ở hình thức đơn giản nhất, môt ca sử dụng chỉ ra các tác nhân trong tương tác và tên của các tương tác. Những thông tin thêm có thể được mô tả dạng chữ viết hoăc các sơ đồ tuần tự, sơ đồ trạng thái UML.

Tập hợp các use case trong sơ đồ thể hiện tất cả những tương tác có thể xảy ra và được mô tả trong hệ thống. Các tác nhân trong tiến trình có thể là con người hoặc hệ thống khác. Tên của các tương tác được ghi trong hình eclipse.

Giả thiết ban đầu:

Bệnh nhân đã gặp nhân viên y tế, các thông tin của họ đã được lưu trữ vào hệ thống. Hiện tại, một y tá đăng nhập vào hệ thống để lấy thông tin bệnh nhân

Thông thường:

Y tá tra cứu thông tin bệnh nhân thông qua họ, nếu có nhiều người thì dùng tiếp tên, ngày sinh. Y tá chọn menu để thêm thông tin khám bệnh cho bệnh nhân. Theo một loạt các hướng dẫn từ hệ thống, y tá thêm thông tin về kết luận bệnh nhân tâm thần như thuốc hiện đang sử dụng, dị ứng thuốc …

Trường hợp bất thường:

Thông tin bệnh nhân không tìm thấy, y tá sẽ thêm một bản ghi mới về thông tin

bệnh nhân.

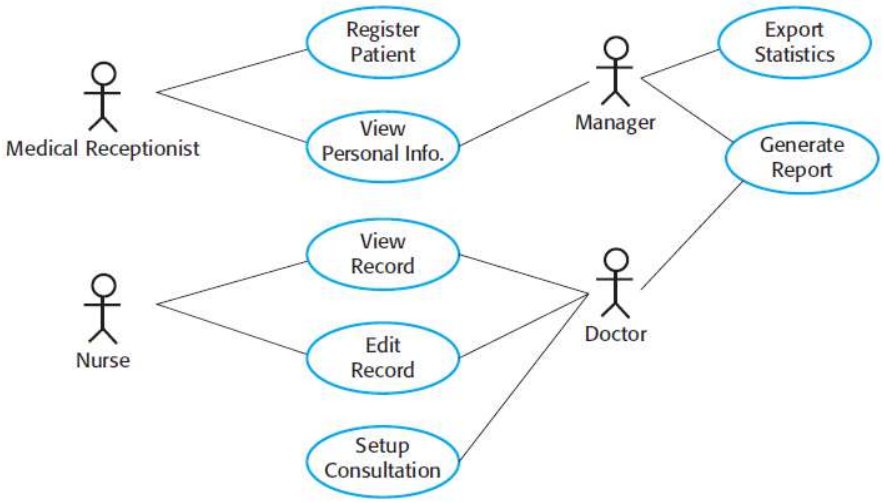
Trường hợp bệnh nhân không thể, không muốn cung cấp thông tin về tình trạng, lịch sử điều trị, y tá phải ghi lại vào phần lịch sử điều trị rằng bệnh nhân không cung cấp thông tin. Hệ thống phải in ra một biên bản sự thiếu thông tin này, và nó là nguyên nhân làm cho việc điều trị bị trì hoãn và đưa cho bệnh nhân ký xác nhận.

Hoạt động khác:

Các thông tin của bệnh nhân có thể được kiểm tra nhưng không được điều chỉnh bởi người khác trong khi thông tin đang được thêm vào.

Giai đoạn kết thúc:

Người dùng đã đăng nhập. Bản ghi về lịch sử khám chữa bệnh của bệnh nhân đã được thêm vào cơ sở dữ liệu, một bản ghi khác được thêm vào nhật ký hệ thống ghi nhận thời gian bắt đầu và thời gian kết thúc phiên làm việc của y tá.



Hình 2-3: Sơ đồ ca sử dụng thiết lập buổi khám bệnh nhân

Mỗi một ca sử dụng nên đính kèm một bản mô tả, nó có thể liên kết với các sơ đồ khác của UML (sơ đồ tương tác) phát triển thành một hoạt cảnh với các thông tin chi tiết.

Ví dụ như bảng mô tả chi tiết đi kèm với ca sử dụng thiết lập kết luận khám bệnh:

Ca sử dụng thiết lập kết luận khám bệnh cho phép một hoặc nhiều bác sĩ làm việc trong nhiều văn phòng khác nhau cùng xem một thông tin bản ghi của bệnh nhân cùng lúc. Bác sĩ đầu tiên cho ra kết luận bằng cách lựa chọn người liên quan từ menu xổ xuống danh sách các bác sỉ trực tuyến. Sau đó thông tin của bệnh nhân được hiển thị trên màn hình nhưng chỉ có bác sĩ đầu tiên mới có thể chỉnh sửa thông tin. Thêm vào đó, một cửa sổ nói chuyện được tạo ra để giúp họ thảo luận. Họ có thể thiết lập phòng họp trực tuyến riêng biệt nếu cần thiết để thảo luận.

Phương pháp hoạt cảnh hoặc ca sử dụng rất hiệu quả trong việc thu thập yêu cầu chức năng, nhưng tỏ ra kém hiệu quả trong việc tìm kiếm các yêu cầu phi chức năng và có cái nhìn cấp cao về quy trình nghiệp vụ.

2.4.3. Chuyển đổi yêu cầu sang dạng chuẩn

Sau khi đã thực hiện các kỹ thuật xác định yêu cầu, người kỹ sư công nghệ phần mềm cần phân loại các loại yêu cầu (chức năng và phi chức năng), tổ chức các yêu cầu và trình bày chúng theo cấu trúc của bản phân tích yêu cầu như ở phần trước đã đề nghị. Sử dụng các sơ đồ, hình ảnh khi cần thiết để tăng tính hiệu quả trong việc truyền tải thông tin. Thông thường sử dụng sơ đồ activity để mô hình quy trình nghiệp vụ, và phân tích rõ hơn bằng bảng mô tả ca sử dụng và sơ đồ ca sử dụng.

Nếu các bước thu thập và lập bản phân tích yêu cầu được lặp lại, các phiên bản khác nhau sẽ ra đời, phiên bản sau bổ sung thêm và tinh chỉnh các yêu cầu phiên bản trước.

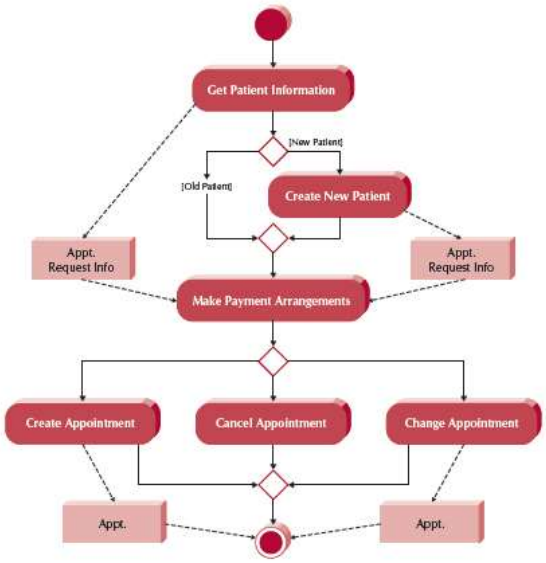
a. Sơ đồ hoạt động (activity diagram)

Sơ đồ hoạt động dùng để mô tả các hoạt động chính và mối quan hệ giữa các hoạt động trong một quá trình.

Sơ đồ hoạt động được sử dụng để mô hình hóa các hành động của tiến trình nghiệp vụ của các đối tượng độc lập. Nhìn một cách khác, sơ đồ hoạt động có thể coi như sơ đồ luồng dữ liệu kết hợp với việc phân tích cấu trúc. Tuy nhiên, không giống như sơ đồ luồng dữ liệu, sơ đồ hoạt động bao gồm các khái niệm chỉ ra cách thức mô hình hóa các hành động xảy ra song song, đồng thời để mô hình hóa mọi thứ từ luồng nghiệp vụ mức cao cho đến chi tiết các phương thức riêng lẻ. Sơ đồ hoạt động có thể mô hình hóa mọi tiến trình. Trong trường hợp này ta sử dụng sơ đồ hoạt động để mô hình hóa tiến trình nghiệp vụ nâng cao.

Các thành phần sơ đồ hoạt động:

Nội dung	Ký hiệu
Hành động: là các hoạt động không chia nhỏ hơn nữa và được đặt tên bởi hành động đó	Action
Hoạt động: được dùng để thể hiện một tập các hành động và được đặt tên bởi tên của của hoạt động đó	Activity
Một nút đối tượng: Thể hiện một đối tượng liên quan đến tập các dòng chảy đối tượng (object flow) và được đặt tên bởi tên của đối tượng đó.	Class Name
Dòng điều khiển (control flow): Thể hiện thứ tự thực hiện	
Dòng đối tượng (object flow): Thể hiện dòng của đối tượng từ một hoạt động này đến một hoạt động khác	
Nút gốc (initial node): Mô tả điểm bắt đầu của một loạt các hoạt động	
Nút cuối (final-activity node): được sử dụng để dừng lại tất cả các dòng điều khiển của các hoạt động.	
Nút cuối của dòng (final –flow node): sử dụng để dùng một dòng điều khiển hoặc một dòng đối tượng.	
Nút quyết định (decision node): được sử dụng để thể hiện việc kiểm tra điều kiện đảm bảo dòng điều khiển hoặc dòng đối tượng đi xuống theo một hướng và được đặt tên bởi điều kiện đó.	[Decision Criteria] [Decision Criteria]
Nút trộn (Merge node): được sử dụng để kết hợp lại các đường quyết định khác nhau sau khi sử dụng nút quyết định.	
Nút ngã ba (Fork node): Được sử dụng để chia các hành động thành một loạt các hành động, hoạt động song song hoặc đồng thời.	
Nút kết hợp (Join node): Được sử dụng để kết hợp lại các hoạt động song song hoặc đồng thời sau khi đã dùng nút ngã ba.	
Làn (Swimlane): Được sử dụng để chia sơ đồ hoạt động thành các dòng, các cột, để kết hợp các hoạt động riêng lẻ cho các đối tượng, cá nhân có quan hệ với hoạt động đó và được đặt tên bởi đối tượng hay cá nhân đó.	Name



Hình 2-4: Sơ đồ hoạt động thiết lập buổi khám

Ví dụ:

Sơ đồ hoạt động bên thể hiện quy trình thiết lập một cuộc hẹn với bác sĩ. Đầu tiên tiếp viên thu thập thông tin bệnh nhân. Nếu là bệnh nhân mới thì tạo một bệnh nhân mới trong hệ thống, nếu đã có thì bỏ qua bước này. Từ thông tin thu thập về cuộc hẹn, sắp xếp một cuộc hẹn mới với bác sĩ. Sau đó có thể có 3 trường hợp, bệnh nhân gặp bác sĩ, bệnh nhân hủy cuộc hẹn, hoặc bệnh nhân đổi thời gian hẹn. Trong hai trường hợp đầu và cuối, có một cuộc hẹn thực sự xảy ra. Cuối cùng là kết thúc một loạt các hoạt động và hoàn thành quy trình lập buổi hẹn với bác sĩ.

Các chu ý khi lập sơ đồ hoạt động:

- Bởi vì sơ đồ hoạt động có thể sử dụng để mô hình hóa mọi loại tiến trình nên chúng ta phải thiết lập bối cảnh và phạm vi cho các hoạt động được mô hình hóa. Sau khi thiết lập phạm vi, cần đặt tên cho sơ đồ.
- Cần phải chỉ ra các hoạt động, dòng điều khiển và dòng đối tượng xảy ra giữa các hoạt động.
- Cần phải chỉ ra những quyết định là một phần của tiến trình.
- Chỉ ra các hoạt động có khả năng xảy ra song song trong quy trình hoạt động.

b. Sơ đồ ca sử dụng (use case diagram)

Sơ đồ ca sử dụng là phần mô tả cơ bản nhất của chức năng hệ thống từ cái nhìn tổng quát đầu tiên của hệ thống. Nó thể hiện người dùng có thể làm gì và hệ thống phản hồi lại các hoạt động của người dùng như thế nào. Việc xây dựng sơ đồ ca sử dụng theo hai bước: Đầu tiên người dùng làm việc với người phân tích viết ra bản viết mô tả ca sử dụng, bước thứ hai, người phân tích chuyển từ bản mô tả bằng chữ viết sang sơ đồ. Cả hai bản trên đều dựa vào các yêu cầu đã thu thập và sơ đồ hoạt động về quy trình nghiệp vụ.

Sơ đồ ca sử dụng chụp lại các tương tác của hệ thống với người dùng hệ thống. Những tương tác này có thể là góc nhìn bên ngoài hoặc góc nhìn chức năng hệ thống từ phía người dùng tương ứng. Mỗi một ca sử dụng thể hiện một chức năng mà người dùng tác động vào hệ thống.

Có nhiều loại sơ đồ ca sử dụng, ở đây chúng ta xét hai loại dựa vào mục đích và lượng thông tin của ca sử dụng: tổng quát và chi tiết, thiết yếu và thực tế.

Một ca sử dụng tổng quát cho phép nhà phân tích và người dùng thống nhất về góc nhìn tổng quát của hệ thống. Các sơ đồ này được tạo ra từ sớm trong lúc thu thập yêu cầu. Tài liệu đi kèm chỉ bao gồm thông tin về ca sử dụng như tên, số ID, tác nhân, loại và một phần mô tả ngắn. Sau khi thống nhất các ca sử dụng tổng quan, nhà phân tích tiếp tục chuyển sang sơ đồ ca sử dụng chi tiết. Một tài liệu ca sử dụng chi tiết chứa đựng nhiều thông tin nhất có thể cần thiết cho ca sử dụng.

Ca sử dụng thiết yếu là một ca sử dụng dùng để mô tả những vấn đề cần thiết, thiết yêu để hiểu chức năng yêu cầu. Một ca sử dụng thực sẽ đi xa hơn, cung cấp các bước cụ thể. Ví dụ một ca sử dụng thực ở phòng khám răng: sắp xếp thời gian hẹn với khung giờ hiệu lực. Trong khi đó nếu trong ca sử dụng chi tiết có thể nói khác" người tiếp viên nên tra cứu dữ liệu trong lịch bằng chương trình MS Exchange để xác định rằng thời gian hẹn có được hay không". Sự khác biệt thể hiện mức độ chi tiết. Vì vậy đối ca sử dụng thực có xu hương dùng để thiết kế chi tiết, cài đặt và kiểm thử.

➢ **Các thành phần cơ bản mô tả ca sử dụng**

Có 3 phần cơ bản của một bản mô tả cả sử dụng: thông tin tổng quan, mối quan hệ, và dòng sự kiện.

- Thông tin tổng quan: tên, id, tác nhân, các bên liên quan và lợi ích, mô tả ngắn, điều kiện khởi tạo.
- Có 4 loại mối quan hệ:
    - **quan hệ liên kết**: giữa tác nhân và ca sử dụng của tác nhân đó
    - **quan hệ mở rộng**: thể hiện sự mở rộng của một ca sử dụng với các hành vi tùy chọn (có thể có hoặc không).
    - **quan hệ bao gồm**: thể hiện mối quan hệ bắt buộc với các ca sử dụng khác, nói cách khác nó được sử dụng trong trường hợp chia nhỏ một ca sử dụng phức tạp thành các ca sử dụng đơn giản.
    - **quan hệ tổng quát**: cho phép các ca sử dụng hỗ trợ kế thừa.

- Dòng sự kiện:
    - **Dòng thông thường**: bao gồm những bước thông thường được thực hiện trong ca sử dụng.
    - **Dòng con**: một vài trường hợp, dòng sự kiện thông thường được chia nhỏ ra thành các dòng sự kiện con để đơn giản hóa dòng sự kiện.
    - **Dòng thay thế hay dòng ngoại lệ**: là các sự kiện thật sự xảy ra nhưng không được xem là thông thường.

Ví dụ ca sử dụng tạo buổi hẹn:

Tên ca sử dụng: tạo cuộc hẹn	ID: 2	Mức độ quan trọng: cao
Tác nhân đầu: Bệnh nhân	Loại ca sử dụng: chi tiết, thiết yếu	
Các bên liên quan và mối quan tâm	Bệnh nhân: muốn thiết lập, thay đổi hoặc hủy hẹn Bác sĩ: muốn đảm bảo nhu cầu của bệnh nhân được đáp ứng một cách kịp thời	
Mô tả ngắn	Ca sử dụng này mô tả cách thức tạo một cuộc hẹn cũng như thay đổi hoặc hủy cuộc hẹn	
Điều kiện khởi tạo	Bệnh nhân gọi điện thoại thiết lập cuộc hẹn hoặc yêu cầu đổi, hoặc hủy hẹn.	
Loại khởi tạo	Từ bên ngoài	
Mối quan hệ	Kết hợp: bệnh nhân Bao gồm: thanh toán cho buổi hẹn Mở rộng: tạo bệnh nhân mới Tổng quát hóa:	
Dòng sự kiện thông thường	1. Bệnh nhân liên hệ với văn phòng để thiết lập hẹn. 2. Bệnh nhân cung cấp tên, địa chỉ cho tiếp viên. 3. Tiếp viên kiểm tra xem bệnh nhân đã có trong cơ sở dữ liệu 4. Tiếp viên thực hiện ca sử dụng trả chi phí cuộc hẹn. 5. Tiếp viên hỏi bệnh nhân có muôn tạo một cuộc hẹn mới hay hủy cuộc hẹn hoặc thay đổi cuộc hẹn đã có.	
Dòng sự kiện con	S1: tạo cuộc hẹn mới **1.** Tiếp viên hỏi bệnh nhân khi nào là thời gian thích hợp	

	2. Tiếp viên so sánh thời gian của bệnh nhân với thời gian còn trống trong cơ sở dữ liệu và lên lịch cuộc hẹn mới. S2: Hủy cuộc hẹn 1. Người tiếp viên hỏi bệnh nhân về thông tin cuộc hẹn cũ 2. Tiếp viên tìm cuộc hẹn đó và hủy nó S3: Thay đổi cuộc hẹn 1. Tiếp viên tiến hành S2 để hủy hẹn 2. Tiếp viên tiến hành S1 để thiết lập cuộc hẹn mới.
Dòng lựa chọn	3a. Tiếp viên thực hiện ca sử dụng tạo bệnh nhân mới S-1, 2a1: tiếp viện lựa chọn thời gian thay thế dựa vào các thời gian còn trống trong cơ sở dữ liệu. S-1, 2a2: Bệnh nhân lựa chọn thời gian phù hợp hoặc quyết định hủy hẹn.

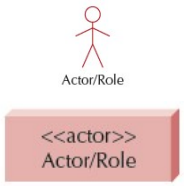
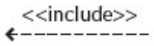
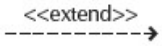
Các chú ý trong viết mô tả ca sử dụng:

1. Viết các bước thực hiện theo đúng cú pháp, chủ ngữ - động từ - đối tượng chịu tác động trực tiếp, gián tiếp.
2. Xác định rõ ai là người khởi đầu các bước.
3. Viết các bước tuần tự xảy ra theo cách nhìn độc lập (sắp xếp từ cách nhìn của người khởi tạo và người nhận tác động).
4. Viết mỗi bước theo cùng mức độ trừu tượng.
5. Chắc chắn mỗi ca sử dụng là tập các bước có ý nghĩa. Để đảm bảo như vậy thì mỗi ca sử dụng phải có tác nhân, dữ liệu vào, xử lý dữ liệu, xuất trả kết quả cho tác nhân.
6. Cố gắng đơn giản hóa hết mức có thể.
7. Viết rõ các điều kiện lặp đối với những tập các bước lặp.

➢ **Các thành phần sơ đồ ca sử dụng**

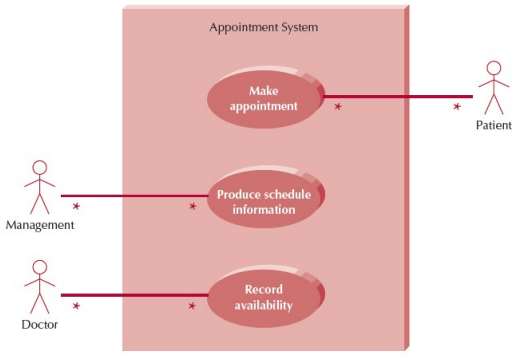
Sơ đồ ca sử dụng mô tả một cách đơn giản nhất các chức năng chính và những người dùng khác nhau tác động vào hệ thống. Nó là một cách đơn giản giúp nhà phân tích và người dùng có thể phát hiện thêm những yêu cầu chưa được đề cập trong bản

viết.

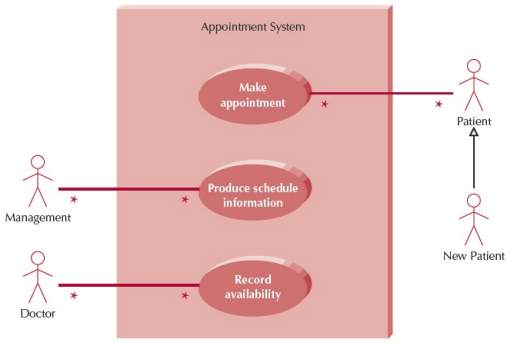
Nội dung	Ký hiệu
Tác nhân - Là con người hoặc hệ thống khác có nguồn lợi từ hệ thống và tương tác với hệ thống. - Được ký hiệu là hình người nếu tác nhân là người hoặc có thể là hình vuông nếu tác nhân không phải là người. - Được đặt tên với tên của vai trò. - Có thể kết nối với các tác nhân khác với mối quan hệ cụ thể hóa hoặc tổng quát hóa. - Được đặt ngoài đường biên chính nhóm ca sử dụng.	
Ca sử dụng - Thể hiện một phần chức năng hệ thống - Có thể mở rộng ca sử dụng khác - Có thể bao gồm ca sử dụng khác. - Được đặt bên trong đường biên hệ thống. - Được đặt tên bởi cụm danh động từ.	
Đường biên hệ thống - Bao gồm tên tổng thể quy trình. - Thể hiện phạm vi của chủ đề, một hệ thống hoặc một quy trình nghiệp vụ đơn lẻ.	
Mối quan hệ kết hợp - Liên kết tác nhân với ca sử dụng mà nó tương tác.	
Mối quan hệ bao gồm - Thể hiện một chức năng bao gồm nhiều ca sử dụng khác. - Mủi tên được vẻ từ ca sử dụng cở sở đến cả ca sử dụng bao gồm.	
Mối quan hệ mở rộng - Thể hiện sự mở rộng của ca sử dụng có thể bao gồm một ca sử dụng nào đó hoặc không. - Mủi tên được vẻ từ ca sử dụng mở rộng đến ca sử dụng cơ sở.	
Mối quan hệ tổng quát - Thể hiện quan hệ từ một ca sử dụng đặc biệt đến các ca sử dụng bình thường khác. - Mủi tên nói từ ca sử dụng đặc biết đến các ca sử dụng cơ sở	

khác.	

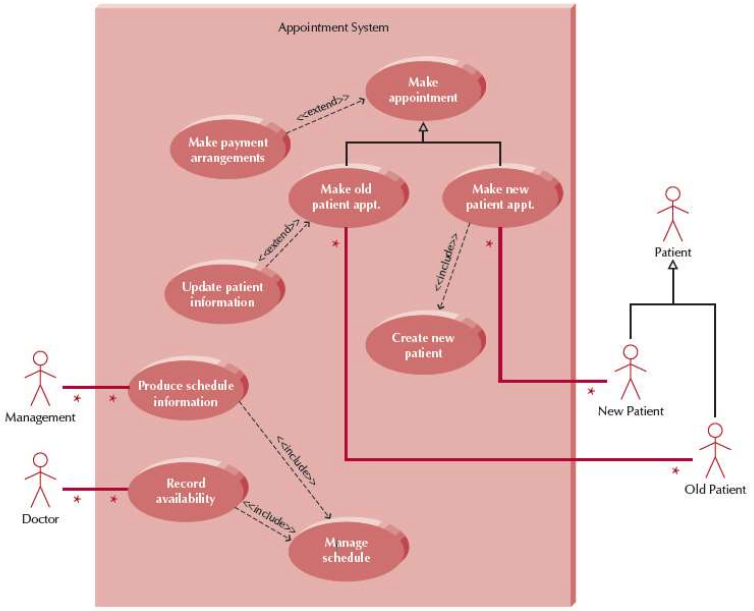
Ví dụ sơ đồ ca sử dụng tạo cuộc hẹn:



Hình 2-5: Sơ đồ ca sử dụng đơn giản



Hình 2-6: Sơ đồ ca sử dụng với mối quan hệ cụ thể hóa tác nhân



Hình 2-7: Biểu đồ ca sử dụng có các mối quan hệ mở rộng và bao gồm

Các bước chú ý trong xây dựng biểu đồ ca sử dụng và bản mô tả ca sử dụng:

- **Chỉ ra các ca sử dụng chính**

  1. Xem lại sơ đồ hoạt động.
  2. Tìm phạm vi, đường biên của chủ đề.
  3. Chỉ ra được các tác nhân và mục tiêu của họ.
  4. Chỉ ra và viết tổng quan cho các ca sử dụng ở trên.
  5. Xem xét lại các ca sử dụng ở trên.

- **Mở rộng ca sử dụng chính**

  6. Lựa chọn một ca sử dụng để mở rộng.
  7. Bắt đầu điền chi tiết vào ca sử dụng đó.
  8. Viết các dòng sự kiện thông thường cho ca sử dụng.
  9. Nếu dòng sự kiện thông thường quá phức tạp, dài dòng thì chia làm nhiều dòng con.
  10. Liệt kê tất các dòng thay thế.
  11. Đối với các dòng thay thế, liệt kê làm thế nào tác nhân và hệ thống hành động.

- **Xác nhận lại ca sử dụng chính**

  12. Xem xét lại tập ca sử dụng đang thực hiện.
  13. Bắt đầu từ điểm đầu tiên.

- **Tạo sơ đồ ca sử dụng**

  14. Vẻ đường biên chủ đề.
  15. Đặt các ca sử dụng vào sơ đồ.
  16. Đặt các tác nhân vào sơ đồ.
  17. Vẻ các mối quan hệ.

2.4.4. Kiểm tra các yêu cầu

Bước kiểm tra các yêu cầu này đảm bảo rằng người phân tích đã thu thập đúng các yêu cầu của hệ thống mà khách hàng mong muốn. Đây là bước rất quan trọng để tránh việc tốn kém chi phí thay đổi yêu cầu của hệ thống về sau. Một số hình thức kiểm tra

được thực hiện trong giai đoạn này như sau:

- **Kiểm tra tính hợp lệ, chính xác**: Một hệ thống liên quan đến nhiều người dùng, nhu cầu sử dụng hệ thống của họ khác nhau, vì vậy cần kiểm tra lại các chức năng như vậy có chính xác và phù hợp không. Mặt khác, khi phân tích các yêu cầu đề ra, có thể phải thêm một số chức năng khác để hỗ trợ thực hiện, cần kiểm chứng lại xem chức năng mới này có phù hợp với quy trình nghiệp vụ của tổ chức hay không.

- **Kiểm tra tính nhất quán**: Xác định lại các yêu cầu trong hệ thống không được xung đột với nhau. Tránh các trường hợp có sự mô tả khác nhau cho cùng một yêu cầu chức năng.

- **Kiểm tra tính đầy đủ**: Kiểm tra bản phân tích yêu cầu phải có đầy đủ tất cả các chức năng cũng như các ràng buộc của người dùng đặt ra.

- **Kiểm tra tính khả thi**: Xem xét lại tất cả các yêu cầu có thể thực hiện được bằng công nghệ hiện tại, cũng cần xem xét thêm về chi phí và thời gian, nhân sự dành cho việc phát triển hệ thống.

- **Kiểm chứng**: Tất cả các yêu cầu nên được viết ra và kiểm chứng giữa hai bên đơn vị khách hàng và đơn vị sản xuất phần mềm để đảm bảo hệ thống xây dựng đáp ứng đúng yêu cầu đã đưa ra.

TỔNG KẾT CUỐI CHƯƠNG

Khảo sát và phân tích yêu cầu là giai đoạn đầu trong quy trình nghiệp vụ, có nhiệm vụ xác định tính khả thi của hệ thống, yêu cầu của hệ thống từ tổng quan cho đến chi tiết, là nguồn tài liệu phục vụ cho quá trình thiết kế.

Có hai loại yêu cầu: yêu cầu chức năng và yêu cầu phi chức năng. Yêu cầu chức năng là yêu cầu những gì hệ thống nên làm, yêu cầu phi chức năng là các ràng buộc về tính chất mà hệ thống cần phải đạt được.

Trong giai đoạn này gồm có 4 hoạt động: xác định tính khả thi, xác định yêu cầu, chuẩn hóa các yêu cầu, xác thực các yêu cầu. Để chuẩn hóa các yêu cầu, người lập trình có thể sử dụng các sơ đồ hoạt động, sơ đồ ca sử dụng để biểu diễn các yêu cầu.

CÂU HỎI ÔN TẬP CHƯƠNG 2

2.1. Vai trò của giai đoạn phân tích và đặc tả yêu cầu là gì?

2.2. Tại sao phải xác định yêu cầu chức năng và phi chức năng? Sự khác nhau giữa hai loại yêu cầu này là gì?

2.3. Các hoạt động chính của giai đoạn phân tích và đặc tả yêu cầu? phân tích từng hoạt động.

2.4. Cấu trúc đề nghị của một bản đặc tả chức năng như thế nào?

2.5. Sử dụng các kỹ thuật có thể xác định yêu cầu của hệ thống bán hàng tại các cửa hàng vừa và nhỏ.

2.6. Sử dụng các kỹ thuật có thể xác định yêu cầu của hệ thống đăng ký học phần tín chỉ.

2.7. Cho hệ thống được mô tả như sau:

Bibishop là cửa hàng bán áo quần trẻ em từ lứa tuổi sơ sinh đến 15 tuổi. Áo quần có rất nhiều loại, từ hàng bình dân đến cao cấp, ngoài áo quần ra còn có thêm các phụ kiện cho trẻ sơ sinh như bỉm, tả, bình sửa …

Hiện tại cửa hàng có 4 nhân viên. Một nhân viên chuyên phụ trách nhập hàng, một nhân viên chuyên phụ trách bán hàng, viết sổ sách, một nhân viên phục vụ sắp xếp, làm vệ sinh, bán hàng tại cửa hàng và một nhân viên phụ trách kinh doanh, chuyển giao hàng đi các huyện.

Công việc cụ thể của từng nhân viên:

- Nhân viên nhập hàng: tìm các nguồn hàng mới, đẹp, các đối tác rồi nhập mẫu hàng về, so sánh, định giá hàng bán ra. Nhân viên này cũng chịu trách nhiệm thanh toán các đơn hàng. Nếu hàng có lỗi, cần phải đổi trả, thì nhân viên này tiến hành đổi trả, nhận lại tiền hoặc hàng tùy theo thỏa thuận với đối tác.

- Nhân viên viên bán hàng tại cửa hàng: chào đón khách hàng, tư vấn khách hàng lựa chọn mẫu áo quần phù hợp, ghi sổ và thu tiền với khách hàng. Hàng ngày thống kê số hàng từng loại đã bán ra, tổng tiền và tổng tiền hàng bán, tổng tiền hàng nhập. Cuối tuần thống kê lại tổng tiền hàng bán, tổng tiền hàng nhập, tổng tiền công nợ và danh sách công nợ, danh sách hàng tồn kho, danh sách hàng cần nhập thêm.

- Nhân viên phụ trách kho: Nhân viên có nhiệm vụ sắp xếp sản phẩm, trang trí cất đặt sản phẩm trong cửa hàng sao cho đẹp mắt, thuận tiện người mua hàng, và dễ

tìm kiếm sản phẩm. Khi khách hàng mua hàng, hoặc đổi trả hàng, nhân viên kiểm tra mẫu mã, đóng gói cho khách hàng. Trong quá trình khách vào mua hàng, nhân viên có trách nhiệm quản lý hàng hóa không cho xảy ra mất mát sản phẩm. Khi cần tìm kiếm hàng hóa, nhân viên có trách nhiệm tìm kiếm nhanh cho khách hàng. Nếu có mất mát sản phẩm, nhân viên quản lý phải chịu trách nhiệm hoàn toàn. Cuối mỗi tuần, nhân viên kho tiến hành kiểm kê hàng hóa trực tiếp tại cửa hàng, so khớp với số liệu của nhân viên bán hàng, kiểm tra số hàng trùng khớp hay không? Nếu mất mát, số liệu không phù hợp, tiến hành điều tra và quy trách nhiệm cho các nhân viên.

- Nhân viên kinh doanh: Nhân viên này có nhiệm vụ mang các mẫu hàng đi tới các cửa hàng chào mời. Tiếp nhận các đơn hàng của khách hàng, vận chuyển hàng hóa đến nơi và thu tiền. Trong quá trình mang hàng đi, nhân viên kinh doanh thực hiện việc mua hàng theo hình thức công nợ và thanh toán tiền sau khi đã thu tiền của khách. Việc xuất hàng và thu tiền đều được nhân viên cửa hàng ghi chép vào sổ sách.

- Những vấn đề gặp phải: khó kiểm tra hàng hóa, số lượng từng loại, giá hàng hóa hay bị nhầm lẫn, việc tính toán lợi nhuận còn nhiều khó khăn.

- Muốn tiến hành mua bán qua mạng

Xác định yêu cầu, sử dụng các sơ đồ hoạt động, sơ đồ ca sử dụng để đặc tả hệ thống.

Chương 3: THIẾT KẾ PHẦN MỀM

Sau khi nắm rõ yêu cầu của hệ thống và tạo lập hồ sơ phân tích thiết kế, quá trình sản xuất phần mềm chuyển sang giai đoạn thiết kế phần mềm. Chương này cung cấp các kiến thức về các kỹ thuật thiết kế theo phương pháp hương đối tượng sử dụng UML bao gồm thiết kế kiến trúc tổng thể, thiết kế sơ đồ lớp,thiết kế cơ sở dữ liệu, thiết kế sơ đồ tuần tự,sơ đồ trạng thái, thiết kế giao diện.

Trước khi vào chương này, người học cần nắm rõ được các yêu cầu phần mềm, và đã biết cách mô hình hóa phần mềm theo sơ đồ hoạt động, sơ đồ ca sử dụng. Kết thúc chương này, người học được trang bị kiến thức đầy đủ để thiết kế một phần mềm theo nội dung phân tích yêu cầu.Các bản thiết kế cần chính xác để tránh các trường hợp thiết kế lại sau khi đã lập trình dẫn đến tón kém thời gian và chi phí.

3.1. ĐẶC ĐIỂM CỦA QUÁ TRÌNH THIẾT KẾ PHẦN MỀM

Nhiệm vụ của giai đoạn thiết kế là chuyển đổi những yêu cầu của hệ thống thành các thành phần của hệ thống được mô tả trên văn bản.

Các bước cần phải làm trong giai đoạn thiết kế:

- Hiểu và mô tả các tương tác với bên ngoài của hệ thống của hệ thống. Các bước này được thực hiện trong quá trình phân tích và đặc tả yêu cầu. Tuy nhiên trong quá trình thiết kế, các bước này cũng có thể được thực hiện lại để làm rõ hơn, chi tiết hơn những yêu cầu chưa rõ và còn thiếu sót.
- Thiết kế kiến trúc hệ thống.
- Chỉ ra các đối tượng chính trong hệ thống.
- Phát triển các mô hình thiết kế.
- Chỉ ra giao diện tương tác.

Giống như các hoạt động sáng tạo khác, quy trình thiết kế không phải là quy trình tuần tự, rõ ràng. Chúng ta thiết kế bằng cách thu thập í tưởng, đề ra giải pháp. Chúng ta thường phải kiểm tra lại, thử lại khi có vấn đề nẩy sinh. Các bước thực hiện trong quy trình này không tuần tự như một sơ đồ đơn giản mà nó có thể lặp lại nhiều lần, và ảnh hưởng lẫn nhau.

3.2. THIẾT KẾ KIẾN TRÚC HỆ THỐNG

Thiết kế kiến trúc là bước thiết kế đầu tiên trong giai đoạn thiết kế phần mềm. Có hai mức độ thiết kế kiến trúc, kiến trúc quy mô nhỏ và kiến trúc trên diện rộng.

- Kiến trúc quy mô nhỏ liên quán đến kiến trúc của từng cá thể chương trình. Ở mức độ này, kiến trúc thể hiện cách phân rã chương trình thành các thành phần con.
- Kiến trúc quy mô lớn kiến trúc phức tạp của hệ thống bao gồm các phần mềm khác, chương trình, và các thành phần của chương trình. Hệ thống này được phân bố trên nhiều máy tính nằm trên phạm vị rộng.

Kiến trúc của hệ thống rất quan trọng bởi vì nó ảnh hưởng đến hiện năng, tính mạnh mẽ, khả năng chuyển giao và duy trì hệ thống. Một thành phần phần mềm được cài đặt để thực hiện một chức năng hệ thống hay một yêu cầu chức năng hệ thống. Trong nhiều hệ thống, các yêu cầu phi chức năng vẫn ảnh hưởng bởi các thành phần riêng rẽ, nhưng rõ ràng chúng phụ thuộc vào kiến trúc hệ thống, cách mà những thành phần phần mềm được tổ chức và kết nối với nhau.

Kiến trúc hệ thống mang lại các lợi ích sau:

- Giúp nhà phân tích giao tiếp với các bên liên quan. Kiến trúc của hệ thống là mức thể hiện cao nhất của hệ thống, chúng có thể sử dụng để thảo luận với nhiều bên liên quan.
- Giúp phân tích hệ thống: Việc xác định kiến trúc hệ thống được xác định từ sớm. Nó quyết định hệ thống có đạt được các yêu cầu ràng buộc như tính hiệu quả, độ tin cậy và khả năng bảo trì hệ thống.
- Tái sử dụng: Mô hình kiến trúc hệ thống thể hiện cách thức làm thế nào các thành phần được tổ chức, tương tác với nhau. Trong một hệ thống, ta có thể nhìn thấy các thành phần được tái sử dụng. Các hệ thống có cùng yêu cầu thường có cùng kiến trúc vì vậy cũng tăng khả năng tái sử dụng.

Kiến trúc hệ thống thông thường được thể hiện bởi các sơ đồ hình khối đơn giản. Mỗi một hình khối thể hiện một thành phần. Hình khối nhỏ nằm trong hình khối to thể hiện thành phần lớn được phân rã thành các thành phần nhỏ.

Việc thiết kế kiến trúc hệ thống là một quy trình sáng tạo, kiến trúc phải làm sao thỏa mãn các yêu cầu chức năng và yêu cầu phi chức năng cho nên không có các bước thứ tự thực hiện. Dựa vào kinh nghiệm, để quyết định kiến trúc của một hệ thống, người ta thường trả lời các câu hỏi sau:

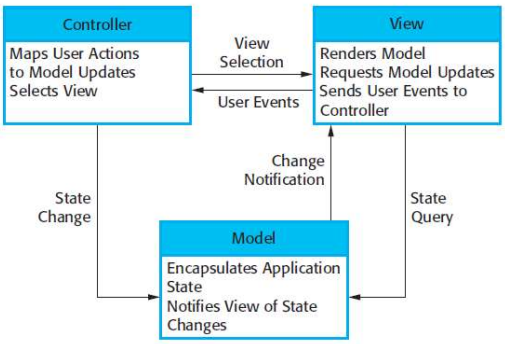
- Có kiến trúc ứng dụng phổ biến nào có thể làm mẫu cho hệ thống đang thiết kế hay không?

- Những kiểu kiến trúc nào có thể sử dụng?
- Phương pháp tiếp cận nào được sử dụng để cấu trúc hóa hệ thống?
- Làm thể nào để phân nhỏ các thành phần hệ thống thành các thành phần nhỏ hơn?
- Chiến lược nào dùng để điều khiển hoạt động các thành phần hệ thống?
- Tổ chức kiến trúc nào là tốt nhất, đảm bảo các yêu cầu phi chức năng của hệ tổng?
- Kiến trúc hệ thống sẽ được đánh giá như thế nào?
- Kiến trúc của hệ thống sẽ được tài liệu hóa như thế nào?

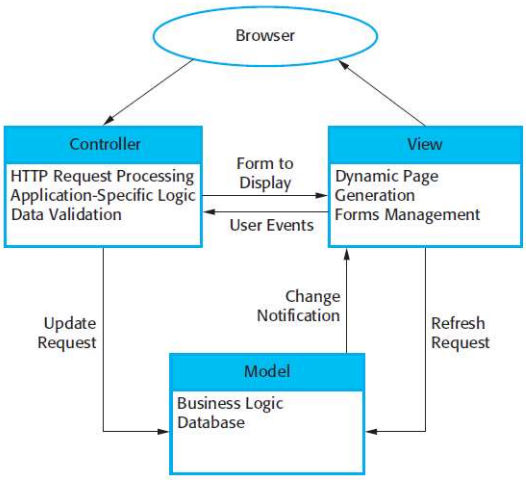
Kiến trúc hệ thống có thể được mô tả bởi nhiều góc nhìn khách nhau, có thể là góc nhìn khái niệm, góc nhìn logic, góc nhìn tiến trình, góc nhìn phát triển hay góc nhìn vật lý.

Các mẫu kiến trúc là các kiến trúc của các hệ thống chung đã được đúc kết từ nhiều thống có tính chất tương tự. Những mẫu kiến trúc phổ biến như:

- **kiến trúc MVC (Model-View-Controller: dữ liệu – góc nhìn –điều khiển)**: Kiến trúc này phân biệt giữa giao diện và tương tác từ dữ liệu hệ thống. Hệ thống được cấu trúc thành ba thành phần logic tương tác với nhau. Thành phần dữ liệu quản lý lý dữ liệu của hệ thống và kết hợp với các thao tác trên dữ liệu. Thành phần giao diện định nghĩa và quản lý giao diện với người dùng. Thành phần điều khiển quản lý tương tác người dùng và chuyển các tương tác này tới thành phần giao diện và thành phần dữ liệu.

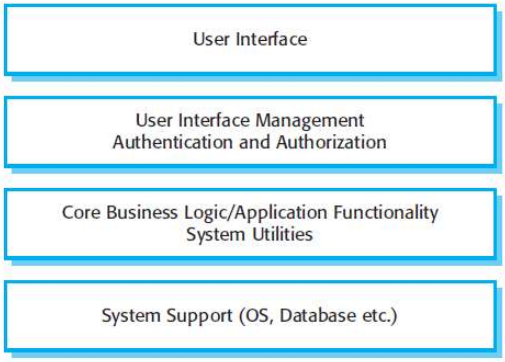


Hình 3-1: Kiến trúc MVC



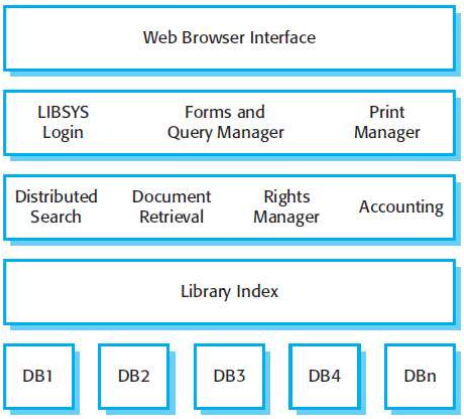
Hình 3-2: Kiến trúc MVC có sử dụng browser

- **Kiến trúc lớp**: Tổ chức hệ thống thành các lớp với các chức năng tương ứng. Mỗi lớp cung cấp dịch vụ cho lớp trên và sử dụng dịch vụ của lớp dưới. Các lớp dưới cung cấp các dịch vụ cốt lõi và được sử dụng xuyên suốt cả hệ thống. Kiến trúc này thường được sử dụng trong các hệ thống lớn, mỗi đội thực hiện một lớp.



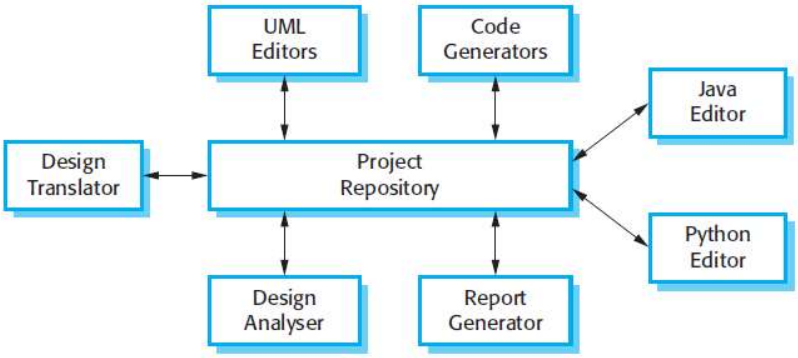
Hình 3-3:Kiến trúc lớp

Ví dụ kiến trúc lớp của hệ thống thư viện:



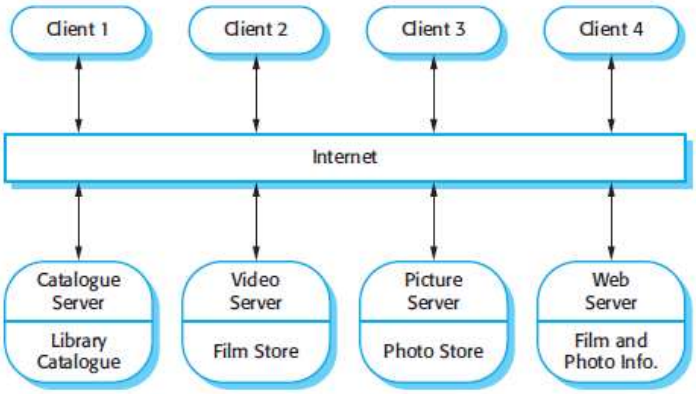
Hình 3-4: Kiến trúc lớp của thư viện

- **Kiến trúc kho (reporsity)**: Tất cả dữ liệu củ hệ thống được quản lý ở trung tâm mà các thành phần hệ thống có thể truy cập được. Các thành phần không có tương tác trực tiếp với nhau mà thông qua trung tâm. Ví dụ kiến trúc kho của trình soạn thảo UML.



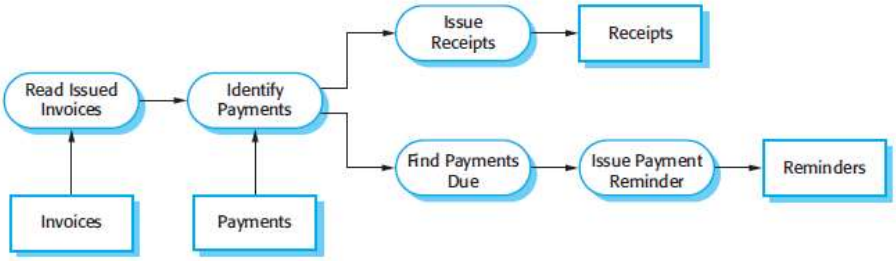
Hình 3-5: Kiến trúc kho

- **Kiến trúc chủ - khách**: trong kiến trúc này, chức năng của hệ thống được tổ chức thành dịch vụ, mỗi dịch vụ được cung cấp từ các máy chủ riêng biệt. Khách là những người dùng truy cập vào máy chủ sử dụng những dịch vụ này. Kiến trúc này thương được sử dụng khi dữ liệu của hệ thống phải có khả năng truy cập từ nhiều địa điểm khác nhau. Ví dụ dưới đây là kiến trúc của thư viện video:



Hình 3-6: Kiến trúc chủ khách

- **Kiến trúc ống – lọc (pile – filter):** Được sử dụng chủ yếu trong các hệ thống xữ lý dữ liệu lớn, ở đó từ dữ liệu đầu vào, qua nhiều bước xữ lý, đưa ra kết quả tương ứng. Quy trình xữ lý dữ liệu được chia thành các bước riêng biệt, mỗi một chuỗi (đường ống) thể hiện một quy trình xữ lý và dữ liệu được chuyễn từ thành phần này sang thành phần khác.



Hình 3-7: Kiến trúc ống – lọc

3.3. THIẾT KẾ CÁC MÔ HÌNH CẤU TRÚC

3.3.1. Xác định các đối tượng

Sau các bước thiết lập bảng mô tả ca sử dụng và sơ đồ ca sử dụng, chúng ta đã xác định được một số đối tượng và các tương tác của chúng trong hệ thống. Có 3 phương pháp phổ biến để xác định đối tượng là phương pháp phân tích bản viết (textual analysis), danh sách phổ biến (common object lists), và mẫu (patterns). Hầu hết các nhà phân tích đều sử dụng kết hợp cả ba phương pháp để đảm bảo không bị thiếu sót bất kỳ đối tượng nào.

Phương pháp phân tích bản viết: là phương pháp phân tích dựa vào bản mô tả ca sử dụng. Nhà phân tích - thiết kế bắt đầu cách xem xét lại tất cả các bản mô tả và sơ đồ ca sử dụng. Thông qua bản mô tả này ta có thể xác định ra những đối tượng, thuộc tính,

hành động, và mối quan hệ có thể có của các đối tượng. Một số hướng dẫn trong phương pháp này như sau:

- Các danh từ thường ám chỉ một lớp đối tượng.
- Các tính từ thường chỉ đến thuộc tính của đối tượng.
- Động từ hoạt động thường chỉ đến hoạt động.
- Động từ chỉ tính chất (ví dụ là) chỉ ra mối quan hệ giữa các đối tượng hay lớp.
- Động từ sở hữu thường chỉ ra mối quan hệ kết hợp hoặc tổng quát hóa.
- Một ngoại động từ (động từ có danh từ bổ nghĩa) chỉ ra một hành động.
- Một nội động từ chỉ ra ngoại lệ.
- Một vị từ hay một cụm động từ chỉ ra một hành động.
- Một trạng từ chỉ ra thuộc tính của mối quan hệ hoặc một hành động.

Phương pháp danh sách đối tượng phổ biến (common object list): phương pháp này chỉ ra danh sách các đối tượng thường xuyên xuất hiện trong hệ thống. Ngoài việc xem xét các ca sử dụng, nhà phân tích – thiết kế xem xét chung về quy trình nghiệp vụ trong thực tế. Một số danh mục đã được liệt kê ra để giúp đỡ họ tìm nhanh hơn:

Các đối tượng hữu hình: ví dụ như sách, bàn ghế, các thiết bị văn phòng … những đối tượng này dễ dàng được nhận biết.

Các biến cố: như buổi họp, chuyến bay.

Các vai trò: ví dụ như bác sĩ, y tá, bệnh nhân …

Các tương tác: tương tác thường là một giao dịch ví dụ như quy trình mua bán…

Các đối tượng khác có thể là địa điểm, các tổ chức, các biên bản, các điều luật, một số trường hợp các giao dịch cần đối tượng để lưu trữ thông tin.

Mẫu: Ý tưởng sử dụng mẫu liên quan đến phát triển hệ thống hướng đối tượng. Mẫu chỉ đơn giãn là một nhóm các lớp kết hợp với nhau cung cấp hướng giải pháp cho những vấn đề phổ biến và được tái sử dụng. Khi cần phần tích một hệ thống nào đó đã có mẫu, ta có thể tham khảo mẫu để tìm ra danh sách các đối tượng và các thuộc tính, hành động của nó.

3.3.2. Xây dựng thẻ CRC và sơ đồ lớp

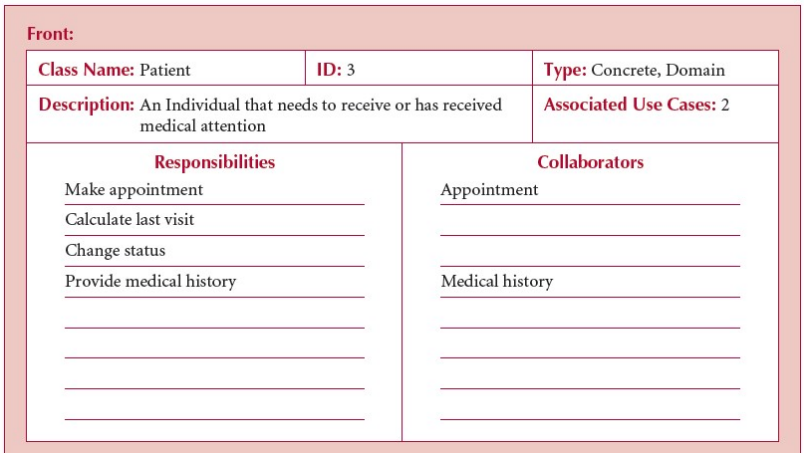
- **Các mối quan hệ trong sơ đồ lớp**
    - **Mối quan hệ thổng quát hóa**: Giúp cho người phân tích - thiết kế có thể

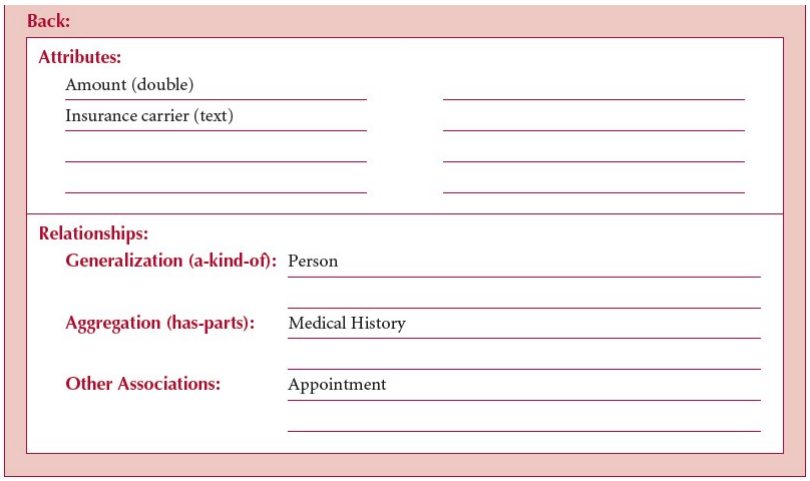
xây dựng các lớp kế thừa các thuộc tính và phương thức của lớp khác.

- **Mối quan hệ tổng hợp:** Nói theo cách thông thường nó có nghĩa như là một phần của cái gì, một phần của một tập hợp. Ví dụ cái cửa là một phần của ngôi nhà, ngôi nhà là một phần của dãy nhà …
- **Mối quan hệ kết hợp**: Là mối quan hệ yếu hơn mối quan hệ tổng hợp. Ví dụ như bệnh nhân sắp xếp buổi hẹn. Có thể nói là bệnh nhân là một phần của buổi hẹn, tuy nhiên có sự khác rõ ràng về mặt ngữ nghĩa giữa cái cửa là một phần ngôi nhà, mối quan hệ này được gọi là mối quan hệ kết hợp.

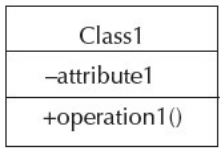
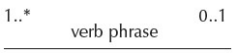
Thẻ CRC (Lớp – trách nhiệm – hợp tác : Class-Responsibility-Collaboration): được sử dụng để mô tả, tài liệu hóa chức năng và sự hợp tác của một lớp. Để xác định các thành phần của thẻ CRC, người ta thường trả lời các câu hỏi: Là cái gì? Biết những gì? Có thể làm gì?

Mỗi thẻ CRC có những thông tin như tên lớp, ID, loại, mô tả, danh sách các ca sử dụng kết hợp, trách nhiệm (responsibilities) và các đối tượng cộng tác (collaborators) ở phần trước và các thuộc tính và mối quan hệ ở phần sau.

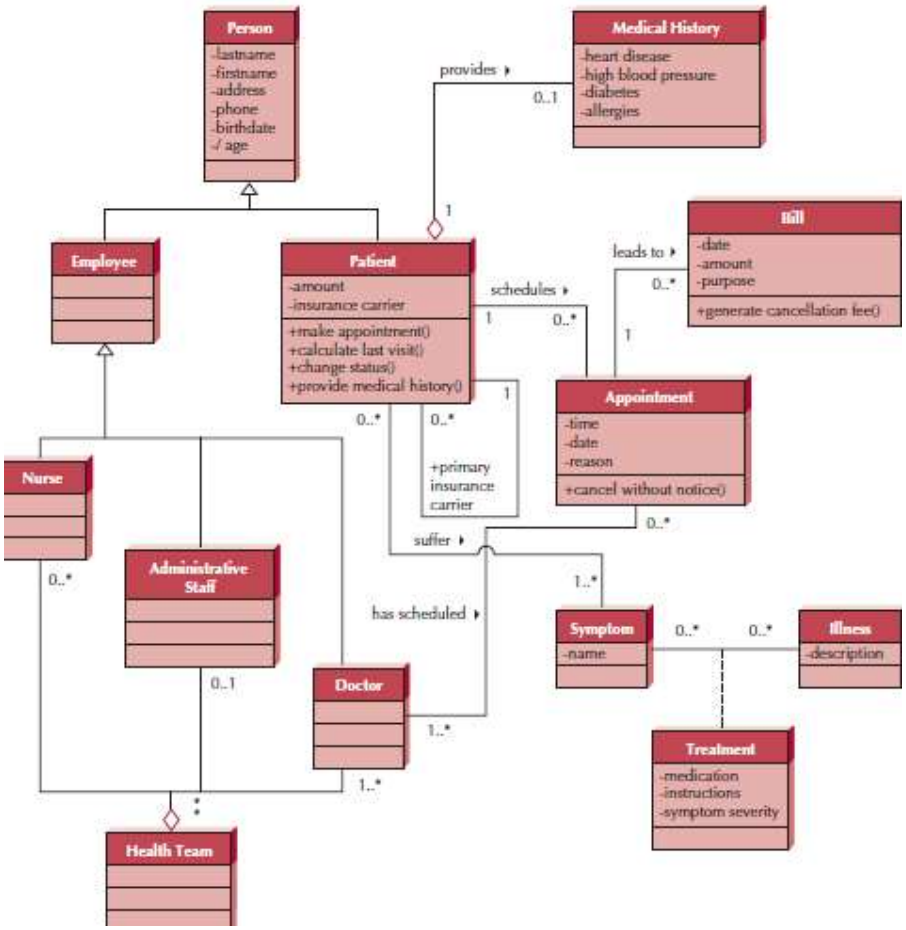




Các thành phần của sơ đồ lớp bao gồm lớp, thuộc tính của lớp, hành động của lớp, các mối quan hệ thể hiện trong bảng tóm tắt dưới đây.

Tên	Ký hiệu
Lớp: - Thể hiện một con người, sự vật, sự việc, nơi chốn mà hệ thống cần lưu trữ thông tin - Có tên, thuộc tính và phương thức - Không nhất thiết phải thể hiện hết các phương thức của các lớp có mặt trong sơ đồ	
Mối quan hệ kết hợp: - Thể hiện mối quan hệ giữa nhiều lớp với nhau, hoặc tự lớp đó với chính mình - Được đặt tên bởi động từ, cụm động từ - Có chứa các ký hiệu nhiều nhất, ít nhất để biểu diễn số lượng thể hiện của lớp có thể kết hợp với thể hiện của lớp kia.	
Mối quan hệ tổng quát hóa	
Mối quan hệ tổng hợp	

Ví dụ:



Hình 3-8: Sơ đồ lớp hệ thống khám chữa bệnh

3.4. SỬ DỤNG CÁC SƠ ĐỒ THIẾT KẾ

Những sơ đồ này thể hiện đối tượng trong hệ thống, đồng thời thể hiện mối quan hệ kết hợp giữa các thực thể. Những mô hình này là trung gian giữa đặc tả hệ thống và cài đặt hệ thống. Chúng chứa thông tin đủ cho người lập trình có thể đưa ra những quyết định cài đặt. Có nhiều loại sơ đồ thiết kế, nhưng không nhất thiết phải sử dụng hết tất cả các sơ đồ, càng giảm sử dụng các sơ đồ thiết kế thì giảm chi phí và thời gian trong giai đoạn thiết kế.

Trong lúc sử dụng UML để thiết kế, hai loại sơ đồ thiết kế thường được sử dụng là:

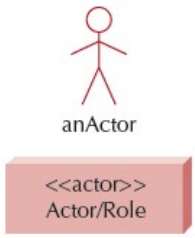
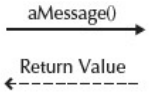
Sơ đồ cấu trúc: Mô tả cấu trúc tỉnh của hệ thống bao gồm các lớp và mối quan hệ giữa các lớp. Các sơ đồ này đã đề cập ở phần trước.

Sơ đồ động: Mô tả cấu trúc động của hệ thống và thể hiện tương tác giữa các đối tượng hệ thống. Tương tác này có thể tài liệu hóa bằng cách thêm một dãy các dịch vụ

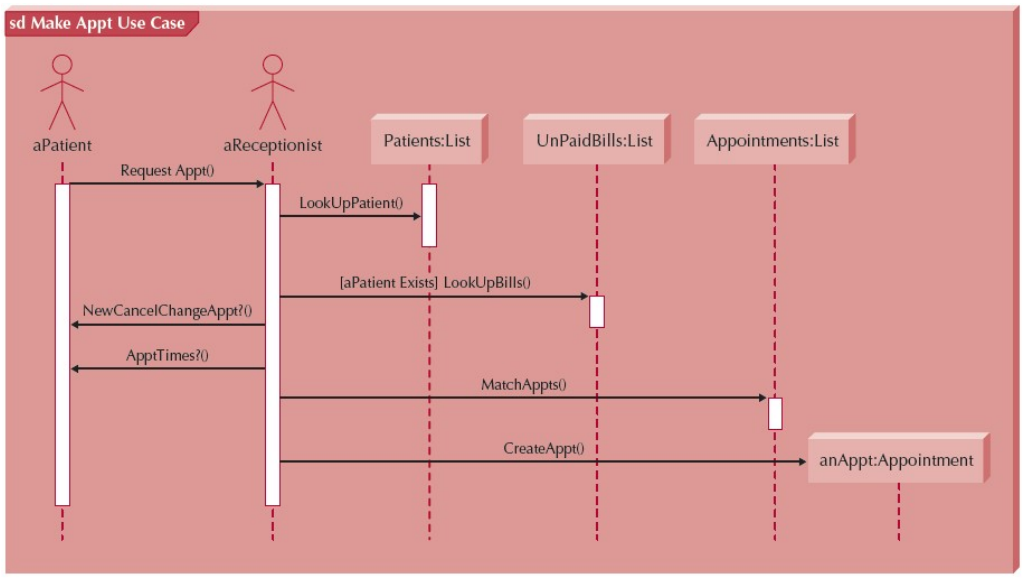
được yêu cầu từ các đối tượng này và trang thái của nó cũng thay đổi tương ứng theo các tương tác đó. Sơ đồ động gồm có 2 sơ đồ: sơ đồ tuần tự và sơ đồ trạng thái.

Sơ đồ tuần tự thể hiện lần lượt các tương tác của các đối tượng diễn ra. Cần phải chú trọng đến các tương tác quan trọng, cụ thể mỗi ca sử dụng nên có một sơ đồ tuần tự thể hiện lại các tương tác kết hợp với các đối tượng.

Các thành phần trong sơ đồ tuần tự:

Tên	Ký hiệu
Tác nhân: - Là người hoặc hệ thống nằm bên ngoài hệ thống - Tham gia vào dãy tuần tự các tương tác gửi và nhận các thông điệp - Được đặt ở phía trên sơ đồ - Là một trong hai ký hiệu bên	
Đối tượng: - Tham gia trong dãy các tương tác bằng cách gửi và nhận thông điệp - Được đặt phía trên sơ đồ	
Một vòng đời: - Thể hiện chu kỳ sống của một đối tượng - Có dấu X tại vị trí mà lớp đó không còn tương tác nữa	
Thực thi hành động: - Là một hình chữ nhật dài hẹp đặt trên vòng đời - Thể hiện khi đối tượng nhận gửi và dữ liệu	
Thông điệp: - Chứa thông điệp từ đối tượng này sang đối tượng khác - Một hành động được gọi được đặt tên là thông điệp và được gửi bởi mủi tên liền, trong khi đó dữ liệu trả về bởi mủi tên nét đứt.	
Hủy đối tượng: - Dấu X được đặt phí cuối của một vòng đời để thể hiện đối tượng không tồn tại nữa	X
Khung: - Chỉ ra bối cảnh của sơ đồ tuần tự	

Ví dụ sơ đồ tuần tự ca sử dụng lập buổi hẹn với bác sĩ:



Hình 3-9: Sơ đồ tuần tự tạo buổi khám với bác sĩ

Đầu tiền tác nhân là bệnh nhân gửi thông điệp cần tạo buổi hẹn tới người tiếp viên. Người tiếp viên tiếp tục gửi thông điệp đến danh sách bệnh nhân tìm bệnh nhân. Kết thúc giai đoạn này bệnh nhân đã tồn tại, tiếp viên tiếp tục gửi thông điệp đến đối tượng là danh sách hóa đơn chưa trả, tìm kiếm hóa đơn của bệnh nhân. Kết thúc giai đoạn này bệnh nhân đã thanh toán tiền. Người tiếp viên tiếp tục gửi thông điệp tới bệnh nhân yêu cầu tạo mới, hủy, hoặc thay đổi buổi hẹn, thời gian gian hẹn. Sau đó, người tiếp viên gửi thông điệp đến danh sách các cuộc hẹn đã có so khớp thời gian. Kết thúc của giai đoạn này là tìm được thời gian hẹn phù hợp. Người tiếp viên gửi thông điệp đến đối tượng buổi hẹn tạo buổi hẹn.

Phương pháp tạo một sơ đồ tương tác:

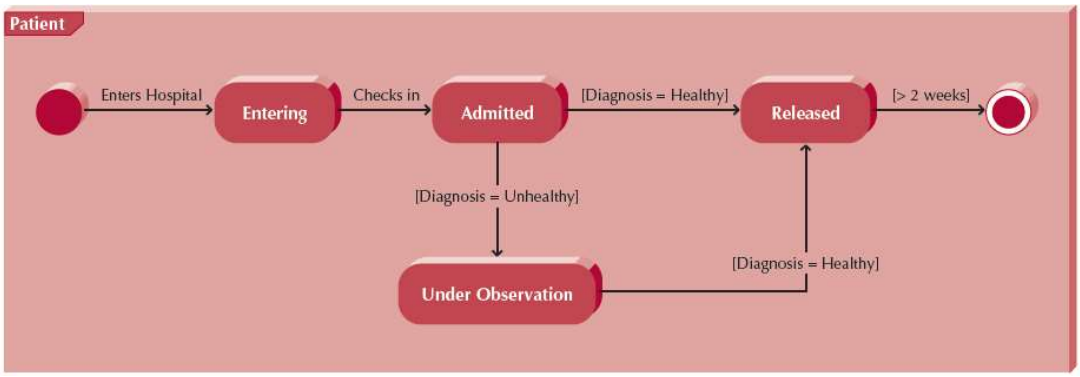
- Xác định phạm vi của sơ đồ
- Chỉ ra các đối tượng sẽ tham gia
- Thiết lập vòng đời cho mỗi đối tượng
- Liệt kê các thông điệp từ đầu cho đến cuối đồ thị dựa trên thứ tự chúng được gửi.
- Thêm các thực thi hoạt động cho mỗi vòng đời đối tượng.
- Kiểm tra lại đối tượng hệ thống.

Đối với sơ đồ tuần tự thể hiện các hành động xuất phát từ đối tượng nào, mà chưa thể hiện được sự biến đổi của đối tượng mỗi khi nhận thông điệp. Sơ đồ trạng thái của UML được sử dụng để thể hiện điều này.

Tên	Ký hiệu

Trạng thái: - Được đặt tên bởi trạng thái của đối tượng	aState
Trạng thái đầu: - thể hiện trạng thái bắt đầu tồn tại của đối tượng	
Trạng thái cuối: - thể hiện kết thúc của hoạt động	
Sự kiện: - tác động gây ra sự chuyển đổi trạng thái, mỗi sự kiện được đi kèm với các điều kiện và hành động - dùng để đặt tên các chuyển tiếp	Sự kiện
Chuyển tiếp (transition) - Biểu diễn các chuyển đổi của đối tượng từ trạng thái đầu sáng trạng thái sau	
Khung: - Chỉ ra bối cảnh của sơ đồ	Context

Ví dụ sơ đồ trạng thái của bệnh nhân khi điều trị tại bệnh viện



Hình 3-10: Sơ đồ trạng thái tạo buổi khám với bác sĩ

Trạng thái của bệnh nhân lúc đầu chuyển sang trạng thái nhập viện, sau khi làm thủ tục nhập viện, bệnh nhân chuyển sang trang thái được chấp nhận. Sau quá trình chẩn đoán, nếu bệnh nhân có sức khỏe tốt thì được trở về nhà. Nếu chuẩn đoán sức khỏe bệnh nhân không tốt thì tiếp tục được giữ lại theo giỏi cho đến khi tốt thì được về nhà.

Phương pháp tạo một sơ đồ trạng thái:

- Xác định bối cảnh phạm vi
- Xác định các trạng thái đầu, trạng thái cuối và các trạng thái ổn định của đối tượng

- Xác định các thứ tự các trạng thái mà đối tượng sẽ trãi qua
- Xác định các sự kiện, hành động, và điều kiện kiểm tra kết hợp với chuyển tiếp
- Kiểm tra lại tính chính xác của đồ thị

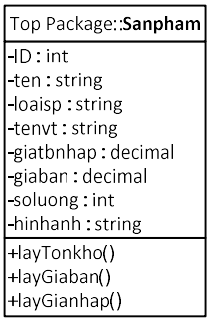
3.5. THIẾT KẾ DỮ LIỆU

Có nhiều hệ quản trị cơ sở dữ liệu hướng đối tượng, hỗ trợ thiết kễ dữ liệu hướng đối tượng. Tuy nhiên, các hệ quản trị cơ sở dữ liệu phổ biến và mạnh hiện nay đều là hệ quản trị dữ liệu quan hệ. Vì vậy, từ sơ đồ lớp mà ta đã các định, tuân theo các bước sau, ta chuyển sơ đồ lớp thành mô hình quan hệ.

- Ánh xạ tất cả các đối tượng cụ thể sang các bảng trong dữ liệu quan hệ. Nếu có lớp trừu tượng có nhiều lớp con không trừu tượng thì chuyển lớp trừu tượng thành bảng trong cơ sở dữ liệu quan hệ.

  Ví dụ:

  Từ đối tượng sản phẩm:



  Chuyển sang bảng dữ trong mô hình quan hệ:

Sanpham	
PK	ID
	ten tenvt giatbnhap giaban soluong hinhanh

- Chuyển các giá trị thuộc tính đơn thành cột của bảng.

  Trong ví dụ trên ta thấy, loại sản phẩm là thuộc tính đa trị nên không được

đưa vào trong bảng sản phẩm, còn các thuộc tính còn lại là đơn trị, nên được vào trong bảng.

- Ánh xạ các phương thức thành các hàm cài đặt trong cơ sở dữ liệu (store procedure) hoặc các mô đun chương trình.

  Trong cơ sở dữ liệu quan hệ, không có phần cài đặt các phương thức cho lớp, chỉ có phần cài đặt các hàm, các trigger trên cơ sở dữ liệu giúp làm tăng tốc độ xử lý dữ liệu. Nếu các hàm nào có thể định nghĩa được trên cơ sở dữ liệu thì định nghĩa, nếu có phần nào không định nghĩa được, ta sẽ sử dụng ngôn ngữ lập trình để thay thế. Phần này sẽ cài đặt trực tiếp vào chương trình bằng ngôn ngữ lập trình tạo lập hệ thống.

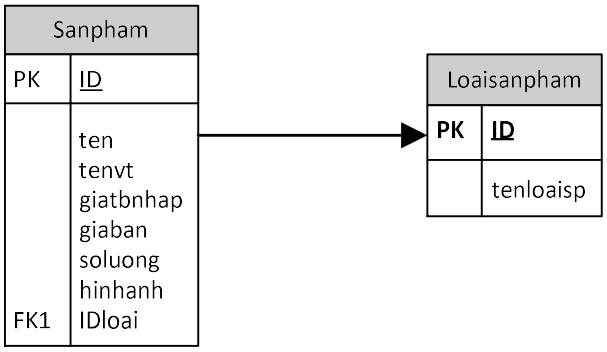
- Ánh xạ các giá trị đơn có mối quan hệ thành phần hoặc kết hợp thành cột khóa của bảng, khóa ngoại của bảng.

- Ví dụ đối tượng sản phẩm có thuộc tính ID là mã sản phẩm là duy nhất, có mối quan hệ kết hợp với các đối tượng như phiếu nhập, phiếu xuất nên được chuyển thành khóa chính.

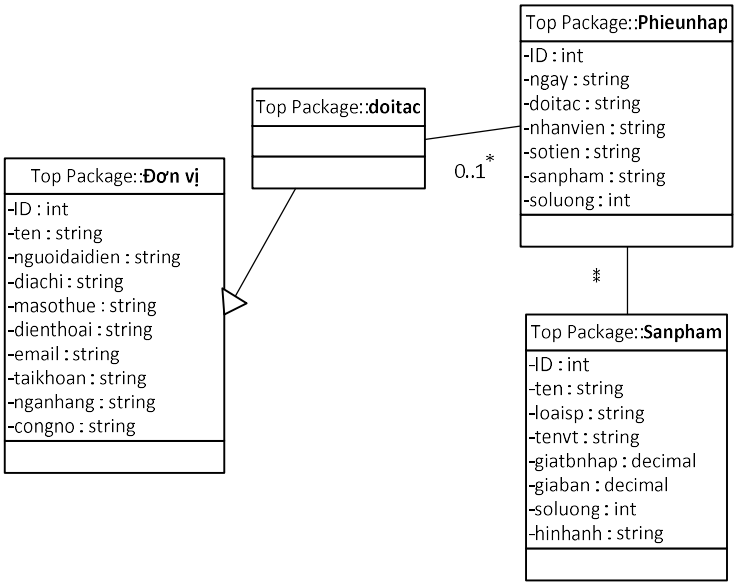
Sanpham	
PK	ID
	ten tenvt giatbnhap giaban soluong hinhanh
FK1	IDloai

- Ánh xạ các giá trị đa trị có mối quan hệ thành phần hoặc kết hợp thành một bảng mới và tạo mối quan hệ một nhiều với bảng gốc mà thuộc tính này được tách ra.

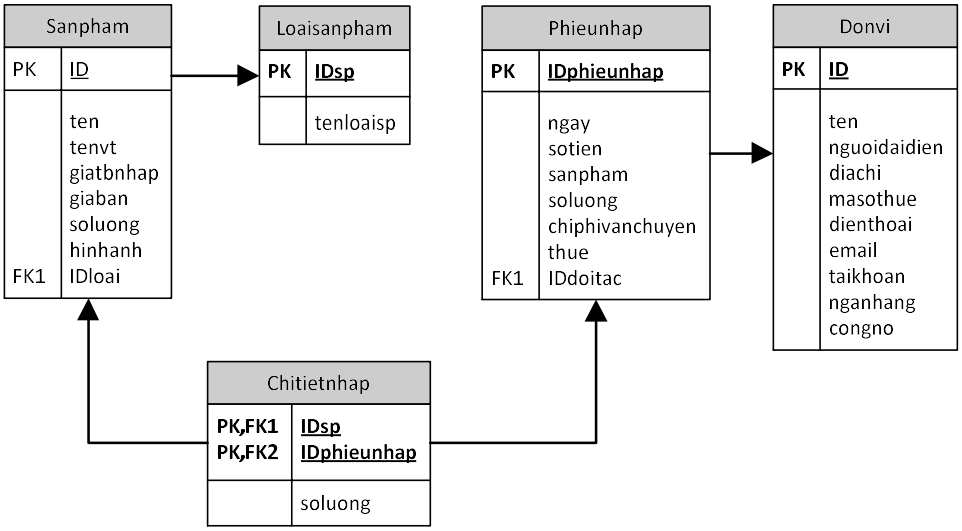
  Ví dụ từ loại sản phẩm, chúng ta tách ra một bảng riêng là loại sản phẩm và có mối quan hệ một nhiều với bảng sản phẩm.



- Ánh xạ mối quan hệ bao gồm hoặc kết hợp nhiều nhiều giữa hai lớp thành một bảng mới có mối quan hệ một nhiều với hai bảng gốc.

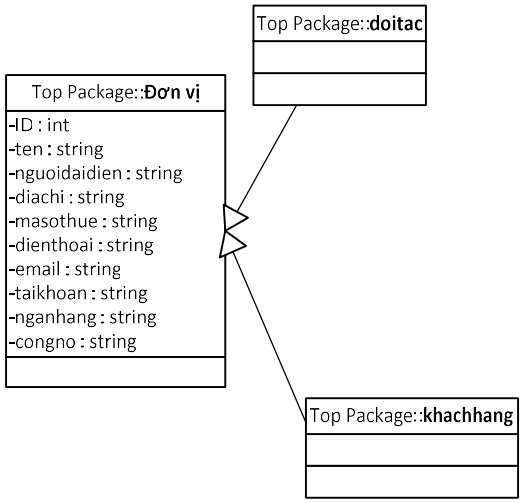


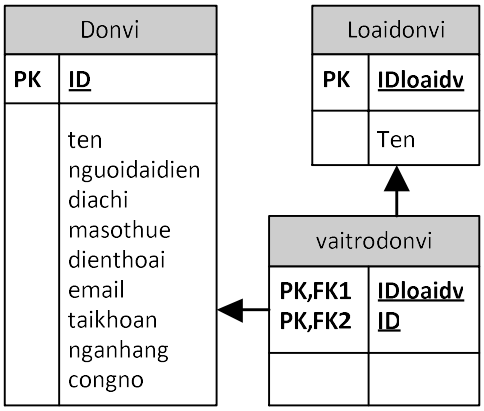
Trong ví dụ trên ta thấy giữa đối tượng phiếu nhập và sản phẩm có mối quan hệ nhiều nhiều, theo quy tắc, ta tách ra làm một bảng riêng, lấy hai khóa chính làm thành khóa ngoại cho bảng mới.



- Đối với các lớp kế thừa, chép tất cả các thuộc tính của lớp cha với lớp con tạo lớp mới. Nếu lớp cha và lớp con giống nhau về thuộc tính, khác nhau về vai trò và các phương thức, ta có thể cài đặt thêm thuộc tính để thể hiện vai trò cho đối tượng và cài đặt chung một bảng.

  Ví dụ các đối tượng là đối tác và khách hàng được kế thừa từ lớp đơn vị, chúng chỉ khác nhau về mặt vai trò, không khác nhau về mặt thuộc tính, cho nên chúng ta cài đặt chung vào một bảng. Một đơn vị có thể vừa là đối tác, vừa là khách hàng, bởi vậy thuộc tính vai trò trở nên đa trị, nên sẽ tách ra làm một bảng riêng. Từ đó ta có kết quả như sau:





Dựa vào các quy tắc trên ta đã chuyển dữ liệu sang mô hình quan hệ mức logic. Tiến hành thiết kế, lựa chọn dữ liệu phù hợp theo từng hệ quản trị, ta có dữ liệu thiết kế ở mức vật lý.

Trong quá trình thiết kế dữ liệu phải chú ý về các mặt không gian, thời gian. Dữ liệu sẽ tăng lên rất nhanh theo thời gian, cần phải lựa chọn cơ sở dữ liệu có quy mô phù hợp. Khi thiết kế, tránh dữ liệu bị trùng lắp cũng làm giảm không gian vùng dữ liệu. Ngoài ra, cần phải đáp ứng truy xuất nhanh cho dữ liệu khi dữ liệu trở nên đồ sộ. Một trong các yếu tố làm truy xuất nhanh dữ liệu đó là người lập trình phải tối ưu hóa các câu truy vấn, điều đó có thể dẫn đến thiết kế dữ liệu không đạt chuẩn và bị trùng lắp dữ liệu. Tùy theo từng trường hợp và kinh nghiệm của nhà thiết kế để lựa chọn biện pháp phù hợp nhất.

Trong quá trình thiết kế dữ liệu, chú ý các tham số liên quan đến các ràng buộc hoặc các yêu cầu phi chức năng nên được lưu trữ riêng thành bảng tham số.

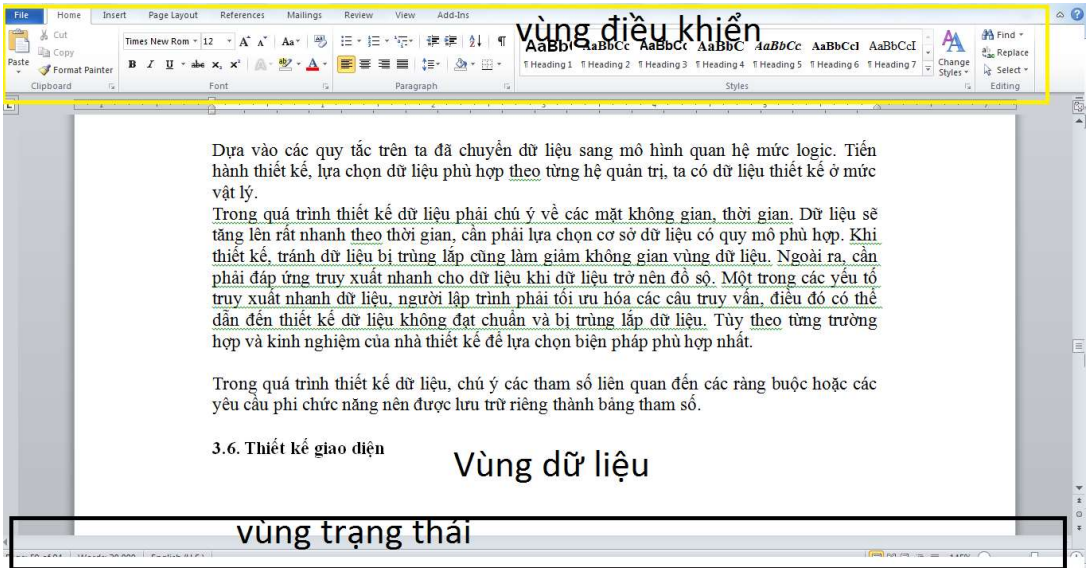
3.6. THIẾT KẾ GIAO DIỆN

Thiết kế giao diện là quá trình định nghĩa hệ thống sẽ tương tác với các thực thể bên ngoài như thế nào. Có thể nói, thiết kế giao diện là một nghệ thuật. Mục tiêu là làm cho giao diện trở nên bắt mắt, tạo cảm giác dễ chịu, và người dùng sử dụng đạt hiệu quả nhất. Vấn đề thường gặp phải đó là nhà thiết kế để quá nhiều thông tin thể hiện trên màn hình. Một mặt nó cung cấp mọi thông tin cần thiết, nhưng mặt khác lại làm rối mắt người dùng, tạo cảm giác khó chịu. Nhà thiết kế cần phải cân bằng giữa hai mặt này.

Các nguyên tắc trong thiết kế giao diện là:

Bố cục: Giao diện nên là một dãy các vùng được sử dụng theo mục đích và thống nhất cả hệ thống. Ví dụ phần phía trên là vùng điều khiển và điều hướng, phần giữa là thông tin đầu vào, đầu ra, và phần dưới thể hiện trạng thái.

Ví dụ:

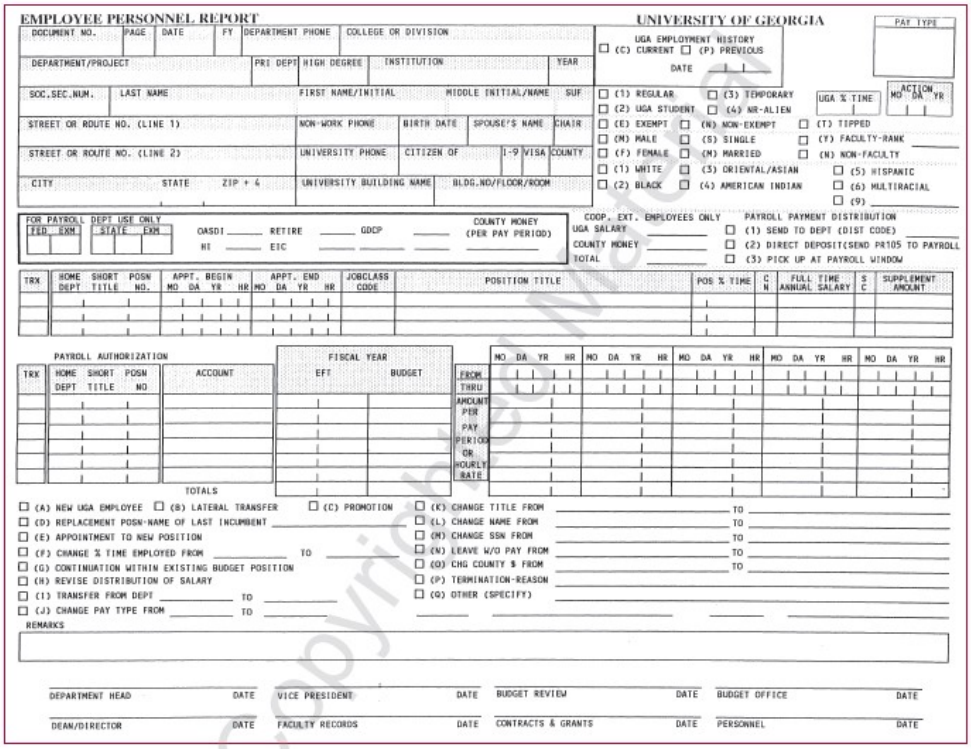


Hình 3-11: Các vùng giao diện

Nội dung: Tạo cho người dùng luôn có ý niệm mình đang ở đâu trong hệ thống, và những thông tin gì đang thể hiện. Tất cả các giao diện nên có tiêu đề. Thực đơn nên thể hiện người dùng đang ở đâu và có nếu có thể, thể hiện từ đâu đến.

Các vùng dữ liệu nên rõ ràng, người dùng dễ dàng xác định vùng dữ người dùng cần. Các ô nhập liệu cần có tiêu đề, các báo cáo cần phải có ngày tạo. Các tiêu đề cần ngắn gọn nhưng đầy đủ nghĩa. Những vùng quan trọng đôi khi có thể sử dụng đóng khung, màu sắc, in nghiêng hay gạch chân để nhấn mạnh.

Ví dụ: trong giao diện báo cáo nhân sự của trường đại học Georgia, có những vùng dữ liệu riêng được đóng khung để dễ phân biệt, có những tiêu đề viết tắt, có những phần viết đầy đủ, tùy thuộc vào vị trí và ý nghĩa.



Hình 3-12: Giao diện báo cáo

Tính thẩm mỹ: Giao diện cần có tính thẩm mỹ để mời gọi người dùng nhưng cũng đảm bảo chức năng hệ thống hoạt động tốt. Tính thẫm mỹ thể hiện qua việc sử dụng cẩn thận các khoảng trống, màu sắc, font chữ. Nên để giao diện có nhiều khoảng trống nhưng đảm bảo các thông tin cần thiết được thể hiện trên màn hình.

Kinh nghiệm người dùng: Cần phải có sự cân bằng giữa yếu tố dễ học sử dụng và sử dụng dễ dàng. Người dùng thông thường được chia làm hai loại, một là người dùng không có kinh nghiệm, không thường xuyên và hai là người dùng thường xuyên có kinh nghiệm. Đối với người dùng thứ nhất, họ mong muốn có một giao diện dễ học, dễ nhận biết các chức năng, có thể không qua tập huấn có thể sử dụng được. Ngược lại, đối với người dùng là chuyên gia, họ mong muốn có giao diện mà thông qua đó hoàn thành một quy trình công việc càng nhanh càng tốt. Điều này có thể dẫn đến một giao diện không thân thiện, cần phải tập huấn mới sử dụng được. Có thể bắt gặp các giao diện này trong các hệ thống đặc thù như bán vé máy bay, ngân hàng … Khi thiết kế, nhà thiết kế cần phải cân bằng giữa 2 yếu tốt này nhưng cũng phụ thuộc vào số người dùng của từng hệ thống.

Tính thống nhất: Tính thống nhất trong thiết kế giao diện cho phép người dùng dự đoán trước được cái gì sẽ xảy ra trước thực hiện một chức năng. Đây là một yếu tố quan trọng tạo ra tính dễ học, dễ sử dụng và thẩm mỹ. Khi người dùng đã quen với

một chức năng, họ sẽ quen làm việc với các chức năng khác nhanh chóng, vì họ biết trước cách tương tác của hệ thống. Giả sử người dùng quen sử dụng giao diện web, một hệ thống có giao diện giống website sẽ giảm thiểu được thời gian học sử dụng hệ thống đi rất nhiều. Ngoài ra cũng có thể thể hiện thông qua sử dụng chung từ ngữ, chung biểu tượng cho những chức năng, nút lệnh giống nhau, các báo cáo có hình thức trình bày gần giống nhau trong việc bố trí các khu vực thông tin giúp người dùng dễ theo giỏi hơn.

Tính tiện dụng: Giao diện phải thiết kế làm sao mang lại tính tiện lợi cho người dùng nhất. Điều này có nghĩa sử dụng ít nhất số lần bấm chuột hoặc phím để di chuyển từ phần này sang phần khác của màn hình. Nhiều nhà thiết kế tuân theo luật 3 lần bấm, có nghĩa từ khi bắt đầu vào sử dụng chức năng, người dùng không bấm quá 3 lần chuột hoặc phím dịch chuyển để hoàn thành chức năng đó.

Tính hiệu quả: Quy trình thực hiện phải tự nhiên nhất, đơn giản nhất, tôn trọng các thói quen người dùng. Tận dụng những bước xử lý trên màn hình, thông báo lỗi dữ liệu ngay khi có thể để hạn chế lỗi.

Tính tiến hóa: Có các tùy chọn về giao diện, các biến cố, hiển thị thêm các thông tin bổ sung giúp người dùng đưa ra quyết định đúng đắn. Đưa các dữ liệu đề nghị để rút ngắn thời gian nhập liệu. Ví dụ hiện thông tin ngày tháng nhập, mã số đề nghị …

3.6.1. Thiết kế giao diện đầu và thực đơn

Giao diện đầu hay còn gọi là giao diện chính là giao diện đầu tiên của hệ thống. Đối với một hệ thống trên winform hay một hệ thống web thì màn hình chính cũng có tính chất như sau:

- Giao diện thể hiện tóm tắt nội dung của hệ thống thông qua các hình ảnh hoặc các thông tin ngắn. Các thông tin thể hiện trên màn hình chính thường ngắn gọn, được sự quan tâm cao của hệ thống.
- Giao diện này phải có tính chất điều hướng cao, người dùng có thể vào thẳng chức năng mà họ quan tâm.
- Giao diện cần tạo cảm giác dễ chịu, thoải mái cho người dùng.

Đối với hệ thống thực đơn, thông thường sẽ được thể hiện trên giao diện chính và các giao diện còn lại. Thực đơn có thể thẻ hiện ở nhiều loại khác nhau:

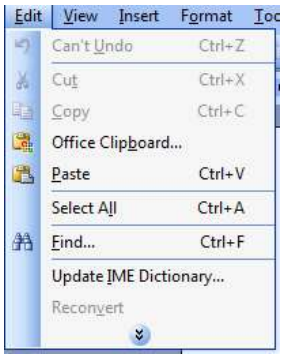
- Thanh thực đơn (menu bar): là thực đơn thể hiện ở dạng chữ, thường nằm ở phía trên cùng của giao diện.

Ví dụ:

Hình 3-12: Thanh thực đơn microsoft word

- Thực đơn sổ xuống (dropdown menu): là thực đơn thể hiện ở dạng chữ, thường là các thực đơn mở rộng cho các thực đơn nằm ngang khác.

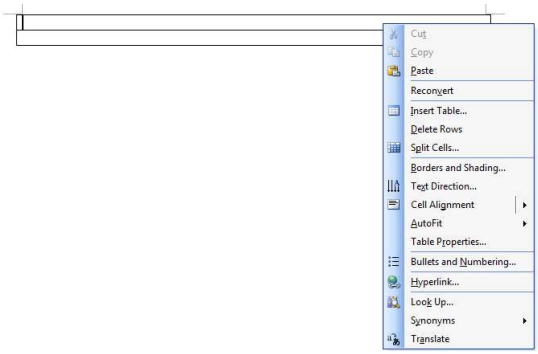
Ví dụ:



Hình 3-13: Thực đơn sổ xuống của microsoft word

- Thực đơn pop-up: Thường là các thực đơn phụ, giúp người dùng điều hướng đến các chức năng liên quan đến một đối tượng nhỏ trong nội dung của giao diện.

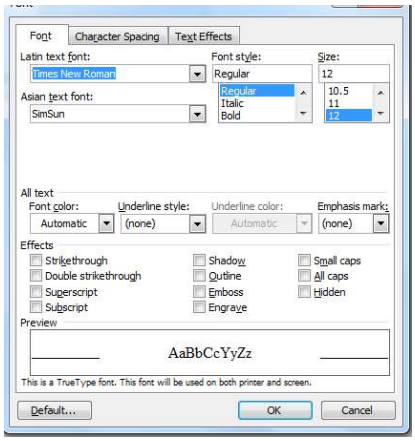
Ví dụ:



Hình 3-14: Thực đơn pop-up của đối tượng bảng trong microsoft word

- Thực đơn dạng thẻ (tab menu): thể hiện dạng thẻ, có nội dung trên thẻ là các từ ngắn gọn, mỗi một giao diện trên thẻ có chức năng liên quan đến nhau, cùng bổ sung cho một chủ đề nào đó.

Ví dụ:



Hình 3-15: Thực đơn tab trong microsoft word

- Thực đơn là thanh công cụ: Các chức năng được thể hiện dưới dạng hình ảnh, các icon nhỏ, có ý nghĩa tượng trưng cho chức năng đó.

Ví dụ:

Hình 3-16: Thanh công cụ trong microsoft word

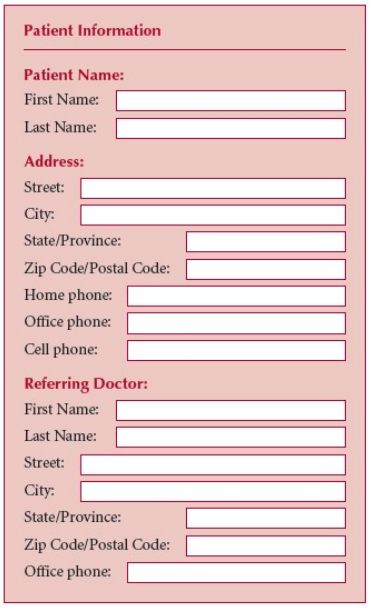
- Ngoài ra thực đơn được thể hiện dưới dạng các nút bấm, hay hình ảnh …

Cách tổ chức thực đơn có thể gom nhóm theo đối tượng hoặc gom nhóm theo chức năng. Các thực đợn không thực hiện được thường ở dưới dạng chữ hoặc hình ảnh mờ.

3.6.2. Giao diện nhập liệu

Đối với giao diện nhập liệu, việc thiết kế các ô nhập liệu phải đảm bảo tính đúng đắn, đầy đủ. Tất cả các thông tin cần thu thập từ người dùng cần phải thiết kế đầy đủ.

Ví dụ:



Hình 3-17: Giao diện nhập liệu tạo buổi hẹn cho bệnh nhân

Khi nhập liệu, cần lựa chọn các hình thức ô nhập liệu khác nhau tạo tính tiện lợi cho người dùng sử dụng.

Ví dụ:

Thay vì sử dụng:

Ta sử dụng radio box:

Các loại ô nhập liệu:

Tên	Ký hiệu
Textbox: - Có thể một dòng hoặc nhiều dòng - Không dùng textbox khi có thể dùng listbox hoặc combobox - Có nhãn đi kèm	Enter Text
Selectbox hay combobox: - Thể hiện danh sách chọn lựa nhưng ở dạng rút gọn - Số lượng lựa chọn thường là ngắn	Enter Text

Listbox: - Thể hiện danh sách các lựa chọn trên một khung rộng - Số lượng lựa chọn có thể nhiều	Column 1 Text Text Text
Checkbox: - Lựa chọn các thông tin có sẵn - Có thể lựa chọn nhiều mục cùng lúc	☑
Radiobox: - Lựa chọn các thông tin đầu vào có sẵn - Chỉ chọn được một trong số các lựa chọn	◉

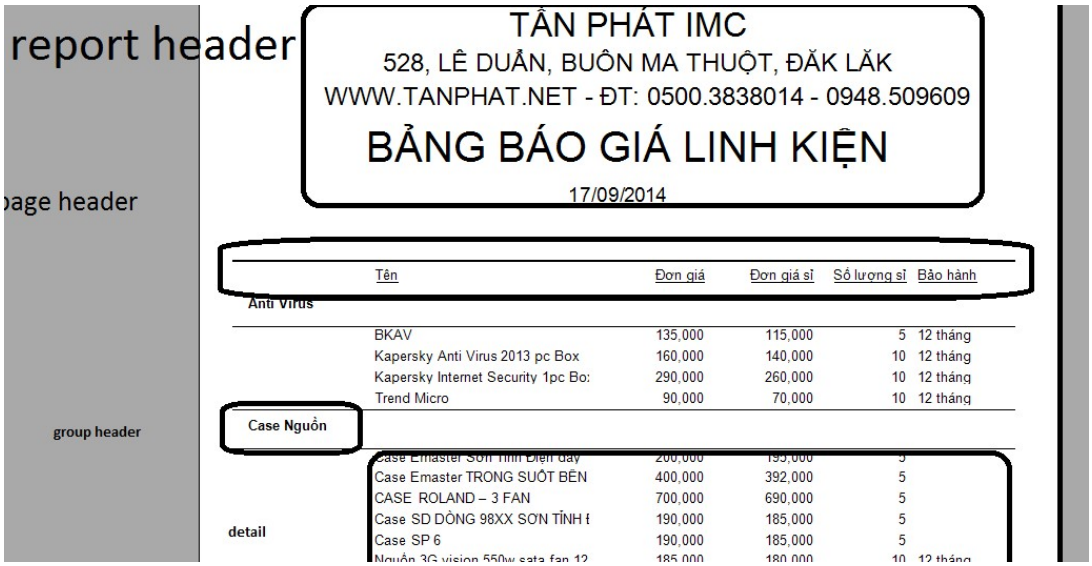
3.6.3. Giao diện báo cáo

Giao diện của báo cáo thông thường tuân thủ theo các quy tắc soạn thảo văn bản. Nội dụng trên báo thường rất nhiều nên có thể sử dụng các kỹ thuật như gạch chân, in đậm, in nghiêng, khung đóng, font chữ, kích thước chữ để nhấn mạnh các thông tin quan trọng. Có thể sử dụng các bảng biểu hoặc biểu đồ để thể hiện ngắn gọn, súc tích với người xem.

Cấu trúc một báo cáo thông thường bao gồm các thành phần:

Tên	Nội dung
Report header	- Phần đầu của báo cáo - Chỉ hiển thị một lần trong phần đầu tiên của báo cáo
Report footer	- Phần cuối cùng của báo cáo - Chỉ hiện thị một lần trong phần cuối của báo cáo
Page header	- Phần đầu của trang giấy - Lập lại đầu mỗi trang báo cáo
Page footer	- Phần cuối trang giấy - Lập lại mỗi cuối trang giấy
Group header	- Phần đầu của mỗi nhóm - Lập lại mỗi nhóm - Mỗi nhóm thì có thể được lặp lại nhiều lần ở trong một trang
Group footer	- Phần cuối mỗi nhóm - Lập lại mỗi nhóm
Detail	- Phần nội dung chi tiết của nhóm hoặc của trang

Ví dụ:

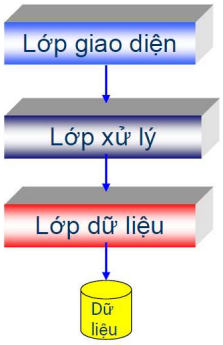


Hình 3-18: Màn hình báo cáo

3.7. THIẾT KẾ XỬ LÝ MÔ HÌNH 3 LỚP

Có nhiều cách thiết kế xử lý để thực hiện được một chức năng nào đó của hệ thống. Trong một số trường hợp, lập trình viên có thể thực hiện các thao tác nhập xuất, xử lý và lưu trữ vào cơ sở dữ liệu tại phần mã lệnh xử lý giao diện. Điều này dẫn tới mã lệnh không rõ ràng, khi cần điều chỉnh về cơ sở dữ liệu hoặc phương pháp xử lý, gần như phải điều chỉnh tại rất nhiều giao diện khác nhau.

Mô hình lập trình 3 lớp ra đời khắc phục điều này với các lớp như sau:



Lớp giao diện (giao tiếp với người sử dụng): chỉ thuần xử lý việc giao tiếp với người sử dụng, nhập xuất, … mà không thực hiện việc tính toán, kiểm tra, xử lý, hay các thao tác liên quan đến cơ sở dữ liệu. Trong lập trình winform, các lớp này là các form được thiết kế bằng cách kéo thả các control và khai báo các đối tượng xử lý để tiếp nhận các thông tin đầu vào.

Lớp xử lý: Lớp này chuyên thực hiện các xử lý, kiểm tra các ràng buộc, các qui tắc ứng xử của phần mềm, các chức năng cốt yếu, … Việc thực hiện này độc lập với cách thiết kế cũng như cài đặt giao diện. Thông tin cho lớp này thực hiện các xử lý của mình được lấy từ lớp giao diện. Trong lập trình hướng đối tượng, đây là các lớp liên quan đến các đối tướng được xử lý trong các giao diện và lưu trữ trong cơ sở dữ liệu.

Lớp dữ liệu: Lớp này chuyên thực hiện các công việc liên quan đến dữ liệu. Dữ liệu có thể lấy từ cơ sở dữ liệu (Access, SQL Server …) hoặc tập tin (text, binary, XML …). Đối với cơ sở dữ liệu, lớp này thực hiện kết nối trực tiếp với cơ sở dữ liệu và thực hiện tất cả các thao tác liên quan đến cơ sở dữ liệu mà phần mềm cần thiết. Đối với tập tin, lớp này thực hiện việc đọc, ghi tập tin theo yêu cầu của phần mềm. Việc thực hiện này do lớp xử lý gọi. Lớp này là cầu nối giữa cơ sở dữ liệu và lớp xử lý.

TỔNG KẾT CUỐI CHƯƠNG

Thiết kế không phải là một giai đoạn độc lập, thông thường giai đoạn này sẽ được gọi lại nhiều lần trong vòng lặp phân tích đến thiết kế cho đến khi hoàn thành. Tuy nhiên trong pha thiết kế, các hoạt động cần có là: thiết kế kiến trúc, thiết kế xử lý, thiết kế đối tượng hay thiết kế dữ liệu và thiết kế giao diện.

Tùy theo quy mô và tính chất phần mềm mà ta lựa chọn kiến trúc phù hợp. Có nhiều mẫu kiến trúc xây dựng sẵn để người lập trình lựa chọn. Sau khi lựa chọn kiến trúc, người thiết kế xác lập quy trình tương tác xử lý từng chức năng bằng các sơ đồ UML như sơ đồ tuần tự, sơ đồ trạng thái. Để gần hơn tới phần cài đặt, nhà thiết kế xác định các đối tượng, thuộc tính và phương thức của nó thông qua sơ đồ lớp, và chuyển sang lược đồ quan hệ nếu cần thiết. Hoạt động cuối cùng là thiết kế giao diện. Thiết kế giao diện phải thỏa mãn những tính chất như tính thẩm mỹ, tính đúng đắn, tính tiến hóa …

CÂU HỎI ÔN TẬP CHƯƠNG III

3.1. Kiến trúc hệ thống là gì? Liệt kê và mô tả một số kiến trúc mẫu?

3.2. Ý nghĩa sơ đồ tuần tự, sơ đồ trạng thái là gì? Khi nào sử dụng hai sơ đồ này?

3.3. Liệt kê các phương pháp xác định đối tượng?

3.4. Ý nghĩa sơ đồ lớp là gì? Khi nào sử dụng sơ đồ lớp?

3.5. Nêu các phương pháp chuyển đổi từ sơ đồ lớp sang lược đồ quan hệ?

3.6. Nêu các tính chất cần có của giao diện phần mềm?

3.7. Có mấy loại giao diện, tính chất của từng loại?

3.8. Theo mô tả và các bản phân tích chức năng của bài 2.7, thực hiện thiết kế xử lý, thiết kế sơ đồ lớp, thiết kế dữ liệu, thiết kế xữ lý, cài đặt phần mềm.

Chương 4: KIỂM TRA CHẤT LƯỢNG PHẦN MỀM

Sau khi phần mềm đã hoàn thành, các tổ chức sản xuất phần mềmthuường tiến hành kiểm tra chất lượng phần mềm. Trong chương này, chúng ta sẽ nghiên cứu về độ tin cậy phần mềm, các chiến lược kiểm tra phần mềm và các công cụ kiểm thử phần mềm.

Để hiểu rõ chương này, người dùng cần có các trãi nghiệm trong lập trình, từ đó có thể áp dụng các cấu trúc lập trình, các công cụ đề nghị trong chương này để thấy kết quả rõ ràng hơn. Sau khi kết thúc chương này, người học biết và vận dụng được các phương pháp kiểm tra chất lượng phần mềm, đồng thời cũng nâng cao kỹ năng lập trình để tăng khả năng chịu lỗi của phần mềm.

4.1. ĐỘ TIN CẬY CỦA PHẦN MỀM

Khi xây dựng phần mềm thường hay gặp những lỗi sau:

- Khi phân tích chất lượng, phần mềm thường gặp một số lỗi như:
- Lỗi chiến lược: ý đồ thiết kế sai
- Phân tích các yêu cầu không đầy đủ hoặc lệch lạc
- Hiểu sai về các chức năng
- Vi phạm nguyên lý đối tượng
- Lỗi tại các thủ tục chịu tải, đây là những lỗi nặng.
- Lỗi lây lan: lỗi được truyền từ chương trình này sang chương trình khác
- Lỗi cú pháp: viết sai quy định của ngôn ngữ.

Do vậy, khi phát triển phần mềm, cần đảm bảo nguyên lý an toàn là: Mọi lỗi dù nhỏ hay lớn đều phải được phát hiện ở một bước nào đó của chương trình, trước khi chương trình hoàn thành.

Độ tin cậy của một hệ phần mềm là độ đo về mức độ tốt của các dịch vụ mà hệ cung cấp cho máy tính. Cần chú ý là người dùng không xét rằng các dịch vụ là quan trọng như nhau: chẳng hạn một hệ điều khiển máy bay có thể rất hiếm khi thất bại, nhưng nếu có thất bại gây ra tai nạn máy bay thì người bị nạn và thân nhân người bị nạn không thể xem hệ đó là đáng tin.

Độ tin cậy là một đặc trưng động của hệ thống, nó là một hàm của số các thất bại phần mềm. Một thất bại phần mềm là một sự kiện thi hành mà khi đó phần mềm hành xử không như người ta mong đợi. Chú ý rằng một thất bại phần mềm khác một hư hỏng phần mềm. Hư hỏng phần mềm là một đặc trưng tĩnh, và nó sẽ gây ra thất bại phần mềm khi mà mã lỗi được thi hành với một tập hợp đặc biệt các thông tin vào. Các hư hỏng không phải luôn luôn xuất đầu lộ diện, vì vậy đọ tin cậy phụ thuộc vào việc sử dụng hệ thống như thế nào.

Lập trình là một là một công đoạn phụ thuộc nhiều vào kỹ xảo cá nhân, sự chú ý đến các chi tiết và kiến thức về việc làm như thế nào để sử dụng các công cụ sẵn có theo cách thức tốt nhất. Nhu cầu các hệ thống đáng tin là đang tăng lên vì vậy cần có các kỹ thuật chuyên biệt nhằm đạt được một hệ thống tin cậy được. Hiện nay, có hai kỹ thuật để tăng độ tin cậy phần mềm khi viết các chương trình ứng dụng là: tránh lỗi và chịu lỗi.

➢ Tránh lỗi:

Tất cả các kỹ sư phần mềm đều muốn tạo ra các phần mềm không có lỗi. Một quá trình phát triển dựa vào việc phát hiện lỗi và trừ khử lỗi chứ không phải là tránh lỗi là một quá trình kém cõi và lỗi thời.

Phần mềm không có lỗi ở đây là phần mềm tuân theo đúng đặc tả. Trong khi đó, có thể có lỗi trong đặc tả hoặc có thể bản đặc tả không phản ánh đúng các nhu cầu của người sử dụng vậy là phần mềm không có lỗi không chắc chắn luôn luôn hành xử như người dùng dự đoán.

Việc phát triển phần mềm không có lỗi là một việc rất đắt đỏ, và khi mà một số lỗi đã được tháo khỏi chương trình thì giá cả cho việc tìm và tháo lỗi còn lại có xu hướng tăng theo hàm số mũ. Do đó một tổ chức có thể quyết định chấp nhận một vài lỗi còn lưu lại. Tính về mặt giá cả thì việc chi trả cho các thất bại của hệ thống do các lỗi đó gây ra ít hơn là đi phát hiện tháo gỡ các lỗi đó trước khi phân phối.

Tránh lỗi và phát triển phần mềm vô lỗi dựa trên:

- Sản phẩm của một đặc tả hệ thống chính xác.
- Chấp nhận một cách tiếp cận thiết kế phần mềm dựa trên việc che dấu thông tin và bao gói thông tin.
- Tăng cường việc duyệt lại trong quá trình phát triển và thẩm định hệ phần mềm.
- Việc lập kế hoạch cẩn thận cho việc thử nghiệm hệ thống để phát hiện ra các lỗi mà các lỗi này chưa được phát hiện trong quá trình duyệt lại và để định lượng độ tin cậy của hệ thống.

➢ Chịu lỗi

Có bốn hoạt động cần phải tiến hành đối với hệ thống chịu lỗi:

- Phát hiện lỗi
- Định ra mức độ thiệt hại
- Hồi phục sau khi gặp lỗi: Hệ thống phải hồi phục về trạng thái mà nó biết là an toàn. Cũng có thể là chỉnh lý trạng thái bị hủy hoại (hồi phục tiến), cũng có thể là lui về một trạng thái trước mà an toàn (hồi phục lùi).
- Chữa lỗi: Cải tiến hệ thống để cho lỗi đó không xuất hiện nữa. Trong nhiều

trường hợp sự thất bại của phần mềm là tàng hình và gây ra bởi một tổ hợp đặc biệt của thông tin vào.

- **Lập trình phòng thủ**

Lập trình phòng thủ là cách phát triển chương trình mà người lập trình giả định rằng các mâu thuẫn hoặc các lỗi chưa được phát hiện có thể tồn tại trong chương trình. Mã sẽ có phần kiểm tra trạng thái hệ thống sau khi biến đổi và phải đảm bảo rằng sự biến đổi trạng thái là kiên định. Nếu phát hiện một mâu thuẫn thì việc biến đổi trạng thái là rút lại và trạng thái phải trở về trạng thái đúng đắn trước đó.

Lập trình phòng thủ là một cách chịu lỗi, mà được tiến hành không cần bộ điều khiển về chịu lỗi. Về cơ bản quá trình vẫn là: phát hiện lỗi, đánh giá lỗi, và phục hồi sau lỗi.

Hồi phục lỗi là một quá trình cải biên không gian trạng thái của hệ thống sao cho hiệu ứng của lỗi là nhỏ nhất và hệ thống có thể tiếp tục vận hành, có thể trong một mức suy giảm. Hồi phục lùi liên quan đến việc lưu trạng thái của hệ thống ở một trạng thái đúng đã biết.

Hồi phục tiến thường là một chuyên biệt ứng dụng, mà kiến thức miền được sử dụng để tính ra các sự chỉnh lý có thể. Tuy nhiên có hai tình thế chung mà khi đó hồi phục tiến có thể thành công:

- Khi dữ liệu bị hư hỏng: Việc sử dụng mã hóa thích hợp bằng cách thêm các dữ liệu dư thừa vào dữ liệu cho phép sửa sai khi phát hiện lỗi.
- Khi cấu trúc nối bị hỏng: Nếu các con trỏ tiến và lùi đã có trong cấu trúc dữ liệu thì cấu trúc đó có thể tái tạo nếu như còn đủ các con trỏ chưa bị hỏng. Kỹ thuật này thường được dùng cho việc sửa chữa hệ thống tệp và cơ sở dữ liệu.

Hồi phục lùi là một kỹ thuật đơn giản liên quan đến việc duy trì các chi tiết của trạng thái an toàn và cất giữ trạng thái đó khi mà một sai đã bị phát hiện. Hầu hết các hệ quản trị cơ sở dữ liệu đều có bộ phục hồi lỗi. Cơ sở dữ liệu chỉ cập nhật dữ liệu một khi giao dịch đã hoàn tất và không phát hiện được vấn đề gì. Nếu giao dịch thất bại thì cơ sở dữ liệu không được cập nhật.

Một kỹ thuật khác là thiết lập các điểm kiểm tra thường kỳ mà chúng là các bản sao của trạng thái hệ thống. Khi mà một lỗi được phát hiện thì trạng thái an toàn đó được tái lưu kho từ điểm kiểm tra gần nhất.

4.2. KHÁI NIỆM KIỂM TRA VÀ CHIẾN LƯỢC KIỂM TRA PHẦN MỀM

4.2.1. Đặc điểm kiểm tra phần mềm

Xác minh và thẩm định một hệ phần mềm là một quá trình liên tục xuyên suốt mọi

giai đoạn của quá trình phần mềm. Xác minh và thẩm định là từ chung cho các quá trình kiểm tra để đảm bảo rằng phần mềm thỏa mãn các yêu cầu của chúng và các yêu cầu đó thỏa mãn các nhu cầu của người dùng.

Xác minh và thẩm định là một quá trình kéo dài suốt vòng đời. Nó bắt đầu khi duyệt xét yêu cầu. Xác minh và thẩm định có hai mục tiêu: phát hiện các khuyết tật trong hệ thống và đánh giá xem hệ thống liệu có dùng được hay không?

Như trên, kiểm tra là một quá trình của tìm kiếm lỗi. Một kiểm tra tốt có khả năng cao tìm kiếm các lỗi chưa được phát hiện. Một kiểm tra thành công là kiểm tra tìm ra các lỗi mới, một kiểm tra tồi là kiểm tra mà không tìm được lỗi. Có hai kiểu lỗi trong ứng dụng và các kiểm tra tốt sẽ xác định cả hai loại đó. Cụ thể:

- Không làm những điều cần phải làm: lỗi bỏ sót thường xuất hiện đối với ứng dụng mới được phát triển.
- Làm những điều không cần làm: lỗi của lệnh đã cư trú trước trong các ứng dụng bảo trì.

Các kiểm tra có mặt tại các mức khác nhau và được tiến hành bởi các cá nhân khác nhau trong quá trình phát triển ứng dụng. Các kiểm tra được tiến hành bởi đội ngũ dự án và kiểm tra được tiến hành bởi các cơ quan bên ngoài để chấp nhận ứng dụng.

Các kiểm tra của đội dự án được gọi là kiểm tra phát triển (Development test). Kiểm tra phát triển bao gồm kiểm tra đơn vị, kiểm tra hệ thống con, kiểm tra tích hợp và các kiểm tra hệ thống.

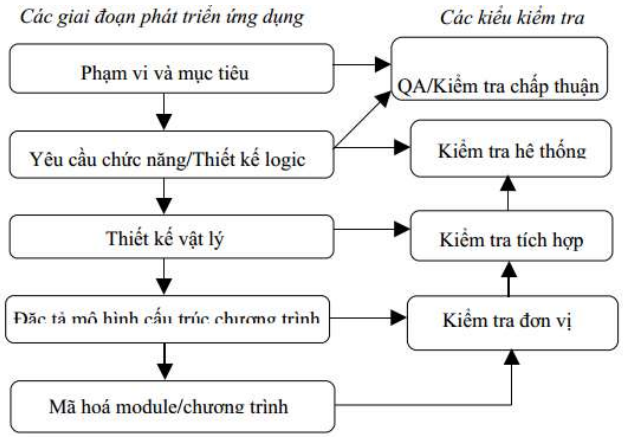
- Kiểm tra đơn vị (Unit test) được tiến hành cho mỗi đơn vị mã nhỏ nhất.
- Kiểm tra tích hợp (Subsystem integration test) kiểm tra mặt logic và xử lý cho phù hợp của các khối, kiểm tra việc truyền tin giữa chúng.
- Kiểm tra hệ thống (System test) đánh giá các đặc tả chức năng có được đáp ứng, các thao tác giao diện người có giống thiết kế không, và các công việc của ứng dụng trong môi trường thao tác mong muốn, đối với các ràng buộc.

Các kiểm tra bởi các cơ quan bên ngoài được gọi là đảm bảo chất lượng (Quality assurance-QA) và kiểm tra chấp nhận (Acceptance test). Người ngoài có thể là người sử dụng hoặc đại diện người dùng, nó tạo một mục đích, đánh giá khách quan về ứng dụng.

Kiểm tra đảm bảo chất lượng tương tự các kiểm tra hệ thống về mặt mục đích và tiến hành, nhưng nó khác là người kiểm tra nằm ngoài sự điều khiển của dự án trưởng. Các báo cáo kiểm tra đảm bảo chất lượng được gửi thường xuyên tới nhóm phát triển và quản lý dự án. Các kiểm tra viên đảm bảo chất lượng lập kế hoạch cho chiến lược

của mình và tiến hành kiểm tra để đảm bảo các ứng dụng thực hiện tất cả các chức năng cần thiết. Kiểm tra đảm bảo chất lượng là kiểm tra cuối cùng được làm trước khi ứng dụng được đưa sản xuất đại trà.

Quan hệ giữa các mức kiểm tra và các giai đoạn trong vòng đời của chương trình được trình bày như sau:



Quá trình tiến hành kiểm tra bắt đầu từ khi thiết kế. Cộng tác viên thực hiện việc kiểm tra nên có khả năng của phân tích viên và lập trình viên để có thể hiểu các đòi hỏi của ứng dụng và kiểu cách tiến hành kiểm tra. Chương trình càng lớn và phức tạp thì càng cần người có kinh nghiệm, đôi khi cần có một đội ngũ kiểm tra. Đội ngũ kiểm tra sử dụng yêu cầu chức năng từ giai đoạn phân tích và đặc tả chương trình, đặc tả thiết kế từ giai đoạn thiết kế như là đầu vào để phát triển chiến lược kiểm tra hệ thống. Khi chiến lược đã được hoàn tất, các buổi thảo luận được tiến hành để kiểm tra lại chiến lược và thông báo tới toàn thể đội. Các nhiệm vụ tại các mức sẽ được ấn định. Thời gian được ước lượng. Đội ngũ kiểm tra làm việc độc lập và song song với đội ngũ lập trình. Họ làm việc với quản trị CSDL để phát triển cơ sở dữ liệu mà có thể hỗ trợ tất cả các mức kiểm tra. Đối với kiểm tra đơn vị, đội kiểm tra kiểm tra kết quả, các chấp nhận các mô đune và chương trình cho kiểm tra tích hợp (integration test). Đội ngũ kiểm tra tiến hành và định lượng các kiểm tra tích hợp và kiểm tra hệ thống.

4.2.2. Chiến lược kiểm tra phần mềm

Có hai kiểu chiến lược kiểm tra. Kiểu thứ nhất liên quan logic được kiểm tra thế nào trong ứng dụng. Chiến lược kiểm tra logic có thể là blackbox hoặc white-box. Chiến lược kiểm tra liên quan đến cách triển khai kiểm tra chia làm hai loại là top-down (từ trên xuống), hoặc bottom-up (từ dưới lên).

- **Chiến lược kiểm tra black-box:**

Chiến lược kiểm tra black-box cho rằng mô đune của chương trình hoặc hệ thống liên quan tới đầu vào và đầu ra. Các chi tiết logic chi tiết được che dấu và không cần phân tích. Chiến lược black-box có tính hướng dữ liệu. Kiểm tra hộp đen, black-box, coi xử lý kết quả như là minh chứng bởi dữ liệu. Khái niệm kiểm tra là black-box không quan tâm logic. Khuynh hướng này hiệu quả đối với các mô đun chức năng đơn và các hệ thống cấp cao. Ba phương pháp chính là:

- **Phân hoạch cân bằng**: Mục đích của phương pháp này là tối thiểu các trường hợp kiểm tra cho trước, các mục dữ liệu vào được chia thành các nhóm dữ liệu có vai trò cân bằng nhau, mỗi nhóm đại diện cho một tập dữ liệu. Nguyên tắc là bằng cách kiểm tra triệt để một mục của mỗi tập hợp, chúng ta có thể chấp nhận tất cả các mục tương đương khác của tập hợp đó cũng sẽ được kiểm tra một cách kỹ càng.
- **Phân tích cực biên**: Phân tích giá trị cực biên là một dạng nghiêm ngặt của phân hoạch cân bằng có sử dụng các giá trị biên hơn là bất cứ giá trị nào trong tập cân bằng. Ví dụ: miền giá trị của tháng là 1 – 12 và ngoài là 0 và 13. Thì 4 kiểm tra ứng với bốn trường hợp sẽ được kiểm tra phân tích cực biên thường xuyên được dùng tại các mức mô đun để xác định các mục dữ liệu đặc trưng cho testing.
- **Đoán lỗi**: Dựa trên cơ sở trực giác và kinh nghiệm, các chuyên gia có thể dễ dàng kiểm tra các điều kiện lỗi bằng cách đoán cái nào dễ xảy ra nhất. Ví dụ, chia 0, nếu mô đun có phép chia, nên dùng phép chia 0. Vì dựa trên trực giác, nên phép thử này khó tìm hết các lỗi.

➢ **Chiến lược kiểm tra white-box**:

White-box hướng tới việc cho rằng logic đặc trưng là quan trọng và cần phải kiểm tra. White-box đánh giá một vài hoặc tất cả mặt logic để kiểm tra được tính đúng đắn của chức năng. White-box hướng về logic - giải thuật. Có ba loại kiểm tra hộp trắng là kiểm tra logic -logic test, chứng minh toán học -mathematical proof và Cleanroom testing.

- Logic test có thể được chi tiết đến mức lệnh. Trong khi thực hiện của mọi lệnh là mục đích đáng khen ngợi, nó có thể không kiểm tra tất cả các điều kiện thuộc chương trình. Ví dụ, ít nhất 2 kiểm tra cho một kiểm tra 2 điều kiện (1 đúng và 1 sai). Cố gắng kiểm tra tất cả các điều kiện lẽ dĩ nhiên là không thể thực hiện được về thực tiễn. Ví dụ trong 1 mô đune có 10 thao tác qua 4 vòng lặp thì có 5,5 triệu trường hợp kiểm tra. Do đó phải có phương pháp lựa chọn.
- **Chứng minh bằng toán học**: Một phương pháp theo cách tiếp cận giảm thiểu sót về 0 là áp dụng suy diễn toán học cho đòi hỏi logic, chứng minh tính đúng đắn của chương trình. Phương pháp này đòi hỏi đặc tả ngôn ngữ dạng hình thức.

- **Cleanroom testing** là mở rộng của phương pháp chứng minh bằng toán học. Lý thuyết của kỹ thuật này là bằng cách ngăn chặn các lỗi tại các đầu vào của quá trình xử lý, giá thành sẽ giảm, độ tin cậy tăng lên và tiệm cận tới không có lỗi.

Cleanroom testing là kỹ thuật kiểm tra toán học hình thức và hội ý (walkthrough). Mục đích là phân tích mọi mô đun thành các chức năng và liên kết. Các phép kiểm tra chức năng dùng kỹ thuật toán học, các kiểm tra liên kết sử dụng thuyết tập hợp.

Sau khi thực hiện kiểm tra (verification), đội kiểm tra độc lập sẽ dịch và thực hiện mã. Dữ liệu kiểm tra được dịch bởi các phân tích đặc tả chức năng và được thực hiện thể hiện các tỷ lệ xác suất của dữ liệu được giả định cho hệ thống thực. Thêm vào các dữ liệu chuẩn thì các loại lỗi nghiêm trọng được tạo để kiểm tra ứng dụng. Bất lợi của phương pháp này bắt buộc là đòi hỏi kỹ năng cao về: toán, thống kê, logic và ngôn ngữ đặc tả hình thức.

➢ **Kiểm tra top-down**

Phương pháp kiểm tra top-down cần một mã ngoài, được hiểu như là một bộ khung để gắn các chức năng gốc, các mô dun, và các phần khác của ứng dụng. Bộ khung này thường bắt đầu với ngôn ngữ điều khiển công việc và logic chính của ứng dụng. Logic chính được kiểm tra và lập khung theo các hệ thống phân rã. Đầu tiên chỉ có các thủ tục thiết yếu và các logic điều khiển được kiểm tra.

Khi các mô đun thiết yếu nhất đã được kiểm tra và chạy tốt thì mã của các mô đun ít quan trọng hơn sẽ được gắn vào khung và tiếp tục kiểm tra. Về lý thuyết cách thức top-down sẽ tìm thấy các lỗi thiết kế sớm hơn trong kiểm tra thao tác (testing process) hơn các khuynh hướng khác. Phương pháp này ít hiệu quả trong việc cải thiện chất lượng của các phần mềm chuyển giao văn bản chất tương tác của kiểm tra.

Kiểm tra top-down dễ dàng hỗ trợ giao diện người dùng và thiết kế màn hình. Trong các ứng dụng tương tác, thường là bộ điều khiển màn hình được kiểm tra đầu tiên. Người dùng có thể kiểm tra sự điều khiển thông qua màn hình với các phát triển tạo mẫu.

➢ **Kiểm tra bottom-up**

Nguyên tắc của bottom-up là kiểm tra mọi thay đổi tại mô đun có thể ảnh hưởng tới chức năng của nó. Trong kiểm tra bottom-up, toàn bộ khối là đơn vị để đánh giá. Tất cả các mô đun được mã hoá và kiểm tra riêng rẽ.

Các trường hợp kiểm tra: các trường hợp kiểm tra là dữ liệu vào được tạo để thể hiện từng khối và toàn bộ hệ thống thoả mãn tất cả các yêu cầu thiết kế.

Mỗi trường hợp kiểm tra nên được phát triển để kiểm tra nghiệm các đòi hỏi thiết kế đặc trưng, thiết kế chức năng, hoặc mã đã được thoả mãn. Hơn nữa các trường hợp kiểm tra cần dự đoán các output.

Mỗi đơn nguyên của ứng dụng (Ví dụ: mô đun, subroutine, program, utility,...) phải được kiểm tra với ít nhất hai trường hợp kiểm tra: một trường hợp chạy tốt và một trường hợp không chạy. Trong trường hợp chạy sai hệ phải đưa được thông báo, quay lại (rollback) được trạng thái ban đầu của giao dịch.

Để chắc chắn rằng các trường hợp được toàn diện nhất, người ta thường dùng ma trận. Chúng được dùng cho:

- Kiểm tra đơn khối để định nhánh logic, điều kiện logic, các phần dữ liệu hoặc biên dữ liệu để kiểm tra trên cơ sở đặc tả chương trình.
- Kiểm tra tổ hợp để định ra yêu cầu về dữ liệu và quan hệ trong số các tương tác.
- Kiểm tra hệ thống để xác định yêu cầu về người dùng và hệ thống từ các yêu cầu chức năng và các yêu cầu chấp nhận.

Phối hợp các kiểm tra trong một chiến lược: mục đích của các nhà kiểm tra là

tìm ra sự cân bằng của các chiến lược cho phép họ chứng minh được ứng dụng chạy tốt mà tối thiểu hoá chi phí máy tính và nhân lực. Sử dụng duy nhất một chiến lược là rất nguy hiểm. Không có cái nào là duy nhất đúng. Nếu chỉ có white-box thì tài nguyên nhân lực và máy là rất tốn kém, nếu chỉ có black-box các vấn đề logic đặc trưng có thể chưa được khám phá.

4.3. KIỂM THỬ VÀ CÁC CÔNG CỤ TRỢ GIÚP KIỂM THỬ

4.3.1. Khái niệm kiểm thử

Kiểm thử một sản phẩm phần mềm là xây dựng một cách có chủ đích những tập dữ liệu và dãy thao tác nhằm đánh giá một số hoặc toàn bộ các tiêu chuẩn của sản phẩm phần mềm đó.

Thử nghiệm có hai mục đích: chỉ ra hệ thống phù hợp với đặc tả và phơi ra được các khuyết tật của hệ thống.

Kiểm thử là chạy thử chương trình với tập dữ liệu giả nên không thể khẳng định tính đúng của chương trình do bản chất quy nạp không hoàn toàn của nó. Trong nhiều trường hợp, việc kiểm thử thường được thực hiện từ những giai đoạn đầu của quá trình cài đặt sản phẩm. Các chương trình nên được kiểm chứng theo hai kỹ thuật: kiểm thử

và chứng minh. Và nếu có thể nên khẳng định tính đúng của chương trình thông qua văn bản chương trình.

Như vây, một chương trình tuyệt đối đúng phải được thực hiện thông qua: tính đúng đắn của thuật toán và tính tương đương của chương trình với thuật toán (được thể hiện ở chứng minh thông qua văn bản chương trình).

Việc kiểm thử chương trình chỉ là nhìn sự kiện đưa ra kết luận do vậy không thể khẳng định một chương trình tuyệt đối đúng bằng kiểm thử. Tuy vậy, bộ dữ liệu kiểm thử phải phủ kín mọi trường hợp cần đánh giá.

Thêm vào đó, trong quá trình kiểm thử, ta thường mắc phải các đặc trưng của nguyên lý chủ quan như sau:

- Bộ dữ liệu kiểm thử không thay đổi trong quá trình xây dựng phần mềm
- Chỉ kiểm thử các trường hợp chính thống, hợp lệ, không quan tâm đến các cận và các sự cố
- Cài đặt chức năng nào thì chỉ kiểm thử riêng chức năng đó, không chỉ kiểm thử tổng hợp chức năng vừa cài đặt với các chức năng đã cài đặt trước đó.
- Người kiểm thử đồng thời là người xây dựng phần mềm.

➢ **Các loại hình kiểm thử:**

- Kiểm thử lược đồ hệ thống: chỉ quan tâm đến các bảng chọn (menu) đánh giá tính hợp lý, khả năng chọn một mục, khả năng di chuyển qua mục khác, tính đủ, tính khoa học của các chức năng.
- Kiểm thử cận dưới.
- Kiểm thử cận trên: cho hệ thống thực hiện đến mức tối hạn.
- Kiểm thử qua sự cố: tạo ra các sự cố để kiểm thử phần mềm.

➢ **Nguyên tắc của kiểm thử:**

- Nguyên tắc khách quan: người kiểm thử không phải là tác giả của phần mềm đang kiểm thử.
- Nguyên tắc ngẫu nhiên: dữ liệu và chức năng được chọn, tuy có chủ đích nhưng không phải xuất hiện theo thứ tự nhất định.
- Nguyên tắc "người sử dụng kém": hệ thống được một người sử dụng có trình độ thấp (ở mức chấp nhận được) dùng thử. (Người này có thể gây các sự cố có thể không lường trước được của hệ thống)
- Nguyên tắc "kẻ phá hoại": hệ thống rơi vào tay có trình độ nghiệp vụ cao, chủ ý phá hoại. "Trình độ" ở đây thuộc lĩnh vực công nghệ thông tin hoặc lĩnh vực phần mềm đang hướng tới.

➢ **Kỹ thuật kiểm thử:**

- Kỹ thuật đối xứng: dựa vào tính đối xứng của các thao tác hoặc tập dữ liệu để xây dựng bộ dữ liệu Kiểm thử.
- Kỹ thuật đám đông.
- Kỹ thuật kiểm thử trên dữ liệu thật: cho hệ thống vận hành với các tập dữ liệu thật đã thu được từ trước để so sánh và đánh giá kết quả.
- Kỹ thuật kiểm thử trên thị trường thật: cho hệ thống vận hành trên thị trường thật (không chính thức) để so sánh với các hệ thống chính được dùng và đánh giá kết quả.
- Kỹ thuật đối sánh: cho thực hiện với một vài sản phẩm khác với cùng các chức năng giống nhau và trên cùng các tập dữ liệu rồi lập bảng so sánh các chức năng.

➢ **Quá trình kiểm thử:**

Trừ hệ thống nhỏ, nói chung không nên kiểm thử nguyên cả khối; quá trình kiểm thử có thể chia 5 giai đoạn:

- Thử đơn vị
- Thử mô đun
- Thử hệ con
- Thử hệ thống
- Thử nghiệm thu: còn gọi thử anpha.

Khi hệ thống được đem bán còn phép thử beta: phân phối hệ thống cho một số người dùng đồng ý dùng thử và báo cáo lại các vấn đề cho người phát triển hệ thống.

Thử hệ thống là rất đắt đỏ, đối với một vài hệ thời gian thực có các ràng buộc thời gian phức tạp thì việc thử có thể mất khoảng một nửa tổng chi phí phát triển.Vì thế mà phải lập kế hoạch thử và khống chế chi phí thử.

Cần chú ý là việc thử liên quan đến việc thiết lập ra các mẫu cho quá trình thử nhiều hơn là mô tả các phép thử.

4.3.2. Phân loại một số công cụ kiểm thử tự động

Vì kiểm thử phần mềm thường chiếm tới 40% tất cả các nổ lực dành cho một dự án xây dựng phần mềm, nên công cụ có thể làm giảm thời gian kiểm thử (không làm giảm tính kỹ lưỡng) sẽ rất có giá trị. Thừa nhận lợi ích tiềm năng này, các nhà nghiên cứu và người thực hành đã phát triển một số thế hệ các công cụ kiểm thử tự động:

- Bộ phân tích tĩnh: Các hệ thống phân tích chương trình này hỗ trợ cho "việc chứng minh" các lý lẽ tĩnh - những mệnh đề yếu kém về cấu trúc và định dạng của chương trình.
- Bộ kiểm toán mã: Những bộ lọc chuyên dụng này được dùng để kiểm tra chất lượng của phần mềm để đảm bảo rằng nó đáp ứng các chuẩn mã hoá tối thiểu.
- Bộ xử lý khẳng định: Những hệ thống tiền xử lý/hậu xử lý này được sử dụng để cho biết liệu những phát biểu do người lập trình nêu, được gọi là các khẳng định, về hành vi của chương trình có thực sự được đáp ứng trong việc thực hiện chương trình thực hay không.
- Bộ sinh tệp kiểm thử: Những bộ xử lý này sinh ra, và điền các giá trị đã xác định, vào các tệp đọc vào điển hình cho chương trình đang được kiểm thử.
- Bộ sinh dữ liệu kiểm thử: Những hệ thống phân tích tự động này hỗ trợ cho người dùng trong việc chọn dữ liệu kiểm thử làm cho chương trình hành xử theo một cách đặc biệt.
- Bộ kiểm chứng kiểm thử: Những công cụ này đo mức bao quát kiểm thử bên trong, thường được diễn tả dưới dạng có liên quan tới cấu trúc điều khiển của sự vật kiểm thử, và báo cáo về giá trị bao quát cho chuyên gia đảm bảo chất lượng.
- Bộ so sánh đầu ra: Công cụ này làm cho người ta có thể so sánh một tập đầu ra từ một chương trình này với một tập đầu ra khác (đã được lưu giữ trước) để xác định sự khác biệt giữa chúng.
- Hệ thống thực hiện ký hiệu: Công cụ này thực hiện việc kiểm thử chương trình bằng cách dùng đầu vào đại số, thay vì giá trị dữ liệu số. Phần mềm được kiểm thử được kiểm thử các lớp dữ liệu, thay vì chỉ là một trường hợp kiểm thử đặc biệt. Đầu ra là đại số và có thể được so sánh với kết quả trông đợi cũng được xác định dưới dạng đại số.
- Bộ mô phỏng môi trường: Công cụ này là một hệ thống dựa trên máy tính giúp người kiểm thử mô hình hoá môi trường bên ngoài của phần mềm thời gian thực và rồi mô phỏng các điều kiện vận hành thực tại một cách động.
- Bộ phân tích luồng dữ liệu: Công cụ này theo dõi dấu vết luồng dữ liệu đi qua hệ thống (tương tự về nhiều khía cạnh với bộ phân tích đường đi) và cố gắng tìm ra những tham khảo dữ liệu không xác định, đặt chỉ số sai và cáclỗi khác có liên quan tới dữ liệu.

Hiện nay việc dùng các công cụ tự động hoá cho kiểm thử phần mềm đang phát triển, và rất có thể là ứng dụng đó sẽ phát triển nhanh trong thập kỷ tới. Các công cụ

kiểm thử có thể sẽ gây ra những thay đổi lớn trong cách chúng ta kiểm thử phần mềm và do đó cải tiến độ tin cậy của các hệ thống dựa trên máy tính.

TỔNG KẾT CUỐI CHƯƠNG

Độ tin cậy phần mềm phản ánh giá trị của phần mềm được sử dụng trong thực tế. Vì vậy việc kiểm tra lỗi phần mềm là việc rất quan trọng đảm bảo phần mềm đáp ứng được yêu cầu người dùng. Trong quá trình lập trình, người lập trình nên sử dụng các phương pháp lập trình tránh lỗi, chịu lỗi. Ngoài ra, các đơn vị,mô đun, phần mềm cần phải qua các giai đoạn kiểm tra, kiểm thử trước khi đưa ra thị trường. Để thực hiện tốt việc kiểm tra, kiểm thử, cần tuân thủ theo đúng nguyên tắc và tránh những lỗi chủ quan thường gặp.

CÂU HỎI ÔN TẬP CHƯƠNG IV

4.1. Thế nào là độ tin cậy phần mềm?

4.2. Thế nào là kiểm tra phần mềm? các chiến lược kiểm tra phần mềm?

4.3. Nên sử dụng chiến lược kiểm tra nào là tốt nhất?

4.5. Thế nào là kiểm thử phần mềm? Liệt kê một số công cụ kiểm thử?

Chương 5: BẢO TRÌ PHẦN MỀM VÀ QUẢN LÝ THAY ĐỔI PHẦN MỀM

Bảo trì là một hoạt động phức tạp nhưng nó lại là vô cùng cần thiết trong chu trình sống của sản phẩm phần mềm để đảm bảo cho phần mềm phù hợp với người sử dụng. Trong chương này, chúng ta sẽ nghiên cứu về giai đoạn bảo trì, đặc điểm, các hình thức và việc quản lý bảo trì.

Trước khi đọc chương này, người học nên lựa chọn một phần mềm đã xây dựng và có bản phân tích yêu cầu đi kèm. Trong qua trình học tập, áp dụng các phương pháp đánh giá và bảo trì trong chương này vào phần mềm trên để hiểu rõ hơn nội dung của chương.

5.1. HOẠT ĐỘNG BẢO TRÌ PHẦN MỀM VÀ PHÂN LOẠI

Bảo trì phần mềm là hoạt động phức tạp. Chúng ta có thể chia hoạt động bảo trì ra làm bốn hoạt động như sau:

- **Bảo trì hiệu chỉnh**: Công việc bảo trì đầu tiên cần phải thực hiện. Nguyên nhân là việc kiểm tra chương trình không thể tránh được lỗi ẩn chứa bên trong một hệ phần mềm lớn. Trong khi sử dụng, các lỗi sẽ được báo về lại cho người phát triển. Bảo trì hiệu chỉnh chính là quá trình phân tích và hiệu chỉnh một hay nhiều lỗi của chương trình.
- **Bảo trì tiếp hợp**: Hoạt động thứ hai diễn ra bởi sự thay đổi thường xuyên của môi trường. Những thế hệ phần cứng, những hệ điều hành mới hay phiên bản mới của các hệ cũ đều đặn xuất hiện; thiết bị ngoại vi và các thành phần hệ thống khác liên tục được nâng cấp và thay đổi. Thời gian hữu dụng của một phần mềm ứng dụng mặt khác lại lâu dài sau thời điểm hệ thống được phát triển đầu tiên. Bảo trì tiếp hợp là hoạt động sửa đổi phần mềm để thích ứng được với những thay đổi của môi trường.
- **Bảo trì hoàn thiện**: Hoạt động thứ ba diễn ra khi một phần mềm đã được hoàn tất thành công. Hoạt động này chiếm hầu hết các công sức tiêu tốn cho việc bảo trì phần mềm. Lúc sử dụng, các yêu cầu về những khả năng mới, các thay đổi những chức năng đã có, và các mở rộng tổng quát được người dùng gửi đến. Để thỏa mãn những yêu cầu phát triển của người sử dụng, ta tiến hành bảo trì hoàn thiện.
- **Bảo trì phòng ngừa**: Bảo trì phòng ngừa là hoạt động bảo trì diễn ra khi phần mềm được thay đổi để cải thiện tính năng bảo trì hay độ tin cậy trong tương lai hoặc để cung cấp một nền tảng tốt hơn cho những mở rộng sau này. Bảo trì phòng ngừa, hoạt động này vẫn còn nhiều xa lạ hiện nay.

Phần mềm khác với phần cứng, không thể tận dụng được, vì vậy hoạt động bảo trì

phần cứng chủ yếu - thay thế các bộ phận bị hỏng hóc hay gãy vỡ - không được kể đến.

Trong thực tế của hoạt động bảo trì, các nhiệm vụ được làm như một phần của bảo trì tiếp hợp và bảo trì hoàn thiện cũng giống như các nhiệm vụ cần làm trong giai đoạn phát triển của một chu trình phần mềm. Để tiếp hợp hay hoàn thiện, chúng ta đều phải xác định yêu cầu, thiết kế lại, tạo mã và kiểm tra phần mềm có được.

5.2. ĐẶC ĐIỂM CỦA BẢO TRÌ PHẦN MỀM

Bảo trì phần mềm cho tới gần đây vẫn còn là một giai đoạn bị coi nhẹ của chu trình phần mềm. Có rất ít các số liệu nghiên cứu tập trung vào đề tài này, và rất ít các phương pháp kỹ thuật được đưa ra. Để hiểu được điểm bản chất của bảo trì, chúng ta sẽ xem xét các vấn đề từ ba góc độ khác nhau:

- Các hoạt động cần thiết để hoàn thành giai đoạn bảo trì và tính toàn vẹn của một cách tiếp cận theo công nghệ phần mềm đối với hiệu quả của những hoạt động đó.
- Chi phí kèm theo giai đoạn bảo trì.
- Các vấn đề thường gặp phải khi tiến hành bảo trì phần mềm.

5.2.1. Bảo trì có cấu trúc đối với bảo trì không cấu trúc

Nếu thành phần có được duy nhất của một cấu hình phần mềm là mã nguồn, hoạt động bảo trì bắt đầu với việc đánh giá chi tiết mã nguồn thường là khá phức tạp với những tài liệu nghèo nàn bên trong. Những đặc điểm tế nhị như cấu trúc phần mềm, các cấu trúc dữ liệu toàn cục, giao diên hệ thống, hoạt động và các ràng buộc thiết kế thường rất khó sáng tỏ và hay bị hiểu lầm. Các thay đổi lặt vặt thường xuyên làm cho các mã rất khó đánh giá. Các kiểm tra hồi quy (lặp lại các kiểm tra trước kia để đảm bảo rằng những thay đổi không tạo ra lỗi trong phần mềm đã hoạt động) là không thể thực hiện được bởi không hề có các bản lưu về các kiểm tra đó. Chúng ta đang tiến hành phép bảo trì không cấu trúc và đang phải trả giá (khi lãng phí công sức và gây tâm trạng thất vọng). Sự trả giá này luôn đi kèm với các phần mềm đã không được phát triển theo những phương pháp đúng đắn.

Nếu có một cấu hình phần mềm hoàn thiện, nhiệm vụ bảo trì bắt đầu bằng việc đánh giá các tài liệu thiết kế. Sau đó là xác định các đặc điểm thuộc cấu trúc quan trọng, các đặc điểm hoạt động và giao diện. Tính toàn vẹn của những sửa đổi và hiệu chỉnh cần thiết sẽ được đánh giá và một kế hoạch sửa đổi sẽ được thiết lập. Thiết kế được thay đổi (sử dụng những kỹ thuật phù hợp với những điều đã bàn luận ở các

chương trước) rồi nhận xét đánh giá. Mã nguồn được phát triển, sau đó tiến hành các kiểm tra hồi quy sử dụng thông tin chứa trong phần "đặc tả kiểm tra" và phần mềm lại được phát hành.

Các mô tả trên đây là phép bảo trì cấu trúc và được tiến hành như là kết quả của những ứng dụng trước đây trong khoa học về công nghệ phần mềm. Mặc dù sự có mặt của một cấu hình phần mềm không đảm bảo được các vấn đề bảo trì nảy sinh, nhưng khối lượng công việc đã được giảm bớt và chất lượng chung của những thay đổi và hiệu chỉnh đã được cải thiện.

5.2.2. Chi phí bảo trì

Chi phí cho việc bảo trì là rõ ràng nhất. Tuy nhiên những chi phí khác khó thấy hơn có thể gây ra mối quan tâm lớn hơn:

- Một chi phí khó xác định của việc bảo trì phần mềm là các cơ hội phát triển bị bỏ qua vì các tài nguyên có sẵn đều dành cho nhiệm vụ bảo trì.
- Sự không hài lòng của người dùng khi các yêu cầu có vẻ hợp lý cho việc sửa chữa hay sửa đổi không được chú ý một cách hợp lý.
- Việc suy giảm chất lượng nói chung do lỗi tạo ra bởi sự thay đổi trong các phần mềm được bảo trì.
- Một yêu cầu bất chợt làm ngắt quãng quá trình phát triển của một nhân viên buộc anh ta tiến hành công việc bảo trì.

Chi phí cuối cùng cho việc bảo trì là sự giảm sút kinh khủng về năng suất lao động (được đo theo số dòng lệnh -LOC- của một người trong một tháng hay số tiền chi phí cho dòng lệnh). Sự giảm sút này xuất hiện khi tiến hành bảo trì đối với các phần mềm cũ. Người ta đã ghi nhận sự giảm sút năng suất lao động theo tỷ lệ 40:1, có nghĩa là chi phí cho việc phát triển trị giá $25.00 trên một dòng lệnh sẽ có thể trị giá tới $1000.00 cho việc bảo trì mỗi dòng lệnh.

Công sức cho việc bảo trì có thể được phân chia thành các thao tác làm việc: phân tích, ước lượng, thay đổi thiết kế, và sửa chữa chương trình nguồn và các thao tác lặp lại: việc cố gắng hiểu chương trình nguồn làm gì, cố gắng sáng tỏ cấu trúc dữ liệu, các thuộc tính giao diện, giới hạn của việc thực hiện,... Biểu thức dưới đây cung cấp một mô hình cho công việc bảo trì:

$$M = p + K^* \exp(c-d),$$

M = toàn bộ các công việc cho việc bảo trì;

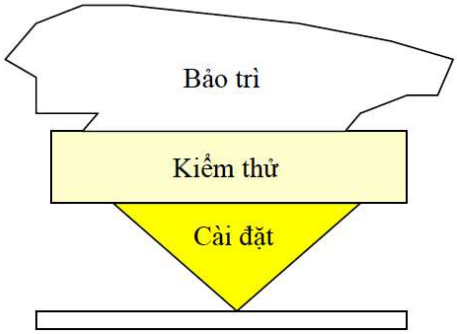
p = công việc làm (như miêu tả ở trên);

K = hằng số kinh nghiệm;

c = đánh giá mức độ phức tạp được tính cho việc thiếu thiết kế về cấu trúc và dữ liệu;

d = đánh giá mức độ hiểu biết về phần mềm.

Mô hình trên đây cho thấy công việc và giá thành có thể tăng lên theo cấp số mũ nếu một phương pháp phát triển phần mềm kém cỏi được sử dụng, tức là thiếu sót của công nghệ phần mềm, và nếu một người hay một nhóm dùng các phương pháp không có giá trị để bảo trì phần mềm. Chi phí cho bảo trì khi phần mềm được phát triển không đúng phương pháp được minh hoạ ở hình sau:



Hình 5-1: Chi phí xây dựng phần mềm

5.2.3. Một số vấn đề khác

Hầu hết các vấn đề liên quan tới việc bảo trì phần mềm đều liên quan tới các sai lệch trong cách xây dựng và phát triển phần mềm. Sự thiếu sót trong việc điều khiển và tổ chức trong hai giai đoạn đầu tiên của một chu trình phần mềm gần như luôn luôn tạo ra các vấn đề giai đoạn cuối.

Nhiều vấn đề kinh điển có thể liên quan tới việc bảo trì phần mềm được trình bày dưới đây:

- Rất khó hoặc không thể theo dõi sự tiến hóa của phần mềm qua các phiên bản. Các thay đổi không được tư liệu hóa.
- Khó theo dõi được các quá trình xử lý được tạo bởi các phần mềm.
- Thường xuyên gặp rất nhiều khó khăn trong việc tìm hiểu chương trình của người khác viết. Những khó khăn này tăng lên khi số thành phần các cấu hình của phần mềm giảm đi. Nếu chỉ có các chương trình nguồn không có tài liệu hướng dẫn thì không nên tìm hiểu phần mềm đó.
- Những người viết phần mềm thường không có mặt để giải thích. Chúng ta không thể trông cậy vào những giải thích cá nhân của các nhà phát triển phần

mềm khi việc bảo trì được yêu cầu.

- Các tài liệu chính xác không có hay thiếu trầm trọng. Phải thừa nhận rằng phải có tài liệu về phần mềm là bước đầu tiên, nhưng tài liệu phải hiểu được và phù hợp với chương trình lại là chuyện khác.
- Phần lớn các phần mềm không thiết kế để thay đổi, trừ phi sử dụng phương pháp thiết kế dùng các khái niệm về phân tách chương trình thành các mô đune độc lập. Việc thay đổi phần mềm sẽ rất khó khăn và dẫn đến xu hướng sai.
- Việc bảo trì phần mềm không được coi là một công việc dễ dàng mà công việc bảo trì phần mềm luôn liên quan tới các sai lệch ở mức độ cao.

5.3. CÔNG VIỆC BẢO TRÌ PHẦN MỀM VÀ MỘT SỐ HIỆU ỨNG LỀ

5.3.1. Khả năng bảo trì

Khả năng bảo trì của phần mềm có thể coi như các khả năng hiểu, hiệu chỉnh, tiếp hợp hoặc có thể thêm vào khả năng phát triển. Khả năng bảo trì là chìa khóa để dẫn đến các phương pháp thiết kế xây dựng phần mềm.

a. Yếu tố kiểm soát

Khả năng bảo trì cơ bản bao gồm nhiều yếu tố. Sự thiếu cẩn trọng trong việc thiết kế, viết chương trình nguồn, kiểm tra có ảnh hưởng tiêu cực đến việc bảo trì có kết quả một phần mềm. Cấu hình yếu kém cũng có các tác động tương tự, thậm chí cả khi từng bước của kỹ thuật xây dựng phần mềm được xem xét một cách cẩn thận.

Thêm vào đó nhiều yếu tố khác liên quan tới phương pháp phát triển phần mềm, như:

- Chất lượng hiệu quả của đội ngũ phần mềm.
- Cấu trúc của hệ thống dễ hiểu.
- Dễ dàng kiểm soát hệ thống.
- Dùng các ngôn ngữ lập trình chuẩn.
- Dùng các hệ điều hành chuẩn.
- Dùng các cấu trúc chuẩn tài liệu.
- Dùng được các tài liệu kiểm tra.
- Phương tiện gỡ rối đi kèm.
- Dùng được các máy tính tốt để thực hiện việc bảo trì.

Các yếu tố được đưa ra ở trên đã phản ánh những đặc điểm về phần cứng cũng như chương trình nguồn được dùng trong việc phát triển phần mềm. Những yếu tố khác chỉ ra sự cần thiết để có được phương pháp chuẩn, chương trình nguồn chuẩn. Có thể, yếu tố quan trọng nhất tác động tới khả năng bảo trì là kế hoạch cho khả năng bảo trì. Nếu

coi phần mềm như là một hệ thống các thành phần sẽ phải chịu những thay đổi không tránh được, thì cơ hội tạo những phần mềm có khả năng bảo trì sẽ tăng thực sự.

b. Đánh giá định lượng

Khả năng bảo trì, như chất lượng hay độ tin cậy là hết sức khó xác định. Tuy nhiên chúng ta có thể đánh giá khả năng bảo trì gián tiếp bằng việc xem xét các thuộc tính của các công việc bảo trì có thể đánh giá được:

- Thời gian nhận biết vấn đề.
- Thời gian trễ do các công việc hành chính.
- Thời gian lựa chọn công cụ bảo trì.
- Thời gian phân tích vấn đề.
- Thời gian xác định sự thay đổi.
- Thời gian hiệu chỉnh (hay sửa đổi) thực sự.
- Thời gian chạy thử cục bộ.
- Thời gian chạy thử tổng thể.
- Thời gian tổng kết bảo trì.
- Tổng thời gian của công việc bảo trì.

Mỗi đánh giá trên thực tế là các dữ liệu đó có thể cung cấp cho người quản lý cùng với chỉ số về hiệu quả của công việc.

5.3.2. Các công việc bảo trì

Những nhiệm vụ liên quan tới việc bảo trì gồm: các tổ chức bảo trì được thiết lập; các thủ tục ghi nhận và đánh giá được miêu tả; và một loạt thứ tự chuẩn của các bước cho mỗi yêu cầu bảo trì phải được định nghĩa. Thêm vào đó, một thủ tục lưu trữ các hồ sơ cho các hoạt động bảo trì được thiết lập và bản tổng kết những tiêu chuẩn đánh giá được vạch rõ.

a. Cơ cấu bảo trì

Mặc dù những tổ chức bảo trì chuẩn không cần được thiết lập, nhưng sự ủy thác trách nhiệm rất là cần thiết kể cả cho các tổ chức phát triển phần mềm nhỏ. Những yêu cầu bảo trì được chuyển qua cho người kiểm soát công việc bảo trì và từ đây chuyển tiếp yêu cầu tới người quản lý hệ thống để đánh giá. Người quản lý hệ thống là thành viên của nhóm nhân viên kỹ thuật. Những nhân viên này có trách nhiệm về một phần nhỏ của chương trình sản phẩm. Khi một yêu cầu được đánh giá, người được ủy quyền quản lý việc thay đổi phải quyết định hành động nào được thực hiện tiếp.

Cơ cấu được miêu tả ở trên phục vụ cho việc thiết lập phạm vi trách nhiệm đối với

công việc bảo trì. Người kiểm soát và người ủy quyền quản lý việc thay đổi có thể là một người hay là một nhóm quản lý và chuyên gia kỹ thuật cao cấp.

b. Báo cáo

Tất cả các yêu cầu về việc bảo trì phần mềm cần được trình bày theo một tiêu chuẩn. Người phát triển phần mềm thường cung cấp một đơn yêu cầu bảo trì còn được gọi là báo cáo các lỗi phần mềm. Báo cáo này được người sử dụng điền vào khi yêu cầu công việc bảo trì. Nếu xuất hiện một lỗi, bản mô tả đầy đủ tình huống dẫn đến lỗi bao gồm dữ liệu, đoạn chương trình và các yêu cầu khác phải được điền đầy đủ vào bản báo cáo. Nếu yêu cầu bảo trì là bảo trì tiếp hợp hay bảo trì hoàn thiện thì một yêu cầu chi tiết sẽ được thảo ra.

Đơn yêu cầu bảo trì sẽ được người kiểm soát bảo trì và người quản lý hệ thống xem xét như phần trước đã nêu. Đơn yêu cầu bảo trì được thiết lập từ bên ngoài và được coi như một cơ sở để đề ra kế hoạch của công việc bảo trì. Ngoài ra trong nội bộ của cơ quan phần mềm một báo cáo thay đổi phần mềm cũng được tạo ra. Nó chỉ ra các công sức đòi hỏi để thỏa mãn một đơn yêu cầu bảo trì, trạng thái ban đầu của yêu cầu sửa đổi, mức ưu tiên của yêu cầu, các dữ liệu cần cho việc sửa đổi,...

c. Lưu giữ các hồ sơ

Thường chúng ta không có đầy đủ các hồ sơ cho tất cả các giai đoạn trong chu kỳ sống của một phần mềm. Thêm nữa không có các hồ sơ về việc bảo trì phần mềm. Do đó chúng ta thường khó có thể tiến hành các công việc bảo trì có hiệu quả, không có khả năng xác định tính chất của các chương trình và khó xác định được giá bảo trì phần mềm.

Các thông tin cần phải lưu giữ trong hồ sơ bảo trì thường:

- Dấu hiệu nhận biết chương trình.
- Số lượng các câu lệnh trong chương trình nguồn.
- Số lượng các lệnh mã máy.
- Ngôn ngữ lập trình được sử dụng.
- Ngày cài đặt chương trình.
- Số các chương trình chạy từ khi cài đặt.
- Số các lỗi xử lý xảy ra.
- Mức và dấu hiệu thay đổi chương trình.
- Số các câu lệnh được thêm vào chương trình nguồn khi chương trình thay đổi.
- Số các câu lệnh được xóa khỏi chương trình nguồn khi chương trình thay đổi.

- Số giờ mỗi người sử dụng cho mỗi lần sửa đổi.
- Ngày thay đổi chương trình.
- Dấu hiệu của kỹ sư phần mềm.
- Dấu hiệu của đơn yêu cầu bảo trì.
- Kiểu bảo trì.
- Ngày bắt đầu và kết thúc bảo trì.
- Tổng số giờ của mỗi người dùng cho việc bảo trì.

d. Xác định giá bảo trì

Việc xác định giá trị bảo trì thường phức tạp do thiếu thông tin. Nếu tiến hành việc lưu giữ các hồ sơ có thể tiến hành một số cách đánh giá về việc thực hiện bảo trì. Theo các chuyên gia thì đánh giá về việc thực hiện bảo trì dựa vào:

- Số lượng trung bình các lỗi xử lý cho một lần chạy chương trình.
- Tổng số giờ của mỗi người dùng cho mỗi loại bảo trì.
- Số lượng trung bình các thay đổi theo chương trình, theo ngôn ngữ lập trình, theo kiểu bảo trì.
- Số giờ trung bình của mỗi người cho một dòng lệnh được thêm vào và xóa đi.
- Số giờ trung bình của mỗi người cho một ngôn ngữ lập trình.
- Thời gian trung bình cho việc bảo trì một đơn yêu cầu bảo trì.
- Tỷ lệ phần trăm của mỗi kiểu bảo trì.

5.3.3. Một số hiệu ứng lề của công việc bảo trì

Sửa đổi phần mềm là công việc nguy hiểm, ta thường gặp ba loại chính của hiệu ứng lề.

a. Hiệu ứng lề của việc thay đổi mã nguồn

Một thay đổi đơn giản tới một câu lệnh đơn cũng có thể đem lại một hậu quả nặng nề. Mặc dù không phải các ảnh hưởng đều là tiêu cực, nhưng việc sửa lỗi luôn dẫn đến các vấn đề phức tạp.

Mặc dù tất cả các thay đổi mã lệnh chương trình đều có thể tạo ra lỗi, nhưng tập hợp các thay đổi sau có thể gây ra nhiều lỗi hơn.

- Một chương trình con bị xóa hay thay đổi.
- Một dòng nhãn bị xóa hay thay đổi.
- Một biến bị xóa hay thay đổi.
- Các thay đổi để tăng khả năng thực hiện.

- Việc mở và đóng file bị thay đổi.
- Các phép toán logic bị thay đổi.
- Việc thay đổi thiết kế chuyển thành các thay đổi lớn về chương trình.
- Các thay đổi ảnh hưởng đến việc chạy thử các trường hợp biên.

b. Hiệu ứng lề của việc thay đổi dữ liệu

Trong quá trình bảo trì, việc sửa đổi thường được tiến hành đối với các phần tử riêng rẽ của cấu trúc dữ liệu. Khi dữ liệu thay đổi, việc thiết kế phần mềm sẽ không còn phù hợp với dữ liệu và lỗi có khả năng xảy ra.

Hiệu ứng lề của dữ liệu xảy ra như là kết quả của việc thay đổi cấu trúc dữ liệu. Các thay đổi dữ liệu sau đây thường gây ra lỗi:

- Định nghĩa lại các hằng số cục bộ và hằng số địa phương.
- Định nghĩa lại cấu trúc bản ghi hay cấu trúc file.
- Tăng hoặc giảm kích thước một mảng.
- Thay đổi dữ liệu tổng thể.
- Định nghĩa lại các cờ điều khiển và các con trỏ.
- Xếp lại các tham số vào ra hay tham số của chương trình con.

Hiệu ứng lề dữ liệu có thể được hạn chế bằng tài liệu thiết kế mô tả cấu trúc dữ liệu và cung cấp một lời chỉ dẫn tham khảo đến từng phần từ dữ liệu, các bản ghi, các file và các cấu trúc khác của các mô đun phần mềm.

c. Hiệu ứng lề của việc thay đổi tài liệu

Việc bảo trì thường tập trung vào cấu hình phần mềm và không tập trung riêng vào việc sửa đổi mã. Sự ảnh hưởng của tài liệu xảy ra khi thay đổi chương trình nguồn mà không thay đổi tài liệu thiết kế và tài liệu hướng dẫn sử dụng.

Bất cứ lúc nào có thay đổi về luồng dữ liệu, cấu trúc phần mềm, các thủ tục hay bất cứ cái gì có liên quan, tài liệu kỹ thuật phải được cập nhật. Tài liệu về thiết kế phản ánh không đúng trạng thái hiện tại của phần mềm có lẽ còn tồi tệ hơn không có tài liệu. Hiệu ứng lề xảy ra trong các lần bảo trì sau đó, khi việc nghiên cứu không kỹ các tài liệu kỹ thuật, dẫn tới sự đánh giá sai về các đặc tính của phần mềm. Đối với người sử dụng, phần mềm tốt chỉ khi có tài liệu hướng dẫn sử dụng chúng.

Các hiệu ứng lề trong tài liệu có thể được giảm về căn bản nếu toàn bộ cấu hình được xem xét trước khi phát hành phiên bản phần mềm tiếp sau. Thực tế một vài yêu cầu bảo hành có thể đòi hỏi không được thay đổi thiết kế của phần mềm hoặc mã chương trình, mà chỉ cần chỉ ra sự thiếu rõ ràng trong tài liệu của người sử dụng. Trong những trường hợp như vậy nổ lực bảo trì tập trung vào tài liệu.

5.4. MỘT SỐ HÌNH THỨC BẢO TRÌ PHẦN MỀM

5.4.1. Bảo trì mã chương trình xa lạ

Các chương trình được gọi là mã chương trình xa lạ nếu:

- Không một thành viên này trong phòng kỹ thuật tiếp tục phát triển chương trình đó nữa.
- Không tiếp tục áp dụng lý thuyết phát triển, và vì vậy tồn tại vấn đề thiết kế nghéo nàn và ít tài liệu (theo tiêu chuẩn ngày nay).
- Cấu trúc khối chưa được thiết kế theo tiêu chuẩn, và các khái niệm về thiết kế có cấu trúc chưa được áp dụng.

Một số đề nghị hữu dụng cho người quản trị hệ thống phải bảo trì các chương trình xa lạ như sau:

- Nghiên cứu chương trình trước khi bạn bị đặt vào "chế độ khẩn cấp". Cố gắng thu nhận được càng nhiều thông tin cơ sở càng tốt...
- Cố gắng làm quen với toàn bộ các luồng điều khiển của chương trình; trước hết bỏ qua các chi tiết về mã chương trình. Sẽ rất có ích khi vẽ cho riêng bạn sơ đồ cấu trúc và sơ đồ luồng hoạt động mức cao, nếu chưa có bản nào tồn tại.
- Đánh giá tình hình hợp lý của tài liệu hiện có; bổ sung thêm các lời chú thích của bản thân bạn vào chương trình nguồn nếu bạn thấy cần thiết.
- Sử dụng tốt các danh sách chỉ dẫn tham khảo, các bảng ký hiệu, và các trợ giúp khác thông thường được chương trình dịch và/hoặc assembler cung cấp.
- Thực hiện sửa đổi chương trình với sự chú ý lớn nhất. Lưu ý tới kiểu và dạng của chương trình tại tất cả các chỗ có thể. Đánh dấu trên chương trình những lệnh bạn đã sửa.
- Đừng loại bỏ chương trình trừ khi bạn chắc chắn nó không được sử dụng nữa.
- Đừng cố sử dụng chung các biến tạm thời và vùng nhớ làm việc mà đã có sẵn trong chương trình. Thêm các biến của riêng bạn để tránh các rắc rối.
- Giữ các bản ghi chép chi tiết (vê hoạt động bảo trì và các kết quả).
- Tránh sự nóng vội vô lý ném chương trình cũ đi và viết lại nó.
- Thực hiện các kiểm tra lỗi.

5.4.2. Công nghệ phản hồi và công nghệ tái sử dụng

Công nghệ phản hồi (Reverse engineering) đối với phần mềm là đơn giản. Tổ chức ngược đối với phần mềm là quá trình phân tích chương trình trong cố gắng biểu diễn lại các chương trình ở mức độ trừu tượng cao hơn mã nguồn. Nó là quá trình khám phá thiết kế. Các công cụ của công nghệ phản hồi lấy ra dữ liệu, kiến trúc, các thông tin thiết kế thủ tục từ chương trình đã tồn tại.

Công nghệ tái sử dụng (Re-engineering) không đơn thuần phát hiện các thông tin thiết kế mà còn dùng các thông tin này để biến đổi hoặc tổ chức lại hệ thống đã tồn tại với mục đích cải thiện chất lượng. Trong nhiều trường hợp, phần mềm ứng dụng lại các chức năng của hệ thống tồn tại. Nhưng trong cùng thời điểm, nhà phát triển phần mềm cũng phải thêm các chức năng mới và/hoặc cải thiện các xử lý.

5.4.3. Bảo trì phòng ngừa

Bảo trì phòng ngừa các phần mềm máy tính là một vấn đề khá mới và còn đang được tranh cãi. Thay vì đợi cho đến khi nhận được yêu cầu bảo trì, các tổ chức phát triển hay bảo trì chọn một chương trình mà:

- Sẽ được sử dụng trong một số năm định trước
- Hiện đang được sử dụng tốt
- Dễ bị thay đổi hoặc nâng cấp trong tương lai gần

Thoạt đầu ý kiến đề nghị phát triển lại một chương trình lớn khi một phiên bản đang làm việc đã tồn tại ta thấy dường như quá phung phí. Nhưng chúng ta hãy xem xét các điểm sau:

- Chi phí để bảo trì một dòng mã lệnh có thể lớn hơn 20 tới 40 lần chi phí cho phát triển ban đầu dòng lệnh đó.
- Thiết kế lại cấu trúc của phần mềm, với sự sử dụng các khái niệm thiết kế hiện tại có thể làm cho việc bảo hành tương lai dễ dàng hơn.
- Bởi vì khuôn mẫu phần mềm đã tồn tại, năng suất phát triển chương trình chắc sẽ cao hơn mức trung bình nhiều.
- Người sử dụng bây giờ đã làm quen với phần mềm. Vì vậy, các đòi hỏi mới và hướng thay đổi có thể tìm ra dễ dàng hơn nhiều.
- Các công cụ CASE dành cho reverse engineering và re-engineering sẽ thực hiện tự động một số phần của công việc.
- Một cấu hình phần mềm sẽ tồn tại trên sự hoàn thành của bảo trì phòng ngừa.

Khi một tổ chức phát triển phần mềm bán phần mềm như là một sản phẩm, bảo trì phòng ngừa được xem như "phiên bản mới" của chương trình. Nhiều hãng phát triển phần mềm lớn có thể có từ 500 tới 2000 sản phẩm chương trình trong phạm vi trách nhiệm của họ. Các chương trình như vậy có thể được xếp theo thứ tự ưu tiên và xem xét lại như các ứng cử cho bảo trì phòng ngừa.

5.4.4. Chiến lược phần mềm thành phần

Như đặc tính cổ điển của bảo hành phần cứng là tháo bỏ phần hỏng và thay thế

bằng phụ tùng mới. Một khái niệm được gọi là nguyên mẫu phần mềm có thể dẫn tới việc phát triển các phụ tùng cho các chương trình.

Nguyên mẫu phần mềm là một quá trình mô hình hóa yêu cầu ngươi dùng trong một hay nhiều mức chi tiết, bao gồm cả các mô hình làm việc. Các tài nguyên của dự án được xếp đặt làm sao để sản xuất các phiên bản phần mềm được mô tả theo yêu cầu phải nhỏ đi. Phiên bản nguyên mẫu làm cho người dùng, người thiết kế và quản trị... có thể xem lại được phần mềm. Quá trình đó sẽ tiếp tục khi được đề nghị, với phiên bản đang chạy chuẩn bị sẵn sàng phát hành sau vài lần làm lại.

Nếu các mức nguyên mẫu khác nhau được phát triển, nó có thể có một bộ các phụ tùng phần mềm có thể được sử dụng khi nhận được yêu cầu bảo trì hiệu chỉnh. Ví dụ một mô đun phân tích có thể được thiết kế và thực hiện theo hai cách khác nhau nhưng có cùng giao diện bên ngoài. Một phiên bản của mô đun có được sử dụng trong phần mềm làm việc. Nếu mô đun đó hỏng, một phụ tùng có thể được lắp thay ngay.

5.5. QUẢN LÝ THAY ĐỔI PHẦN MỀM

Các ứng dụng thường xuyên phải thiết kế lại do sự phân công của một nhóm quản lý mới, dự án vượt quá ngân sách, ứng dụng chậm và có nhiều lỗi, và sự thiếu tin tưởng của chủ sử dụng về việc các kỹ sư phần mềm hiểu rõ các yêu cầu của mình,...

Các thay đổi có thể là các yêu cầu, thiết kế, chương trình, giao diện, phần cứng hoặc phần mềm phải mua. Phần lớn các thay đổi bắt nguồn từ bên trong tổ chức phát triển ứng dụng, nhưng cũng có thể được kích hoạt từ các tác nhân bên ngoài, ví dụ như thay đổi về luật.

Việc quản lý thay đổi ứng dụng giúp cho nhóm triển khai bỏ qua những ý thích chợt nảy ra của người sử dụng trong khi vẫn cho phép thực hiện các yêu cầu hợp lý.

5.5.1. Các thủ tục quản lý thay đổi

Quản lý điều khiển thay đổi có hiệu lực từ khi sản phẩm đầu tiên được chấp nhận là hoàn thiện cho đến khi dự án kết thúc. Trước tiên, các sản phẩm công việc cơ sở được tạo lập để đưa vào quản lý. Một sản phẩm công việc cơ sở là một sản phẩm được coi là hoàn thiện và là cơ sở cho các công việc hiện tại khác của nhóm triển khai dự án. Ví dụ như, một tài liệu cơ sở là bản quy định yêu cầu chức năng sau khi nó đã được chấp nhận bởi người sử dụng.

Đây là ví dụ quá trình của các thao tác yêu cầu thay đổi của một đặc tả chức năng:

- Tạo yêu cầu mở.

- Khai báo file tác động
- Phê chuẩn file về thời gian và chi phí do chủ, người sử dụng ký.
- Hoàn thiện danh sách và kiểm soát về thay đổi của người điều hành dự án.
- File tài liệu liên quan đến thay đổi. Nếu tài liệu hoặc chương trình bị thay đổi, thì xác định ngày và các mục cập nhật đã hoàn thiện. Nếu các thủ tục hoặc thử nghiệm bị thay đổi, xác định các ngày mà việc sửa đổi xảy ra.
- Mẫu yêu cầu đóng file được chủ/người sử dụng thông qua.
- Tóm tắt các ngày tháng, quá trình và chi phí.

Trước tiên, tài liệu cơ sở được giữ nguyên, sau đó thêm vào các yêu cầu thay đổi. Khi quy định chức năng được cập nhật để điều tiết thay đổi, nó được đóng băng lại và công việc lại tiếp tục. Ba yêu cầu trước có thể được thêm vào ứng dụng nếu chúng không làm thay đổi ứng dụng nhiều. Chúng cũng có thể bị bỏ qua cho đên sau khi ứng dụng đã được thực hiện.

Các thay đổi có thể phân loại theo một số cách:

- Thứ nhất, chúng được phân theo kiểu như loại bỏ lỗi, cải tiến thực hiện hoặc thay đổi chức năng.
- Thứ hai, thay đổi phân loại thành yêu cầu và lựa chọn
- Thứ ba, phân theo độ ưu tiên như khẩn cấp, lệnh với một ngày kết thúc yêu cầu, lệnh với ngày bắt đầu yêu cầu hoặc ưu tiên thấp.
- Thông thường, kiểu loại bỏ lỗi là khẩn cấp theo yêu cầu, trong khi thay đổi chức năng là bảo dưỡng lệnh theo yêu cầu, và cải tiến thực hiện là lựa chọn và có thể không có ưu tiên.

Việc biết được loại yêu cầu thay đổi quyết định xem liệu nó có cần phải chịu điều khiển thay đổi hay không. Các thay đổi khẩn cấp thường phá vỡ thủ tục điều khiển thay đổi do các công việc thực hiện tuần tự nhưng chúng lại được tài liệu hoá sau khi thay đổi đã kết thúc. Tất cả các loại thay đổi khác đều phải tuân theo các điều khiển thay đổi. Ví dụ như thay đổi về yêu cầu chức năng có thể xảy ra bất cứ lúc nào, nhưng khi quy định yêu cầu chức năng được thông qua thì nó đóng băng cho đến khi ứng dụng hoạt động. Các thay đổi phải chịu sự điều khiển thay đổi: chúng được thêm vào danh sách yêu cầu thay đổi để xem xét trong tương lai trừ khi đó là một thiết kế khẩn cấp.

Một thủ tục điều khiển thay đổi yêu cầu người sử dụng phải đệ trình một lời yêu cầu thay đổi chính thức cho người điều hành dự án:

- Người sử dụng gửi cho người điều hành dự án và người chủ một mẫu yêu cầu

thay đổi.

- Người điều hành dự án và kỹ sư phần mềm triển khai một khai báo tự động. Vào lúc đó, danh sách kiểm soát của người điều hành dự án được dùng để xác định tất cả các hoạt động và thay đổi công việc có liên quan tới yêu cầu.
- Yêu cầu thay đổi được thảo luận với chủ sử dụng để vạch ra các thay đổi về ưu tiên, tiến trình và chi phí.
- Thoả thuận được chính thức hoá và chủ sử dụng thông qua thay đổi về tiến trình và chi phí.
- Sử dụng khai báo tác động, ứng dụng và tất cả các tài liệu có liên quan được thay đổi.
- Thực hiện thay đổi: khi các nhiệm vụ hoàn thành, xoá nhiệm vụ trong danh sách kiểm soát của người điều hành dự án.
- Chủ sử dụng thông qua việc đóng yêu cầu và yêu cầu được đóng.
- Người điều hành dự án và kỹ sư phần mềm định nghĩa các tác động tiến trình và chi phí của thay đổi. Sau đó các thay đổi được bàn bạc với người sử dụng. Dựa trên thương lượng với người sử dụng, thay đổi được gán một ưu tiên hoạt động, và chi phí và tiến trình được thay đổi.

Yêu cầu, ngày dự định hoạt động, thay đổi tiến trình và tăng chi phí được thêm vào một file quá trình dự án. Các thay đổi có thể được quản lý bởi một nhân viên điều khiển thay đổi, là một người có nhiệm vụ bảo dưỡng quá trình dự án và các bản ghi điều khiển thay đổi, và hàng tháng in ra một bản báo cáo điều khiển thay đổi. Một file điều khiển thay đổi chứa tất cả các yêu cầu, thư từ và tài liệu về các thay đổi. Một yêu cầu thay đổi mở có thể được tạo ra khi yêu cầu được đưa ra và một số lượng thay đổi được gán. Yêu cầu thay đổi mở nằm trong file cho đến khi yêu cầu được hoàn thành, đóng và được báo cáo.

Khi thay đổi được thực hiện, các mục có ảnh hưởng được cập nhật, bao gồm tư liệu tương ứng, mã,... Một danh sách kiểm soát của người điều hành dự án được dùng để loại bỏ các hoạt động đã được yêu cầu. Tài liệu mới được nhân viên điều khiển thay đổi sắp xếp và phân phối nó cho những người có quan tâm. Ngày hoàn thành thay đổi được đưa vào file điều khiển thay đổi. Thay đổi được xác định khi được đóng trong báo cáo tình trạng tới và yêu cầu mở được chuyển từ file điều khiển thay đổi sang.

Dựa trên tổ chức này, người điều hành hệ thống có thể theo dõi các yêu cầu thay đổi của dự án để nhận biết sự thành công trong nhóm các yêu cầu. Chi phí thay đổi chung của một năm thường được sử dụng như là một chỉ tiêu để chỉ ra xem ứng dụng đang có triển vọng hay cần vứt bỏ hay cần công nghệ hoá lại. Trong những trường hợp

này, cả chi phí và số lượng các yêu cầu thay đổi đều được theo dõi thông qua quá trình điều khiển thay đổi. Các báo cáo tổng kết bởi dự án thay đổi trong một thời kỳ nhất định, hoặc so sánh theo thời kỳ có thể được triển khai.

5.5.2. Ghi quyết định theo thời gian

Khi bắt đầu một dự án, người điều hành dự án và kỹ sư phần mềm quyết định sử dụng các công cụ để lưu trữ quá trình quyết định. Có nghĩa là có thể dùng công cụ điện tử hoặc một phiên bản viết tay và các quyết định được duy trì dưới dạng văn bản. Với công cụ điện tử, các bản sao điện tử được lưu trữ. Với công cụ ghi tay, phiên bản cũ được cập nhật và đổi tên khi một tài liệu thay đổi. Ví dụ như, các quy định chức năng của công ty ABC có thể được đặt tên là ABCFS-mmddyy, trong đó ABC là công ty, FS là viết tắt của quy định chức năng (Functional Specification) và mmddyy là ngày tháng. Phần ngày tháng của tên sẽ thay đổi do bất kỳ một thay đổi quan trọng nào của tài liệu.

5.5.3. Quản lý thay đổi tài liệu

Các thay đổi tài liệu có thể được xác định bởi một bảng nội dung các thay đổi tại đầu mỗi tài liệu. Bảng nội dung các thay đổi bao gồm ngày hiệu lực, các phần bị ảnh hưởng của tài liệu và một tóm tắt về thay đổi. Mục đích của bảng nội dung các thay đổi là để tóm tắt tất cả các thay đổi cho người đọc.

Các thay đổi nên được đánh dấu đỏ trong văn bản để xác định được bộ phận thay đổi. Nếu nội dung cũ là quan trọng thì nó có thể được đưa vào trong chú ý, được ghi ngày tháng, và được dán nhãn là phiên bản trước. Cần nhớ rằng phải giữ tài liệu phiên bản cũ để dùng cho quá trình phát triển.

TỔNG KẾT CUỐI CHƯƠNG

Hoạt động bảo trì là hoạt động nằm ở giai đoạn cuối chu trình phát triển phần mềm, nhưng cũng là giai đoạn chiếm nhiều thời gian và chi phí nhất. Trước khi thực hiện bảo trì phải tiến hành đánh giá, ước lượng khả năng bảo trì. Nếu trong phạm vi cho phép về chi phí cũng như tính khả thi, nên lựa chọn hình thức và công nghệ phù hợp cho việc bảo trì. Trong nhiều trường hợp, việc phát triển tiếp phần mềm dựa trên các bản phân tích cũ mang lại hiệu quả hơn so với việc thay đổi mã nguồn của phần mềm cũ. Trong quá trình bảo trì, cần lưu lại các hoạt động, thay đổi trong phần mềm tạo thành hồ sơ quản lý thay đổi phần mềm để dễ theo giỏi và phục vụ cho các công việc bảo trì tiếp tục trong tương lai.

CÂU HỎI ÔN TẬP CHƯƠNG V

5.1. Phân loại các hoạt động bảo trì? Nên sử dụng loại bảo trì nào?

5.2. Các đặc điểm của việc bảo trì? Đưa ra các giải pháp làm giảm chi phí bảo trì?

5.3. Liệt kê các hoạt động của việc bảo trì?

5.4. Tại sao phải quản lý thay đổi phần mềm? các hoạt động trong quản lý thay đổi phần mềm?

TÀI LIỆU THAM KHẢO

[1]Ian Sommerville. *Software Engineering. Nine Edition*, Addison-Wasley.2011.

[2]Alan Dennies. *Systems Analysis and Design with UML Version 2.0. Second Edition,* Wiley, 2005.

[3] Roger S.Pressman. *Kỹ nghệ phần mềm*, Tập I,II,III. NXB Giáo dục,1997

[4] Nguyễn Tiến Huy, *Giáo trình nhập môn công nghệ phần mềm*, NXB Đại học Khoa Học Tự Nhiên, 2002.